

Local Risk Dynamics - Full Project Bundle

Generated: 2026-03-09T15:50:11.701154

Documentation and Top-Level Files

README.md

Local Risk Dynamics

Deterministic vs Stochastic Locality in Linear Models for Financial Risk

Overview

Financial markets are inherently non-stationary: relationships between risk factors and returns evolve across time, regimes, and uncertainty conditions. Traditional global linear regression assumes constant factor sensitivities, which often leads to model mis-specification in real-world financial environments.

This project studies an alternative paradigm:

Can complex, regime-dependent financial risk behavior be approximated using ensembles of small, local linear models instead of a single global model?

We investigate this through a structured comparison of four linear modeling philosophies that differ in how they define:

- Locality
- Uncertainty

Core Research Idea

All models in this repository lie on a conceptual spectrum:

Stationary → Deterministic Local → Stochastic Global → Stochastic Local

The central hypothesis:

Non-linear financial dynamics can be effectively approximated using collections of locally valid linear models, where locality may be enforced deterministically (distance-based) or stochastically (sampling-based).

Quick Start

Installation

```
# Clone repository
git clone https://github.com/HarshDubey-12/local-risk-dynamics.git
cd local-risk-dynamics
```

```
# Install dependencies
pip install -r requirements.txt
```

Run Experiments

```
# Launch notebook for exploratory data analysis
jupyter notebook notebooks/01_dataset_exploration.ipynb
```

```
# Run baseline global linear model
jupyter notebook notebooks/02_global_linear_baseline.ipynb
```

```
# Run LWLR deterministic locality experiments
jupyter notebook notebooks/03_lwlr_deterministic.ipynb
```

```
# Run Monte Carlo subsampled linear regression
jupyter notebook notebooks/04_mc_subsampled_linear.ipynb
```

```
# Run MCLLR (proposed) stochastic locality model
jupyter notebook notebooks/05_mcllr_proposed.ipynb
```

```
# Compare all model results
jupyter notebook notebooks/06_results_comparison.ipynb
```

Dataset

We use the Fama-French Five-Factor dataset, a canonical benchmark in asset pricing and quantitative finance.

Data location: data/raw/F-F_Research_Data_5_Factors_2x3.csv

Features (risk factors) at time t

- Market excess return (MKT-RF)
- Size factor (SMB)
- Value factor (HML)
- Profitability factor (RMW)
- Investment factor (CMA)

Prediction target

y_{t+1} = next-period market excess return

This formulation ensures:

- True forecasting (no look-ahead bias)
- Risk-adjusted interpretation
- Industry relevance in portfolio and factor modeling

Data Loading Example

```
```python
import pandas as pd
import numpy as np

Load Fama-French data
df = pd.read_csv('data/raw/F-F_Research_Data_5_Factors_2x3.csv', skiprows=3)

Parse date column
df['Date'] = pd.to_datetime(df['Date'].astype(str), format='%Y%m%d')

Convert returns to decimal form (divide by 100)
factors = ['Mkt-RF', 'SMB', 'HML', 'RMW', 'CMA', 'RF']
df[factors] = df[factors] / 100

Prepare features and targets
X = df[['Mkt-RF', 'SMB', 'HML', 'RMW', 'CMA']].values
y = df['Mkt-RF'].shift(-1).dropna().values # next-period market return

print(f"Shape: X={X.shape}, y={y.shape}")
print(f"Date range: {df['Date'].min()} to {df['Date'].max()}")
```
```

Scientific Framing

This work is not merely a finance regression comparison. It is a study of deterministic vs stochastic locality in linear model ensembles for non-stationary environments. While finance provides a realistic testbed, the framework generalizes to energy demand forecasting, dynamic pricing, sensor modeling, and time-varying control systems.

Mathematical Foundation of Locality

From Global Linearity to Stochastic Locality

The repository's models form a precise theoretical ladder:

Global Linear Regression $\rightarrow$ LWLR $\rightarrow$ Monte Carlo LR $\rightarrow$ MCLLR

Each step relaxes one structural assumption:

Model | Relaxes | Limitation Remaining

---|---:|---

Global LR | Stationarity | No locality
 LWLR | Globality | Deterministic locality, no uncertainty
 MC Linear | Determinism | No geometric locality
 MCLLR | – | Stochastic locality

Thus MCLLR is a stochastic generalization of local regression, not merely an ensemble.

Kernel Similarity and Local Geometry

For a query state x_0 :

$$w_i(x_0) = \exp(-||x_i - x_0||^2 / (2 \tau^2))$$

These kernel weights encode geometric similarity and deterministic locality but are not probabilities.

Probability Normalization – The Critical Bridge

To enable sampling we normalize weights to obtain a discrete probability distribution over data indices:

$$p_i(x_0) = w_i(x_0) / \sum_j w_j(x_0)$$

This preserves relative similarity, ensures the probabilities sum to one, and converts deterministic geometry into probabilistic locality.

Stochastic Local Sampling

Fixed-Size Local Neighborhoods (canonical MCLLR)

$$S_k \sim \text{Multinomial}(m, p(x_0))$$

Where m is the stochastic neighborhood size; each sample defines a plausible local regime realization and yields a local regression.

Random-Size Locality (extensions)

Bernoulli inclusion

Poisson sampling

These model uncertainty in regime width, not only membership.

Diversity-Aware and Continuous Extensions

Weighted sampling without replacement

Determinantal point processes

Kernel density / GP-style resampling

The MCLLR Estimator

For r Monte Carlo realizations:

$$y^{(k)}(x_0) = x_0^T \beta_{S_k}$$

Mean prediction:

$$\hat{y}(x_0) = (1/r) \sum_{k=1..r} y^{(k)}(x_0)$$

Predictive variance:

$$\sigma^2(x_0) = (1/r) \sum_k (y^{(k)}(x_0) - \hat{y}(x_0))^2$$

Thus MCLLR computes expectation and variance over stochastic local linear models.

Interpretation of Core Parameters

τ (bandwidth): geometric locality in LWLR

m (subsample size): stochastic locality scale (analogue of bandwidth)

r (MC repetitions): number of stochastic local worlds simulated (controls estimation stability)

Monte Carlo convergence: estimation error $\propto 1/r$.

Regime Interpretation

MCLLR does not explicitly label regimes. Instead it models a distribution over locally valid linear behaviors; regime uncertainty emerges implicitly from probabilistic local membership and stochastic sampling.

Unified Theoretical Statement

Monte Carlo Local Linear Regression estimates predictions as expectations over locally fitted linear models drawn from a kernel-defined probability distribution, modeling non-stationary dynamics through stochastic locality rather than deterministic regime partitioning.

Model Hierarchy & Usage Examples

1) Global Linear Regression – Stationary Baseline

```
```python
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```
```

2) Locally Weighted Linear Regression (LWLR) – Deterministic Locality

```
```python
from src.models.lwlr import LocallyWeightedLinearRegression
```

```
lwlr = LocallyWeightedLinearRegression(bandwidth=1.0)
lwlr.fit(X_train, y_train)
y_pred = lwlr.predict(X_test)
```
```

3) Monte Carlo Subsampled Linear Regression – Stochastic Global Ensemble

```
```python
from src.models.mc_linear import MonteCarloLinearRegression

mc_model = MonteCarloLinearRegression(n_simulations=100, subsample_size=50)
mc_model.fit(X_train, y_train)
y_mean, y_std = mc_model.predict(X_test, return_std=True)
```
```

4) Monte Carlo Local Linear Regression (MCLLR) – Stochastic Locality (Proposed)

```
```python
from src.models.mcllr import MonteCarloLocalLinearRegression

mcllr = MonteCarloLocalLinearRegression(
 bandwidth=1.0,
 n_simulations=50,
 subsample_size=30
)

mcllr.fit(X_train, y_train)
y_pred, y_unc = mcllr.predict(X_test, return_uncertainty=True)
```
```

Monte Carlo Local Linear Regression (MCLLR) – Design Architecture

The following section documents the core design philosophy of MCLLR, defining its structural choices before implementation. These decisions reflect a unified principle: **stochastic locality through kernel-weighted probabilistic sampling, with uncertainty realization via Monte Carlo aggregation**.

Query-Dependent Locality

What: The algorithm executes an outer loop over test points and an inner loop over stochastic simulations. Kernel weights depend on each query point x_0 , making locality query-specific rather than pre-computed globally.

Why: In time-series financial data, different regimes have different local geometries. A fixed global weighting scheme cannot capture this. By recomputing weights for each prediction point, the model adapts its notion of "nearby" observations to the specific context of the query.

Role: This design ensures that MCLLR produces **personalized neighborhoods** for each x_0 , enforcing that stochastic samples are drawn from geometrically relevant regions of the feature space. The outer loop structure is not a computational artifact but a deliberate design choice encoding query dependence.

Squared Euclidean Distance (No Square Root)

What: Kernel weight computation uses squared Euclidean distance: $||x_i - x_0||^2$, not $||x_i - x_0||$.

Why: Avoiding the square root operation achieves two objectives: (1) numerical stability in floating-point arithmetic when distances are very small, preventing underflow and precision loss; (2) computational efficiency during large-scale prediction, eliminating expensive square root evaluations for every distance pair.

Role: This choice preserves monotonic distance ordering (the ranking of nearby points remains unchanged) while improving numerical robustness. It is standard in kernel methods and aligns MCLLR with LWLR implementation practices in the literature.

****Gaussian Kernel for Weight Computation****

What: Weights are computed via the Gaussian (RBF) kernel:

$$w_i(x_0) = \exp(-||x_i - x_0||^2 / (2\tau^2))$$

Why: The Gaussian kernel provides smooth, continuous locality: points near x_0 receive high weight, distant points receive exponentially decaying weight. This is theoretically justified for regression neighborhoods because it enforces smooth transitions between local regimes rather than sharp cutoffs (which would create discontinuities in predictions).

Role: This serves as the ****geometric locality function****. The bandwidth parameter τ controls the spatial extent of locality. The kernel is deterministic and problem-independent; its output is treated as raw similarity, not probability.

****Normalization of Weights Only Before Sampling****

What: Weights are normalized to probabilities exclusively to enable sampling:

$$p_i = w_i / \sum_j w_j$$

Normalization does ****not**** occur before regression; it occurs only to define the probability distribution for drawing the stochastic neighborhood.

Why: Normalization serves one purpose: to ensure sampling probabilities sum to one. Inside each simulation, OLS regression uses original weights only if explicitly weighting is demanded; however, in MCLLR's core design, the key insight is that normalization is probabilistic, not geometric. This separation prevents double-counting locality.

Role: This normalization is the ****critical bridge**** between deterministic geometry and stochastic locality. It converts kernel similarities (unnormalized) into a discrete probability measure over observations, enabling principled sampling.

****Weighted Random Sampling Without Replacement****

What: Given probabilities p_i derived from normalized weights, MCLLR draws a fixed-size sample S_k of size m without replacement from the training set, where each observation has probability proportional to p_i .

Why: Sampling without replacement is preferred over multinomial sampling (with replacement) because: (1) it preserves observation ****uniqueness**** within a neighborhood, enforcing that each local regression uses distinct observations; (2) it avoids redundancy in the local feature space, maintaining geometric diversity; (3) it models a concrete "plausible local world" rather than abstract reweighting.

Role: This mechanism ****realizes stochasticity**** in locality selection. Each simulation k draws a different local subset, inducing variation in local linear models and ultimately in predictions. The randomness captures uncertainty in which observations best characterize the local regime.

****Plain OLS Inside Each Simulation****

What: Within each simulation, the selected subsample S_k is fit using ordinary least squares (OLS)—ordinary, unweighted regression—not weighted regression.

Why: Locality has already been enforced via the sampling step: by construction, S_k contains observations with high kernel weight relative to x_0 . Applying weighted regression again within the subsample would ****double-count locality****, giving disproportionate influence to nearby points even among nearby points. OLS inside the sampled neighborhood is sufficient and theoretically correct.

Role:** This choice ensures *clean separation of concerns****: sampling enforces geometric locality; OLS fits the local surface. The combination avoids compound locality effects that would distort local linear approximation.

****Monte Carlo Aggregation (Mean and Standard Deviation)****

***What:** After fitting r independent local linear models via sampling and OLS, predictions aggregate as:

$$\hat{y}(x_0) = (1/r) \sum_{k=1}^r \hat{y}^{\{k\}}(x_0)$$

$$\sigma^2(x_0) = (1/r) \sum_{k=1}^r (\hat{y}^{\{k\}}(x_0) - \hat{y}(x_0))^2$$

***Why:** Monte Carlo integration produces both point estimates (mean) and uncertainty quantification (variance). The standard deviation reflects sampling variability across stochastic local models, encoding our epistemic uncertainty about which local regime applies to x_0 . This is interpretable as a predictive distribution.

Role:** This aggregation is the *uncertainty mechanism**** of MCLLR. Variance does not reflect training noise (as in classical frequentist regression); it reflects regime uncertainty—the ensemble spread of plausible local linear behaviors. This is essential for risk-aware decision-making in finance.

****No Model-Switching Logic****

***What:** MCLLR does not contain hidden conditional logic that changes model behavior based on hyperparameters. Hyperparameters τ (bandwidth) and m (subsample size) directly scale locality geometry and stochastic neighborhood size, but do not trigger regime switches or algorithmic branches.

Why:** This maintains interpretability and ensures *smooth theoretical degeneracy****. MCLLR must naturally reduce to simpler models as follows:

- Large τ (broad bandwidth) $\rightarrow$ all observations weighted nearly equally $\rightarrow$ sampling is nearly uniform $\rightarrow$ MCLLR $\rightarrow$ MC Linear Regression
- Large m (subsample size) $\rightarrow S_k$ approaches the full training set $\rightarrow$ local models become global models $\rightarrow$ MCLLR $\rightarrow$ LWLR
- Both large τ and large m $\rightarrow$ all observations uniformly sampled in large subsets $\rightarrow$ MCLLR $\rightarrow$ Global Linear Regression

These limiting behaviors must emerge from the same code path, not from explicit conditional branches. This guarantees theoretical consistency and prevents ad-hoc model selection.

Role:** This design ensures MCLLR is a *unified, parameter-continuous framework**** encompassing global and local approaches. It validates that MCLLR is a generalization, not a disjoint ensemble method.

****Epsilon-Based Numerical Stability Safeguard****

***What:** When computing normalized weights $p_i = w_i / (\sum_j w_j)$, if the sum $\sum_j w_j$ is numerically very small (below machine epsilon), a small constant ϵ is added to prevent division by zero or catastrophic precision loss.

***Why:** In rare edge cases—such as prediction far from the training support where all weights become vanishingly small—floating-point computation may produce NaN or Inf. The epsilon safeguard ensures numerical stability without altering the model's structural logic or changing weights materially.

Role:** This is a *robustness safeguard**** that keeps the algorithm running correctly in numerical boundary conditions without compromising design philosophy. It is not a modeling choice but an engineering necessity.

****Clear Separation of Responsibilities****

The MCLLR architecture enforces a clean functional decomposition:

| Component | Function | Responsibility |
|---------------|--|---|
| Kernel | $w_i(x_0) = \exp(-\ x_i - x_0\ ^2 / (2\tau^2))$ | Geometric locality; deterministic similarity |
| Normalization | $p_i = w_i / \sum_j w_j$ | Probability assignment; sampling distribution |
| Sampling | $S_k \sim p_i$ without replacement | Stochastic neighborhood realization |
| OLS | $\beta_k = (S_k^T S_k)^{-1} S_k^T y_k$ | Local surface fitting within neighborhood |
| Aggregation | $\hat{y} = \text{mean}, \sigma^2 = \text{var over } k$ | Uncertainty quantification; ensemble prediction |

This separation ensures each component has a single, well-defined purpose. Changes to one (e.g., kernel choice) do not cascade through the pipeline; design modifications remain localized and testable.

****Conceptual Summary****

Monte Carlo Local Linear Regression is a stochastic realization of local linear approximations whose uncertainty emerges from probabilistic neighborhood selection. By decoupling deterministic geometric locality (kernels) from stochastic membership (sampling), MCLLR captures both regime-dependent non-linearity and estimation uncertainty in a unified, theoretically coherent framework.

Experimental Goals

- Prediction accuracy across regimes
- Stability vs variance trade-offs
- Uncertainty calibration
- Robustness to non-stationarity

Validation Roadmap (Implemented)

The following checklist converts the research hypothesis into code-level validation steps.

- [x] Rolling non-stationary evaluation
- [x] Time-series hyperparameter tuning
- [x] MCLLR ablation analysis
- [x] Uncertainty quality metrics
- [x] Stronger regularized baseline

1) Rolling non-stationary evaluation

Theory

- A single fixed train/test split can hide regime sensitivity.
- Walk-forward folds test whether performance remains stable as regimes shift.

Programming

- Module: ``src/evaluation/rolling.py``
- Key APIs:
 - ``walk_forward_splits(...)``
 - ``evaluate_walk_forward(...)``
 - ``rolling_results_to_frame(...)``
 - ``summarize_rolling_metrics(...)``

Example

```
```python
from src.evaluation.rolling import evaluate_walk_forward, rolling_results_to_frame

folds = evaluate_walk_forward(
 model_builder=lambda: model,
 X=X,
 y=y,
 train_window=240,
 test_window=24,
 step=24,
)
df_folds = rolling_results_to_frame(folds)
```
```

2) Time-series hyperparameter tuning

Theory

- Hyperparameters (``tau``, ``m``, ``r``) must be selected using chronological validation, not random splits, to avoid leakage and over-optimistic selection.

Programming

- Module: ``src/Experiments/tuning.py``
- API: ``tune_model_with_walk_forward(...)``

Example

```
```python
from src.config import ModelCofig
from src.Experiments.tuning import tune_model_with_walk_forward
```

```

leaderboard, best = tune_model_with_walk_forward(
 base_model_config=ModelCofig(
 model_type="mcllr",
 bandwidth=1.0,
 n_simulations=50,
 subsample_size=80,
 fit_intercept=True,
 random_state=42,
),
 param_grid={
 "bandwidth": [0.5, 1.0, 2.0],
 "subsample_size": [50, 80, 120],
 "n_simulations": [30, 50],
 },
 X=X,
 y=y,
 train_window=240,
 test_window=24,
 objective_metric="rmse",
)
...

```

### 3) MCLLR ablation analysis

#### Theory

- To validate the claim "geometric + stochastic locality is necessary", each component must be removed and compared.

#### Programming

- Models added:
  - `mcllr\_no\_geometry` (stochastic sampling only)
  - `mcllr\_no\_stochasticity` (deterministic local subset only)
- File: `src/models/mcllr\_ablations.py`
- Factory support: `src/models/factory.py`

#### Interpretation

- If full `mcllr` consistently outperforms both ablations in rolling tests, the core thesis is empirically supported.

### 4) Uncertainty quality metrics

#### Theory

- Uncertainty-aware modeling is not only "having a std output".
- It must be calibrated and decision-useful.

#### Programming

- Added metrics in `src/evaluation/metrics.py`:
  - `coverage\_95`
  - `interval\_width\_95`
  - `gaussian\_nll`
  - `interval\_score\_95`
- Integrated into `src/evaluation/evaluator.py` for all models that return uncertainty.

### 5) Stronger regularized baseline

#### Theory

- A stronger linear baseline prevents over-claiming MCLLR gains over weak comparators.
- Ridge is a robust baseline under multicollinearity and noisy factors.

#### Programming

- Model: `ridge\_linear`
- File: `src/models/ridge\_linear.py`
- Config support in `src/config.py` and `src/models/factory.py`

### 6) End-to-end rolling experiment runner

#### Programming

- Module: `src/Experiments/rolling\_runner.py`
- Runs all models from one config over walk-forward folds.

```

Example
```python
from src.Experiments.rolling_runner import RollingExperimentRunner

runner = RollingExperimentRunner(
    "experiments/config_comparison.yaml",
    train_window=240,
    test_window=24,
    step=24,
)
fold_df, summary_df = runner.run()
```

```

## 7) Unified experiment config for claim testing

### Programming

- File: `experiments/config\_comparison.yaml`
- Includes:
  - `global\_linear`
  - `ridge\_linear`
  - `lwlr`
  - `mc\_linear`
  - `mcllr`
  - `mcllr\_no\_geometry`
  - `mcllr\_no\_stochasticity`

### Practical criterion for claim acceptance

- Full MCLLR should:
  - win on rolling `rmse/mae` mean across folds,
  - maintain competitive variance (`std`) across folds,
  - show credible uncertainty quality (`coverage\_95`, lower interval score/NLL),
  - outperform both ablations consistently.

### Project Structure

```

```
local-risk-dynamics/
  data/
    raw/           # Original Fama-French data
    processed/     # Preprocessed datasets
  notebooks/
    01_dataset_exploration.ipynb
    02_global_linear_baseline.ipynb
    03_lwlr_deterministic.ipynb
    04_mc_subsampled_linear.ipynb
    05_mcllr_proposed.ipynb
    06_results_comparison.ipynb
  src/
    data/
      loader.py
      preprocessing.py
    models/
      global_linear.py
      lwlr.py
      mc_linear.py
      mcllr.py
    evaluation/
      metrics.py
      backtesting.py
    utils/
      kernels.py
      sampling.py
      plotting.py
  experiments/
    config_global.yaml
    config_lwlr.yaml
    config_mc.yaml

```

```

    config_mcllr.yaml
    figures/
    README.md
    requirements.txt
    ...

```

Repository Philosophy

Separates theory (README), engineering (src/), and evidence (notebooks/figures), mirroring quantitative research repositories and ML paper implementations.

Project Status

Current stage: complete theoretical foundation + structured research scaffold.

Next milestones:

- Full MCLLR implementation
- Empirical comparison
- Industry implications

Contributing

```

```bash
Create a feature branch
git checkout -b feature/your-feature-name

Commit with meaningful messages
git commit -m "add: descriptive commit message"

Push and open a pull request
git push origin feature/your-feature-name
```

```

License

This project is licensed under the MIT License. See LICENSE file for details.

Key Takeaway

Financial markets are locally linear but globally non-linear. Understanding risk may therefore require many small linear views of reality—combined through deterministic structure, stochastic sampling, or both.

If you'd like, I can next produce a NeurIPS/ICLR-style paper draft, a formal mathematical appendix, or implement the full MCLLR algorithm and notebooks.

requirements.txt

```

# Python requirements
pandas
numpy
matplotlib
jupyter
pytest

```

test_mcllr.py

```

#!/usr/bin/env python
"""
Unit tests for MCLLR project using pytest.
Tests models, metrics, kernels, and sampling utilities.
"""

import numpy as np
import pytest
import pandas as pd
import matplotlib
matplotlib.use('Agg') # Use non-GUI backend for testing

```

```

from src.data.loader import load_fama_french
from src.data.preprocessing import time_series_split
from src.models.global_linear import GlobalLinearRegression
from src.models.lwlr import LocallyWeightedLinearRegression
from src.models.mc_linear import MonteCarloLinearRegression
from src.models.mcllr import MonteCarloLocalLinearRegression
from src.evaluation.metrics import rmse, mae, mape, r2, explained_variance, median_ae, coverage_95, avg_std
from src.utils.kernels import GaussianKernel, EpanechnikovKernel
from src.utils.sampling import subsample_indices
from src.utils.plotting import (
    plot_predictions_vs_true, plot_uncertainty, plot_kernel_weights,
    plot_model_comparison, plot_time_series, plot_residuals
)
from src.evaluation.backtesting import sharpe_ratio, max_drawdown, volatility, sortino_ratio, calmar_ratio,
backtest_summary

```

```
@pytest.fixture
```

```

def sample_data():
    """Load and split sample data for tests."""
    X, y, dates = load_fama_french('data/raw/F-F_Research_Data_5_Factors_2x3.csv')
    X_train, X_test, y_train, y_test, _, _ = time_series_split(X, y, dates, 0.8)
    return X_train, X_test, y_train, y_test

```

```
class TestDataLoading:
```

```

    def test_load_fama_french(self):
        X, y, dates = load_fama_french('data/raw/F-F_Research_Data_5_Factors_2x3.csv')
        assert X.shape[0] == y.shape[0] == dates.shape[0]
        assert X.shape[1] == 5 # 5 factors
        assert isinstance(dates[0], (np.datetime64, pd.Timestamp))

    def test_time_series_split(self, sample_data):
        X_train, X_test, y_train, y_test = sample_data
        assert X_train.shape[0] + X_test.shape[0] == X_train.shape[0] + X_test.shape[0] # total rows
        assert X_train.shape[1] == X_test.shape[1] == 5
        assert y_train.shape[0] == X_train.shape[0]
        assert y_test.shape[0] == X_test.shape[0]

```

```
class TestModels:
```

```

    def test_global_linear(self, sample_data):
        X_train, X_test, y_train, y_test = sample_data
        model = GlobalLinearRegression()
        model.fit(X_train, y_train)
        # interface
        assert model.is_fitted
        assert isinstance(model.model_name, str)
        assert isinstance(model.hyperparameters, dict)

        y_pred = model.predict(X_test)
        assert y_pred.shape == y_test.shape
        assert model.score(X_test, y_test) > 0 # basic sanity
        # uncertainty api
        y_pred2, y_std = model.predict_with_uncertainty(X_test[:5])
        assert y_pred2.shape == (5,)
        assert y_std is None

    def test_lwlr(self, sample_data):
        X_train, X_test, y_train, y_test = sample_data
        model = LocallyWeightedLinearRegression(bandwidth=1.0)
        model.fit(X_train, y_train)
        assert model.is_fitted
        assert isinstance(model.model_name, str)
        assert isinstance(model.hyperparameters, dict)

```

```

y_pred = model.predict(X_test[:10]) # test subset
assert y_pred.shape == (10,)
assert model.score(X_test[:10], y_test[:10]) > 0
y_pred2, y_std = model.predict_with_uncertainty(X_test[:5])
assert y_pred2.shape == (5,)
assert y_std is None

def test_mc_linear(self, sample_data):
    X_train, X_test, y_train, y_test = sample_data
    model = MonteCarloLinearRegression(n_simulations=5, subsample_size=20, random_state=42)
    model.fit(X_train, y_train)
    assert model.is_fitted
    assert isinstance(model.model_name, str)
    assert isinstance(model.hyperparameters, dict)

    y_pred, y_std = model.predict(X_test[:5], return_std=True)
    assert y_pred.shape == (5,)
    assert y_std.shape == (5,)
    assert np.all(y_std >= 0)
    # stochastic predictions change each call; just verify shape
    y_pred2, y_std2 = model.predict_with_uncertainty(X_test[:5])
    assert y_pred2.shape == (5,)
    assert y_std2.shape == (5,)

def test_mcllr(self, sample_data):
    X_train, X_test, y_train, y_test = sample_data
    model = MonteCarloLocalLinearRegression(bandwidth=1.0, n_simulations=5, subsample_size=20, random_state=42)
    model.fit(X_train, y_train)
    assert model.is_fitted
    assert isinstance(model.model_name, str)
    assert isinstance(model.hyperparameters, dict)

    y_pred, y_std = model.predict(X_test[:5], return_std=True)
    assert y_pred.shape == (5,)
    assert y_std.shape == (5,)
    assert np.all(y_std >= 0)
    assert model.score(X_test[:5], y_test[:5]) > 0
    # results vary due to randomness
    y_pred2, y_std2 = model.predict_with_uncertainty(X_test[:5])
    assert y_pred2.shape == (5,)
    assert y_std2.shape == (5,)

class TestMetrics:
    def test_rmse(self):
        y_true = np.array([1, 2, 3])
        y_pred = np.array([1.1, 2.1, 2.9])
        assert rmse(y_true, y_pred) > 0
        assert rmse(y_true, y_true) == 0

    def test_mae(self):
        y_true = np.array([1, 2, 3])
        y_pred = np.array([1.1, 2.1, 2.9])
        assert mae(y_true, y_pred) > 0
        assert mae(y_true, y_true) == 0

    def test_r2(self):
        y_true = np.array([1, 2, 3, 4, 5])
        y_pred = np.array([1, 2, 3, 4, 5])
        assert r2(y_true, y_pred) == 1.0

    def test_coverage_95(self):
        y_true = np.array([1, 2, 3])
        y_pred = np.array([1, 2, 3])
        y_std = np.array([0.1, 0.1, 0.1])
        coverage = coverage_95(y_true, y_pred, y_std)

```

```

    assert 0 <= coverage <= 1

class TestKernels:
    def test_gaussian_kernel(self):
        kernel = GaussianKernel(bandwidth=1.0)
        X = np.array([[0], [1], [2]])
        x0 = np.array([0])
        weights = kernel(X, x0)
        assert weights.shape == (3,)
        assert weights[0] == 1.0 # at zero distance
        assert np.all(weights >= 0)

    def test_epanechnikov_kernel(self):
        kernel = EpanechnikovKernel(bandwidth=1.0)
        X = np.array([[0], [0.5], [1.5]])
        x0 = np.array([0])
        weights = kernel(X, x0)
        assert weights.shape == (3,)
        assert weights[0] == 0.75 # Epanechnikov at 0 is 3/4
        assert weights[2] == 0.0 # distance > bandwidth

class TestSampling:
    def test_subsample_indices(self):
        indices = subsample_indices(100, 10, random_state=42)
        assert len(indices) == 10
        assert all(0 <= i < 100 for i in indices)
        assert len(set(indices)) == 10 # unique

    def test_subsample_indices_with_weights(self):
        weights = np.ones(100) / 100
        indices = subsample_indices(100, 10, weights=weights, random_state=42)
        assert len(indices) == 10
        assert len(set(indices)) == 10

class TestPlotting:
    def test_plot_predictions_vs_true(self):
        y_true = np.array([1, 2, 3, 4, 5])
        y_pred = np.array([1.1, 2.1, 2.9, 4.1, 4.9])
        plot_predictions_vs_true(y_true, y_pred) # Test no error

    def test_plot_uncertainty(self):
        y_pred = np.array([1, 2, 3])
        y_std = np.array([0.1, 0.2, 0.1])
        plot_uncertainty(y_pred, y_std)

    def test_plot_kernel_weights(self):
        distances = np.array([0, 1, 2, 3])
        weights = np.array([1.0, 0.5, 0.1, 0.0])
        plot_kernel_weights(distances, weights, 'Test Kernel')

    def test_plot_model_comparison(self):
        # Mock EvaluationResult objects
        class MockResult:
            def __init__(self, r2_val):
                self.metrics = {'r2': r2_val}

        results = {
            'Model1': MockResult(0.8),
            'Model2': MockResult(0.9),
            'Model3': MockResult(0.7)
        }
        plot_model_comparison(results, 'r2')

```

```
def test_plot_time_series(self):
    y = np.random.randn(100).cumsum()
    plot_time_series(y, title='Test Time Series')

def test_plot_residuals(self):
    y_true = np.array([1, 2, 3, 4, 5])
    y_pred = np.array([1.1, 2.1, 2.9, 4.1, 4.9])
    plot_residuals(y_true, y_pred)

class TestBacktesting:
    def test_sharpe_ratio(self):
        returns = np.array([0.01, 0.02, -0.01, 0.03])
        assert sharpe_ratio(returns, annualize=False) > 0

    def test_max_drawdown(self):
        returns = np.array([0.1, -0.2, 0.05, -0.15])
        md = max_drawdown(returns)
        assert md < 0 # drawdown is negative

    def test_volatility(self):
        returns = np.array([0.01, 0.02, -0.01])
        vol = volatility(returns, annualize=False)
        assert vol > 0

    def test_backtest_summary(self):
        returns = np.array([0.01] * 300) # 300 days
        summary = backtest_summary(returns)
        assert 'sharpe_ratio' in summary
        assert summary['total_return'] > 0
```

Source, Config, Script, and Output Files

src\config.py

```

from dataclasses import dataclass, asdict
from typing import Literal, Dict, Any
import yaml
from pathlib import Path

@dataclass
class ModelCofig:
    """Configuration for any model in the framework."""
    model_type: Literal[
        "global_linear",
        "lwlr",
        "mc_linear",
        "mcllr",
        "mcllr_no_geometry",
        "mcllr_no_stochasticity",
        "ridge_linear",
    ]
    fit_intercept: bool = True
    random_state: int | None = None
    # Model specific params
    bandwidth: float | None = None # LWLR, MCLLR
    n_simulations: int | None = None # MC, MCLLR
    subsample_size: int | None = None # MC, MCLLR
    alpha: float | None = None # Ridge baseline

    def __post_init__(self):
        # validate based on model_type
        if self.model_type == "lwlr" and self.bandwidth is None:
            raise ValueError("Bandwidth is required for LWLR")
        if self.model_type in ["mc_linear", "mcllr", "mcllr_no_geometry"] and self.n_simulations is None:
            raise ValueError(f"n_simulations is required for {self.model_type}")
        if self.model_type in ["mc_linear", "mcllr", "mcllr_no_geometry", "mcllr_no_stochasticity"] and self.subsample_size is None:
            raise ValueError(f"subsample_size is required for {self.model_type}")

    @classmethod
    def from_yaml(cls, path: str | Path) -> "ModelCofig":
        with open(path) as f:
            config_dict = yaml.safe_load(f)
            return cls(**config_dict)

    def to_yaml(self, path: str | Path):
        with open(path, 'w') as f:
            yaml.dump(asdict(self), f)

@dataclass
class ExperimentConfig:
    """Experiment-level settings."""
    dataset_path: str
    train_ratio: float
    models: list[ModelCofig]
    random_state: int = 42

    @classmethod
    def from_yaml(cls, path: str | Path) -> "ExperimentConfig":
        with open(path) as f:
            config_dict = yaml.safe_load(f)
            config_dict["models"] = [

```

```

        ModelCofig(**m) for m in config_dict["models"]
    ]
    return cls(**config_dict)

```

src\data\loader.py

```

"""
Data loader utilities

This module provides dataset loading functions for the research project.

TODO:
- Implement `load_fama_french()` to read `data/raw/F-F_Research_Data_5_Factors_2x3.csv`.
- Add type hints and tests.
"""

# placeholder: implementations should live here
from __future__ import annotations

from pathlib import Path
from typing import Tuple

import numpy as np
import pandas as pd

FACTOR_COLUMNS = ["Mkt-RF", "SMB", "HML", "RMW", "CMA"]
ALL_RETURN_COLUMNS = FACTOR_COLUMNS + ["RF"]

def load_fama_french(
    csv_path: str | Path,
) -> Tuple[np.ndarray, np.ndarray, pd.DatetimeIndex]:
    """
    Load and preprocess the Fama-French Five-Factor dataset.

    Parameters
    -----
    csv_path : str or Path
        Path to the raw Fama-French CSV file.

    Returns
    -----
    X : np.ndarray of shape (n_samples, 5)
        Feature matrix containing factor returns at time t.

    y : np.ndarray of shape (n_samples,)
        Target vector containing next-period market excess return (t+1).

    dates : pd.DatetimeIndex
        Dates aligned with X and y after preprocessing.

    Notes
    -----
    - Returns are converted from percentages to decimals.
    - Target is shifted by one period to ensure true forecasting.
    - Final row is dropped due to NaN after shifting.
    """

    csv_path = Path(csv_path)

    if not csv_path.exists():
        raise FileNotFoundError(f"Fama-French file not found: {csv_path}")

    # Load CSV (skip metadata lines 0-3, header is on line 4)
    # use skipinitialspace to trim leading spaces in numeric columns

```

```

df = pd.read_csv(csv_path, skiprows=4, skipinitialspace=True)

# Rename first column to 'Date' if unnamed
if "Date" not in df.columns:
    df = df.rename(columns = {df.columns[0]: "Date"})

# Drop any extraneous metadata rows that sneaked in (e.g. repeated headers or notes)
# Keep only entries where the Date column looks like YYYYMM digits
df = df[df["Date"].astype(str).str.match(r"^\d{6}$")]

# Parse dates (YYYYMM format)
df["Date"] = pd.to_datetime(df["Date"].astype(str), format="%Y%m")

# Convert percentage returns -> decimals (ensure numeric type first)
df[ALL_RETURN_COLUMNS] = df[ALL_RETURN_COLUMNS].apply(pd.to_numeric, errors='coerce') / 100.0

# Feature matrix at time t
X = df[FACTOR_COLUMNS].to_numpy(dtype = float)

# Target: next-period market excess return
y = df["Mkt-RF"].shift(-1).to_numpy(dtype = float)

# Remove last row (NaN after shift)
valid_mask = ~np.isnan(y)
X = X[valid_mask]
y = y[valid_mask]
dates = df.loc[valid_mask, "Date"]

return X, y, pd.DatetimeIndex(dates)

```

src\data\pipeline.py

```

from pathlib import Path
from dataclasses import dataclass
import hashlib
import numpy as np
import pandas as pd
from .loader import load_fama_french

@dataclass
class Dataset:
    """Immutable dataset container with metadata."""
    X: np.ndarray
    y: np.ndarray
    dates: pd.DatetimeIndex
    feature_names: list
    split_metadata: dict # tracks train/test split info.
    source_hash: str # Has of raw data for reproducibility.

    def __post_init__(self):
        assert len(self) > 0
        assert self.X.shape[0] == len(self.y) == len(self.dates)

    def __len__(self):
        return len(self.y)

    def describe(self) -> dict:
        return{
            "n_samples": len(self),
            "n_features": self.X.shape[1],
            "features": self.feature_names,
            "date_range": f"{self.dates[0]}to{self.dates[-1]}",
            "source_hash": self.source_hash,
        }

```

```

class DataPipeline:
    """Unified data handling with caching and versioning."""

    def __init__(self, cache_dir: Path = None):
        self.cache_dir = cache_dir or Path("data/cache")
        self.cache_dir.mkdir(exist_ok=True)

    def load_fama_french(self, path: str | Path) -> Dataset:
        """Load with hash-based caching."""
        path = Path(path)
        source_hash = self._get_file_hash(path)

        # Use the canonical loader to produce consistent features/targets.
        X, y, dates = load_fama_french(path)
        feature_names = ["Mkt-RF", "SMB", "HML", "RMW", "CMA"]
        dataset = Dataset(
            X=X,
            y=y,
            dates=dates,
            feature_names=feature_names,
            split_metadata={"raw_source": str(path)},
            source_hash=source_hash,
        )
        return dataset

    def _get_file_hash(self, path: Path) -> str:
        with open(path, 'rb') as f:
            return hashlib.md5(f.read()).hexdigest()[:8]

```

src\data\preprocessing.py

```

"""
Data preprocessing helpers

Place common preprocessing routines here (scaling, feature engineering, etc.).

TODO:
- Implement preprocessing pipeline used by notebooks.
- Add tests and docs.
"""

# placeholder for preprocessing utilities
from __future__ import annotations

from typing import Tuple

import numpy as np
import pandas as pd

def time_series_split(
    X: np.ndarray,
    y: np.ndarray,
    dates: pd.DatetimeIndex,
    train_ratio: float = 0.8,
) -> Tuple[
    np.ndarray,
    np.ndarray,
    np.ndarray,
    np.ndarray,
    pd.DatetimeIndex,
    pd.DatetimeIndex,
]:

```

```

"""
Perform a chronological train/test split for time-series data.

Parameters
-----
X : np.ndarray of shape (n_samples, n_features)
    Feature matrix ordered in time.

y : np.ndarray of shape (n_samples,)
    Target vector aligned with X.

dates : pd.DatetimeIndex of length n_samples
    Timestamps aligned with X and y.

train_ratio : float, default=0.8
    Fraction of data used for training.
    Must satisfy 0 < train_ratio < 1.

Returns
-----
X_train, X_test : np.ndarray
    Chronologically split feature matrices.

y_train, y_test : np.ndarray
    Chronologically split targets.

dates_train, dates_test : pd.DatetimeIndex
    Corresponding timestamps for each split.

Raises
-----
ValueError
    If input lengths mismatch or train_ratio is invalid.

Notes
-----
- No shuffling is performed.
- Prevents look-ahead bias in financial forecasting.
- Deterministic and reproducible.
"""

# -----Validation-----
n_samples = len(X)

if not (len(y) == n_samples == len(dates)):
    raise ValueError("X, y, and dates must have the same length.")

if not (0.0 < train_ratio < 1.0):
    raise ValueError("train_ratio must be between 0 and 1(exclusive).")

# -----Split index-----
split_idx = int(np.floor(n_samples*train_ratio))

if split_idx == 0 or split_idx == n_samples:
    raise ValueError("train_ratio results in empty or test split.")

# -----Chronological split-----
X_train, X_test = X[:split_idx], X[split_idx:]
y_train, y_test = y[:split_idx], y[split_idx:]
dates_train, dates_test = dates[:split_idx], dates[split_idx:]

return X_train, X_test, y_train, y_test, dates_train, dates_test

```

```

"""
Backtesting helpers

Functions for financial performance evaluation and backtesting.
Includes risk-adjusted metrics and rolling-window analysis.
"""

import numpy as np
from typing import Optional, Tuple

def sharpe_ratio(returns: np.ndarray, risk_free_rate: float = 0.0, annualize: bool = True) -> float:
    """Sharpe ratio: (mean return - rf) / std return."""
    excess_returns = returns - risk_free_rate
    mean_excess = np.mean(excess_returns)
    std_excess = np.std(excess_returns, ddof=1)
    if std_excess == 0:
        return np.nan
    ratio = mean_excess / std_excess
    return ratio * np.sqrt(252) if annualize else ratio # assuming daily returns

def max_drawdown(returns: np.ndarray) -> float:
    """Maximum drawdown: largest peak-to-trough decline."""
    cumulative = np.cumprod(1 + returns)
    running_max = np.maximum.accumulate(cumulative)
    drawdowns = (cumulative - running_max) / running_max
    return float(np.min(drawdowns))

def volatility(returns: np.ndarray, annualize: bool = True) -> float:
    """Volatility: standard deviation of returns."""
    vol = np.std(returns, ddof=1)
    return vol * np.sqrt(252) if annualize else vol

def sortino_ratio(returns: np.ndarray, risk_free_rate: float = 0.0, annualize: bool = True) -> float:
    """Sortino ratio: (mean return - rf) / downside std."""
    excess_returns = returns - risk_free_rate
    downside = excess_returns[excess_returns < 0]
    if len(downside) == 0:
        return np.nan
    mean_excess = np.mean(excess_returns)
    downside_std = np.std(downside, ddof=1)
    ratio = mean_excess / downside_std
    return ratio * np.sqrt(252) if annualize else ratio

def calmar_ratio(returns: np.ndarray, risk_free_rate: float = 0.0) -> float:
    """Calmar ratio: annualized return / max drawdown."""
    if len(returns) < 252:
        # Not enough data for annualization
        total_return = np.prod(1 + returns) - 1
        md = max_drawdown(returns)
        return total_return / abs(md) if md != 0 else np.nan
    else:
        ann_return = np.prod(1 + returns)**(252 / len(returns)) - 1
        md = max_drawdown(returns)
        return ann_return / abs(md) if md != 0 else np.nan

def rolling_backtest(
    y_true: np.ndarray,
    y_pred: np.ndarray,
    window_size: int = 252,

```

```

    step: int = 1
) -> Tuple[np.ndarray, np.ndarray]:
    """
    Rolling window backtest: compute metrics over sliding windows.

    Returns arrays of Sharpe ratios and max drawdowns for each window.
    Assumes y_true and y_pred are returns or errors.
    """
    sharpes = []
    mdds = []
    for i in range(0, len(y_true) - window_size + 1, step):
        window_true = y_true[i:i+window_size]
        window_pred = y_pred[i:i+window_size]
        # Simple strategy: assume returns based on prediction accuracy
        # For simplicity, use absolute error as "loss"
        errors = np.abs(window_true - window_pred)
        # Convert to pseudo-returns (negative for errors)
        pseudo_returns = -errors # or some other logic
        sharpes.append(sharpe_ratio(pseudo_returns))
        mdds.append(max_drawdown(pseudo_returns))
    return np.array(sharpes), np.array(mdds)

def backtest_summary(
    returns: np.ndarray,
    risk_free_rate: float = 0.0
) -> dict:
    """Compute comprehensive backtest summary."""
    return {
        'total_return': float(np.prod(1 + returns) - 1),
        'annualized_return': float(np.prod(1 + returns)**(252 / len(returns)) - 1) if len(returns) >= 252 else
np.nan,
        'volatility': volatility(returns),
        'sharpe_ratio': sharpe_ratio(returns, risk_free_rate),
        'max_drawdown': max_drawdown(returns),
        'sortino_ratio': sortino_ratio(returns, risk_free_rate),
        'calmar_ratio': calmar_ratio(returns, risk_free_rate),
    }

```

src\evaluation\evaluator.py

```

from dataclasses import dataclass
from typing import List, Dict, Any, Callable, Optional
import numpy as np
from ..evaluation import metrics as metrics_module
from ..evaluation import backtesting as backtesting_module
from ..models.base import LocalRiskModel
from ..data.pipeline import Dataset

@dataclass
class EvaluationResult:
    """Unified evaluation output."""
    model_name: str
    metrics: Dict[str, float]
    predictions: np.ndarray
    uncertainties: np.ndarray | None = None
    ground_truth: np.ndarray = None

    def __repr__(self):
        metric_str = ", ".join(f"{k}: {v:.4f}" for k, v in self.metrics.items())
        return f"<EvaluationResult {self.model_name} | {metric_str}>"

class Evaluator:
    """Unified evaluation pipeline matching assumptions."""

```

```

@staticmethod
def evaluate(
    model: LocalRiskModel,
    X_test: np.ndarray,
    y_test: np.ndarray,
    extra_metrics: Optional[Dict[str, Callable[[np.ndarray, np.ndarray], float]]] = None,
) -> EvaluationResult:
    """Evaluate a model on the test set and return a full result object.

    Parameters
    -----
    model : LocalRiskModel
        Fitted model implementing ``predict_with_uncertainty``.
    X_test : np.ndarray
        Feature matrix for testing.
    y_test : np.ndarray
        Ground truth targets.
    extra_metrics : dict, optional
        Additional metric functions taking (y_true, y_pred) and
        returning a float. They will be merged into the default set.
    """

    y_pred, y_unc = model.predict_with_uncertainty(X_test)

    # basic regression scores
    metrics: Dict[str, float] = {
        "rmse": metrics_module.rmse(y_test, y_pred),
        "mae": metrics_module.mae(y_test, y_pred),
        "mape": metrics_module.mape(y_test, y_pred),
        "r2": metrics_module.r2(y_test, y_pred),
        "explained_variance": metrics_module.explained_variance(y_test, y_pred),
        "median_ae": metrics_module.median_ae(y_test, y_pred),
    }

    # calibration for models that return uncertainty
    if y_unc is not None:
        metrics["coverage_95"] = metrics_module.coverage_95(y_test, y_pred, y_unc)
        metrics["avg_std"] = metrics_module.avg_std(y_unc)
        metrics["interval_width_95"] = metrics_module.interval_width_95(y_unc)
        metrics["gaussian_nll"] = metrics_module.gaussian_nll(y_test, y_pred, y_unc)
        metrics["interval_score_95"] = metrics_module.interval_score_95(
            y_test, y_pred, y_unc
        )

    # compute any extras provided by caller
    if extra_metrics:
        for name, func in extra_metrics.items():
            try:
                metrics[name] = func(y_test, y_pred)
            except Exception:
                metrics[name] = np.nan

    return EvaluationResult(
        model_name=model.model_name,
        metrics=metrics,
        predictions=y_pred,
        uncertainties=y_unc,
        ground_truth=y_test,
    )

@staticmethod
def compare_models(
    models: List[LocalRiskModel],
    X_test: np.ndarray,
    y_test: np.ndarray,

```

```

    extra_metrics: Optional[Dict[str, Callable[[np.ndarray, np.ndarray], float]]] = None,
) -> Dict[str, EvaluationResult]:
    """Evaluate all models on same test set.

    Parameters mirror ``evaluate``; ``extra_metrics`` are passed through.
    """
    return {
        model.model_name: Evaluator.evaluate(model, X_test, y_test, extra_metrics)
        for model in models
    }

@staticmethod
def backtest(
    model: LocalRiskModel,
    X_test: np.ndarray,
    y_test: np.ndarray,
    risk_free_rate: float = 0.0
) -> Dict[str, float]:
    """Backtest a model assuming y_test represents returns."""
    y_pred, _ = model.predict_with_uncertainty(X_test)
    # Simple backtest: assume strategy based on prediction sign or something
    # For simplicity, use prediction error as basis for returns
    # In practice, this would be more sophisticated
    errors = y_test - y_pred
    # Pseudo-returns: positive if error is small
    pseudo_returns = -np.abs(errors) # negative for large errors
    return backtesting_module.backtest_summary(pseudo_returns, risk_free_rate)

```

src\evaluation\metrics.py

```

"""
Evaluation metrics

Standalone implementations of common regression and backtesting
scores. These are written in pure NumPy so they can be imported from
anywhere in the project and tested independently of the evaluation
pipeline.

Each function accepts ``y_true`` and ``y_pred`` arrays (and optionally
``y_std`` for calibration metrics) and returns a scalar float.
"""

import numpy as np
from typing import Optional

def rmse(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """Root mean squared error."""
    return float(np.sqrt(np.mean((y_true - y_pred) ** 2)))

def mae(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """Mean absolute error."""
    return float(np.mean(np.abs(y_true - y_pred)))

def mape(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """Mean absolute percentage error.

    Expressed as a fraction (not percent); division-by-zero yields ``np.nan``
    for the offending elements.
    """
    with np.errstate(divide='ignore', invalid='ignore'):
        return float(np.nanmean(np.abs((y_true - y_pred) / y_true)))

```

```

def r2(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """Coefficient of determination."""
    ss_res = np.sum((y_true - y_pred) ** 2)
    ss_tot = np.sum((y_true - np.mean(y_true)) ** 2)
    return float(1 - ss_res / ss_tot) if ss_tot != 0 else np.nan

def explained_variance(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """Explained variance score."""
    return float(1 - np.var(y_true - y_pred) / np.var(y_true))

def median_ae(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """Median absolute error."""
    return float(np.median(np.abs(y_true - y_pred)))

def coverage_95(
    y_true: np.ndarray, y_pred: np.ndarray, y_std: np.ndarray
) -> float:
    """Fraction of observations within 1.96 sigma of prediction."""
    return float(np.mean(np.abs(y_true - y_pred) <= 1.96 * y_std))

def avg_std(y_std: np.ndarray) -> float:
    """Mean of per-sample standard deviations."""
    return float(np.mean(y_std))

def interval_width_95(y_std: np.ndarray) -> float:
    """Average width of a nominal 95% predictive interval."""
    return float(np.mean(2 * 1.96 * y_std))

def gaussian_nll(y_true: np.ndarray, y_pred: np.ndarray, y_std: np.ndarray) -> float:
    """Gaussian negative log-likelihood using model mean and std."""
    sigma = np.maximum(np.asarray(y_std, dtype=float), 1e-8)
    residual = np.asarray(y_true, dtype=float) - np.asarray(y_pred, dtype=float)
    nll = 0.5 * np.log(2 * np.pi * sigma**2) + 0.5 * (residual**2) / (sigma**2)
    return float(np.mean(nll))

def interval_score_95(
    y_true: np.ndarray, y_pred: np.ndarray, y_std: np.ndarray, alpha: float = 0.05
) -> float:
    """Proper interval score for central (1-alpha) intervals; lower is better."""
    z = 1.96
    lower = y_pred - z * y_std
    upper = y_pred + z * y_std
    width = upper - lower
    below_penalty = (2.0 / alpha) * np.maximum(lower - y_true, 0.0)
    above_penalty = (2.0 / alpha) * np.maximum(y_true - upper, 0.0)
    return float(np.mean(width + below_penalty + above_penalty))

```

src\evaluation\rolling.py

```

from __future__ import annotations

from dataclasses import dataclass
from typing import Callable, Dict, List

```

```

import numpy as np
import pandas as pd

from .evaluator import Evaluator, EvaluationResult

@dataclass
class RollingFoldResult:
    """Evaluation output for one chronological fold."""

    fold_id: int
    train_start: int
    train_end: int
    test_start: int
    test_end: int
    result: EvaluationResult

def walk_forward_splits(
    n_samples: int,
    train_window: int,
    test_window: int,
    step: int | None = None,
) -> List[tuple[int, int, int, int]]:
    """Return chronological (train_start, train_end, test_start, test_end) folds."""
    if train_window <= 0 or test_window <= 0:
        raise ValueError("train_window and test_window must be positive.")
    if train_window + test_window > n_samples:
        raise ValueError("train_window + test_window cannot exceed n_samples.")

    step = test_window if step is None else step
    if step <= 0:
        raise ValueError("step must be positive.")

    folds: List[tuple[int, int, int, int]] = []
    train_start = 0
    while True:
        train_end = train_start + train_window
        test_start = train_end
        test_end = test_start + test_window
        if test_end > n_samples:
            break
        folds.append((train_start, train_end, test_start, test_end))
        train_start += step

    if not folds:
        raise ValueError("No valid walk-forward folds for provided settings.")
    return folds

def evaluate_walk_forward(
    model_builder: Callable[[], object],
    X: np.ndarray,
    y: np.ndarray,
    train_window: int,
    test_window: int,
    step: int | None = None,
    extra_metrics: Dict[str, Callable[[np.ndarray, np.ndarray], float]] | None = None,
) -> List[RollingFoldResult]:
    """Evaluate a model over rolling chronological folds."""
    folds = walk_forward_splits(
        n_samples=len(y),
        train_window=train_window,
        test_window=test_window,
        step=step,
    )

```

```

results: List[RollingFoldResult] = []
for fold_id, (tr_s, tr_e, te_s, te_e) in enumerate(folds):
    model = model_builder()
    model.fit(X[tr_s:tr_e], y[tr_s:tr_e])
    eval_result = Evaluator.evaluate(
        model=model,
        X_test=X[te_s:te_e],
        y_test=y[te_s:te_e],
        extra_metrics=extra_metrics,
    )
    results.append(
        RollingFoldResult(
            fold_id=fold_id,
            train_start=tr_s,
            train_end=tr_e,
            test_start=te_s,
            test_end=te_e,
            result=eval_result,
        )
    )
return results

def rolling_results_to_frame(results: List[RollingFoldResult]) -> pd.DataFrame:
    """Flatten fold metrics into a DataFrame."""
    rows = []
    for fold in results:
        row = {
            "fold_id": fold.fold_id,
            "train_start": fold.train_start,
            "train_end": fold.train_end,
            "test_start": fold.test_start,
            "test_end": fold.test_end,
            "model": fold.result.model_name,
        }
        row.update(fold.result.metrics)
        rows.append(row)
    return pd.DataFrame(rows)

def summarize_rolling_metrics(frame: pd.DataFrame) -> pd.DataFrame:
    """Return mean/std summary for all numeric metric columns."""
    metric_cols = [
        c
        for c in frame.columns
        if c not in {"fold_id", "train_start", "train_end", "test_start", "test_end", "model"}
        and pd.api.types.is_numeric_dtype(frame[c])
    ]
    summary = frame[metric_cols].agg(["mean", "std"]).T.reset_index()
    summary.columns = ["metric", "mean", "std"]
    return summary

```

src\Experiments\rolling_runner.py

```

from __future__ import annotations

from pathlib import Path
from typing import Callable, Dict

import numpy as np
import pandas as pd

from ..config import ExperimentConfig

```

```

from ..data.loader import load_fama_french
from ..evaluation.rolling import (
    evaluate_walk_forward,
    rolling_results_to_frame,
    summarize_rolling_metrics,
)
from ..models.factory import Modelfactory

class RollingExperimentRunner:
    """Run walk-forward evaluation for all models in an experiment config."""

    def __init__(
        self,
        config_path: str | Path,
        train_window: int,
        test_window: int,
        step: int | None = None,
    ) -> None:
        self.config = ExperimentConfig.from_yaml(config_path)
        self.train_window = train_window
        self.test_window = test_window
        self.step = step

    def run(
        self,
        extra_metrics: Dict[str, Callable[[np.ndarray, np.ndarray], float]] | None = None,
    ) -> tuple[pd.DataFrame, pd.DataFrame]:
        X, y, _ = load_fama_french(self.config.dataset_path)

        all_fold_frames = []
        all_summary_frames = []
        for model_config in self.config.models:
            def builder():
                return Modelfactory.create(model_config)

            fold_results = evaluate_walk_forward(
                model_builder=builder,
                X=X,
                y=y,
                train_window=self.train_window,
                test_window=self.test_window,
                step=self.step,
                extra_metrics=extra_metrics,
            )
            fold_frame = rolling_results_to_frame(fold_results)
            summary_frame = summarize_rolling_metrics(fold_frame)
            summary_frame["model"] = model_config.model_type

            all_fold_frames.append(fold_frame)
            all_summary_frames.append(summary_frame)

        folds = pd.concat(all_fold_frames, ignore_index=True)
        summary = pd.concat(all_summary_frames, ignore_index=True)
        return folds, summary

```

src\Experiments\runner.py

```

from pathlib import Path
from typing import List, Callable
import pandas as pd
import numpy as np

from ..config import ExperimentConfig

```

```

from ..data.pipeline import DataPipeline
from ..data.preprocessing import time_series_split
from ..models.factory import Modelfactory
from ..evaluation.evaluator import Evaluator

class ExperimentRunner:
    """Orchestrate multi-model training and evaluation."""

    def __init__(self, config_path: str | Path):
        self.config = ExperimentConfig.from_yaml(config_path)
        self.pipeline = DataPipeline()
        self.results = {}

    def run(
        self,
        extra_metrics: dict[str, Callable[[np.ndarray, np.ndarray], float]] | None = None,
    ) -> pd.DataFrame:
        """Execute full experiment.

        Parameters
        -----
        extra_metrics : dict, optional
            Additional evaluation functions (y_true, y_pred) -> float. They
            will be forwarded to ``Evaluator.evaluate``.
        """

        # 1. Load data once
        print("Loading dataset...")
        dataset = self.pipeline.load_fama_french(self.config.dataset_path)
        print(f" {dataset.describe()}")

        # 2. Split data once
        print("Splitting data...")
        X_train, X_test, y_train, y_test, _, _ = time_series_split(
            dataset.X, dataset.y, dataset.dates, self.config.train_ratio
        )
        print(f" Train: {len(y_train)}, Test: {len(y_test)}")

        #3. Train and evaluate each model
        results = []
        for model_config in self.config.models:
            print(f"\nTraining {model_config.model_type}...")

            model = Modelfactory.create(model_config)
            model.fit(X_train, y_train)

            eval_result = Evaluator.evaluate(
                model, X_test, y_test, extra_metrics=extra_metrics
            )
            results.append(eval_result)
            print(f" {eval_result}")

        #4. Compile results into Dataframe
        results_df = pd.DataFrame([
            {
                "model": r.model_name,
                **r.metrics,
            }
            for r in results
        ])

        self.results = results_df
        return results_df

    def save_results(self, path: str | Path):
        """Persist results."""

```

```
self.results.to_csv(path, index=False)
print(f"Results saved to {path} ")
```

src\Experiments\sweep.py

```
from itertools import product
from typing import Dict, List, Any
from pathlib import Path
import pandas as pd

# Import the missing classes
from ..config import ExperimentConfig
from .runner import ExperimentRunner

class HyperparameterSweep:
    """Grid search over model configurations."""

    def __init__(self, base_config_path: str | Path):
        self.base_config = ExperimentConfig.from_yaml(base_config_path)

    def sweep(
        self,
        model_type: str,
        param_grid: Dict[str, List[Any]]
    ) -> pd.DataFrame:
        """Run all combinations of hyperparameters."""

        param_names = list(param_grid.keys())
        param_values = list(param_grid.values())

        results = []
        for values in product(*param_values):
            config_dict = dict(zip(param_names, values))

            # Create variant config
            config = self.base_config
            model_config = [m for m in config.models if m.model_type == model_type][0]
            for param, value in config_dict.items():
                setattr(model_config, param, value)

            runner = ExperimentRunner(self.base_config)
            runner.config = config
            result = runner.run()
            result["hyperparams"] = str(config_dict)
            results.append(result)

        return pd.concat(results, ignore_index=True)
```

src\Experiments\tracking.py

```
from pathlib import Path
from datetime import datetime
import json
import shutil
from typing import Dict, Any
import pandas as pd

class ExperimentTracker:
    """Centralized experiment metadata tracking."""

    def __init__(self, log_dir: Path = Path("experiment/logs")):
        self.log_dir = log_dir
```

```

        self.log_dir.mkdir(exist_ok = True)

    def log_experiment(
        self,
        config_path: str,
        results_df: pd.DataFrame,
        metadata: Dict[str, Any] = None,
    ) -> str:
        """Save experiment with metadata."""

        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        exp_dir = self.log_dir / timestamp
        exp_dir.mkdir(exist_ok = True)

        # Save Config
        shutil.copy(config_path, exp_dir / "config.yaml")

        # Save results
        results_df.to_csv(exp_dir / "result.csv", index = False)

        # Save metadata
        meta = {
            "timestamp": timestamp,
            "config_file": str(config_path),
            "dataset_hash": metadata.get("dataset_hash"),
            "git_commit": self._get_git_commit(),
            **(metadata or {}),
        }
        with open(exp_dir / "metadata.json", "w") as f:
            json.dump(meta, f, indent = 2)

        return str(exp_dir)

    @staticmethod
    def _get_git_commit() -> str:
        try:
            import subprocess
            return subprocess.check_output(
                ["git", "rev-parse", "HEAD"]
            ).decode().strip()
        except:
            return "unkown"

```

src\Experiments\tuning.py

```

from __future__ import annotations

from dataclasses import replace
from itertools import product
from typing import Any, Dict, List

import numpy as np
import pandas as pd

from ..config import ModelCofig
from ..evaluation.rolling import (
    evaluate_walk_forward,
    rolling_results_to_frame,
    summarize_rolling_metrics,
)
from ..models.factory import Modelfactory

def _iter_grid(param_grid: Dict[str, List[Any]]) -> List[Dict[str, Any]]:

```

```

names = list(param_grid.keys())
values = [param_grid[k] for k in names]
return [dict(zip(names, combo)) for combo in product(*values)]

def tune_model_with_walk_forward(
    base_model_config: ModelCofig,
    param_grid: Dict[str, List[Any]],
    X: np.ndarray,
    y: np.ndarray,
    train_window: int,
    test_window: int,
    objective_metric: str = "rmse",
    minimize: bool = True,
    step: int | None = None,
) -> tuple[pd.DataFrame, Dict[str, Any]]:
    """Grid-search model hyperparameters using walk-forward validation."""
    grid = _iter_grid(param_grid)
    if not grid:
        raise ValueError("param_grid produced no configurations.")

    rows: List[Dict[str, Any]] = []
    for params in grid:
        model_config = replace(base_model_config, **params)

        def builder():
            return ModelFactory.create(model_config)

        folds = evaluate_walk_forward(
            model_builder=builder,
            X=X,
            y=y,
            train_window=train_window,
            test_window=test_window,
            step=step,
        )
        fold_frame = rolling_results_to_frame(folds)
        summary = summarize_rolling_metrics(fold_frame)
        metric_row = summary[summary["metric"] == objective_metric]
        if metric_row.empty:
            raise ValueError(f"Metric '{objective_metric}' not found in fold summary.")

        rows.append(
            {
                "params": params,
                "objective_metric": objective_metric,
                "objective_mean": float(metric_row["mean"].iloc[0]),
                "objective_std": float(metric_row["std"].iloc[0]),
                "n_folds": int(len(folds)),
            }
        )

    leaderboard = pd.DataFrame(rows)
    leaderboard = leaderboard.sort_values(
        by=["objective_mean", "objective_std"],
        ascending=[minimize, True],
    ).reset_index(drop=True)
    best = leaderboard.iloc[0].to_dict()
    return leaderboard, best

```

src\models\base.py

```

from abc import ABC, abstractmethod
from typing import Tuple, Optional

```

```

import numpy as np

class LocalRiskModel(ABC):
    """ Base protocol for all the regression models in the framework."""

    @abstractmethod
    def fit(self, X: np.ndarray, y: np.ndarray) -> "LocalRiskModel":
        """Fit model to training data."""
        pass

    @abstractmethod
    def predict(self, X: np.ndarray) -> np.ndarray:
        """Return point predictions."""
        pass

    def predict_with_uncertainty(self, X: np.ndarray) -> Tuple[np.ndarray, Optional[np.ndarray]]:
        """Return (predictions, Uncertainty), Optional for stochastic models."""
        return self.predict(X), None

    @property
    @abstractmethod
    def is_fitted(self) -> bool:
        """Check if model has been fitted."""
        pass

    @property
    @abstractmethod
    def model_name(self) -> str:
        """Unique identifier for model type."""
        pass

    @property
    @abstractmethod
    def hyperparameters(self) -> dict:
        """Return hyperparameter dict for logging/comparison"""
        pass

```

src\models\factory.py

```

from typing import Type, Dict
from .base import LocalRiskModel
from .global_linear import GlobalLinearRegression
from .lwlr import LocallyWeightedLinearRegression
from .mc_linear import MonteCarloLinearRegression
from .mc_llr import MonteCarloLocalLinearRegression
from .mc_llr_ablations import MCLLRNoGeometry, MCLLRNoStochasticity
from .ridge_linear import RidgeLinearRegression
from ..config import ModelCofig

class ModelFactory:
    """Registry pattern for model instantiation."""

    _registry: Dict[str, Type[LocalRiskModel]] = {
        "global_linear": GlobalLinearRegression,
        "lwlr": LocallyWeightedLinearRegression,
        "mc_linear": MonteCarloLinearRegression,
        "mc_llr": MonteCarloLocalLinearRegression,
        "mc_llr_no_geometry": MCLLRNoGeometry,
        "mc_llr_no_stochasticity": MCLLRNoStochasticity,
        "ridge_linear": RidgeLinearRegression,
    }

    @classmethod
    def create(cls, config: ModelCofig) -> LocalRiskModel:

```

```

"""Instantiate model from config."""
model_cls = cls._registry.get(config.model_type)
if model_cls is None:
    raise ValueError(f"Unknown model: {config.model_type}")

# Build model-specific kwargs to avoid passing unsupported args.
if config.model_type == "global_linear":
    kwargs = {
        "fit_intercept": config.fit_intercept,
    }
elif config.model_type == "lwlr":
    kwargs = {
        "bandwidth": config.bandwidth,
        "fit_intercept": config.fit_intercept,
    }
elif config.model_type == "mc_linear":
    kwargs = {
        "n_simulations": config.n_simulations,
        "subsample_size": config.subsample_size,
        "fit_intercept": config.fit_intercept,
        "random_state": config.random_state,
    }
elif config.model_type == "mcllr":
    kwargs = {
        "bandwidth": config.bandwidth,
        "n_simulations": config.n_simulations,
        "subsample_size": config.subsample_size,
        "fit_intercept": config.fit_intercept,
        "random_state": config.random_state,
    }
elif config.model_type == "mcllr_no_geometry":
    kwargs = {
        "n_simulations": config.n_simulations,
        "subsample_size": config.subsample_size,
        "fit_intercept": config.fit_intercept,
        "random_state": config.random_state,
    }
elif config.model_type == "mcllr_no_stochasticity":
    kwargs = {
        "bandwidth": config.bandwidth,
        "subsample_size": config.subsample_size,
        "fit_intercept": config.fit_intercept,
    }
elif config.model_type == "ridge_linear":
    kwargs = {
        "alpha": config.alpha if config.alpha is not None else 1.0,
        "fit_intercept": config.fit_intercept,
    }
else:
    raise ValueError(f"Unknown model: {config.model_type}")

return model_cls(**kwargs)

@classmethod
def register(cls, name: str, model_cls: Type[LocalRiskModel]):
    """register new model type dynamically."""
    cls._registry[name] = model_cls

```

src\models\global_linear.py

```

from __future__ import annotations
from typing import Tuple

import numpy as np

```

```

"""
Global Linear Regression model for Local Risk Dynamics.

This module implements a stationary baseline using closed-form
Ordinary Least Squares (OLS) with a numerically stable pseudo-inverse.

Scientific role
-----
Serves as the control model assuming constant factor sensitivities
across time. Performance gaps versus local or stochastic models
quantify the effect of non-stationarity in financial data.
"""

from ..models.base import LocalRiskModel

class GlobalLinearRegression(LocalRiskModel):
    """
    Closed_form Ordinary Least Squares linear regression.

    Parameters
    -----
    fit_intercept : bool, default = True
        whether to include an intercept term in the model

    Attributes
    -----
    coef_ : np.ndarray of shape (n_features)
        Estimated slope coefficients.

    inetercept_ : float
        Estimated intercept term (0.0 if fit_intercept = False).

    n_features_ : int
        Number of input features seen during fitting.

    is_fitted_ : bool
        Indicates whether the model has been fitted.

    """
    def __init__(self, fit_intercept: bool = True) -> None:
        self.fit_intercept = fit_intercept

        # Attributes set during fitting
        self.coef_: np.ndarray | None = None
        self.intercept_: float | None = None
        self.n_features_: int | None = None
        self.is_fitted_: bool = False

    # -----
    # Internal Utilities
    # -----

    def _add_intercept(self, X: np.ndarray) -> np.ndarray:
        """Add a column of ones to X if intercept is enabled."""
        if not self.fit_intercept:
            return X
        ones = np.ones((X.shape[0], 1), dtype=X.dtype)
        return np.hstack((ones, X))

    def _check_is_fitted_(self)->None:
        """Raise an error if the model is not fitted."""
        if not self.is_fitted_:
            raise RuntimeError("Model must be fitted before calling predict or score.")

    # -----

```

```

# Core API
# -----

def fit(self, X: np.ndarray, y: np.ndarray) -> "GlobalLinearRegression":
    """
    Fit the global linear regression model using closed-form OLS.

    parameters
    -----
    X : np.ndarray of shape (n_samples, n_features)
        Training feature matrix.

    y : np.ndarray of shape (n_samples,)
        Target vector.

    Returns
    -----
    self : GlobalLinearRegression
        Fitted model instance.
    """

    # ----- Validation -----
    if X.ndim != 2:
        raise ValueError("X must be a 2D array.")

    if y.ndim != 1:
        raise ValueError("y must be a 1D array.")

    if X.shape[0] != y.shape[0]:
        raise ValueError("X and y must have same number of samples.")

    if X.shape[0] == 0:
        raise ValueError("Cannot fit model on empty dataset.")

    # ----- Store feature count -----
    self.n_features_ = X.shape[1]

    # ----- Design matrix -----
    X_design = self._add_intercept(X)

    # ----- Closed form OLS via pseudo-inverse -----
    beta = np.linalg.pinv(X_design) @ y

    # ----- Separate intercept and coefficients -----
    if self.fit_intercept:
        self.intercept_ = float(beta[0])
        self.coef_ = beta[1:]
    else:
        self.intercept_ = 0.0
        self.coef_ = beta

    self.is_fitted_ = True
    return self

# -----

def predict(self, X: np.ndarray) -> np.ndarray:
    """
    Predict target values using fitted linear model.

    parameters
    -----
    X : np.ndarray of shape (n_samples, n_features)
        Input feature matrix

    Returns
    """

```

```

-----
y_pred : np.ndarray of shape (n_samples,)
    Predicted values.
"""

self._check_is_fitted_()

if X.ndim != 2:
    raise ValueError("X must be a 2D array.")

if X.shape[1] != self.n_features_:
    raise ValueError(
        f"Expected {self.n_features_} features, got {X.shape[1]}"
    )

X_design = self._add_intercept(X)

beta = (
    np.concatenate([self.intercept_, self.coef_])
    if self.fit_intercept
    else self.coef_
)

return X_design @ beta

# -----

def score(self, X: np.ndarray, y: np.ndarray) -> float:
    """
    Compute Mean Squared Error (MSE) of predictions.

    Parameters
    -----
    X : np.ndarray of shape (n_samples, n_features)
        Feature Matrix.

    y : np.ndarray of shape (n_samples,)
        True target values.

    Returns
    -----
    mse : float
        Mean squared prediction error.
    """

    self._check_is_fitted_()

    if y.ndim != 1:
        raise ValueError("y must be a 1D array.")

    if X.shape[0] != y.shape[0]:
        raise ValueError("X and y must contain the same number of samples.")

    y_pred = self.predict(X)
    mse = np.mean((y - y_pred) ** 2)

    return float(mse)

# -----
# Interface compliance
# -----

@property
def is_fitted(self) -> bool:
    return self.is_fitted_

```

```

@property
def model_name(self) -> str:
    return "global_linear"

@property
def hyperparameters(self) -> dict:
    return {"fit_intercept": self.fit_intercept}

```

src\models\lwlr.py

```

"""
Locally Weighted Linear Regression (LWLR) for Local Risk Dynamics.

This module implements deterministic locality using Gaussian
kernel-weighted least squares solved at prediction time.

Scientific role
-----
Introduces geometric locality into linear regression by allowing
coefficients to vary smoothly across feature space.

Serves as the deterministic locality baseline against which
stochastic local models (MCLLR) are evaluated.
"""

from __future__ import annotations
import numpy as np
from typing import Optional

from ..utils.kernels import GaussianKernel, Kernel

from ..models.base import LocalRiskModel

class LocallyWeightedLinearRegression(LocalRiskModel):
    """
    Gaussian Kernel Locally Weighted Linear Regression (LWLR).
    """

    def __init__(
        self,
        bandwidth: float,
        fit_intercept: bool = True,
        kernel: Kernel | None = None,
    ) -> None:
        if bandwidth <= 0:
            raise ValueError("bandwidth must be strictly positive.")

        self.bandwidth = float(bandwidth)
        self.fit_intercept = fit_intercept
        # allow custom kernel; default to Gaussian
        self.kernel = kernel if kernel is not None else GaussianKernel(self.bandwidth)

        self.X_train_: Optional[np.ndarray] = None
        self.y_train_: Optional[np.ndarray] = None
        self.n_features_: Optional[int] = None
        self.is_fitted_: bool = False

    # -----

    def _add_intercept(self, X: np.ndarray) -> np.ndarray:
        if not self.fit_intercept:
            return X
        ones = np.ones((X.shape[0], 1), dtype=X.dtype)

```

```

        return np.hstack((ones, X))

def _check_is_fitted(self) -> None:
    if not self.is_fitted_:
        raise RuntimeError("Model must be fitted before prediction.")

# -----

def fit(self, X: np.ndarray, y: np.ndarray) -> "LocallyWeightedLinearRegression":
    if X.ndim != 2:
        raise ValueError("X must be 2D.")
    if y.ndim != 1:
        raise ValueError("y must be 1D.")
    if X.shape[0] != y.shape[0]:
        raise ValueError("X and y must match in sample size.")
    if X.shape[0] == 0:
        raise ValueError("Empty dataset.")

    self.X_train_ = X.astype(float)
    self.y_train_ = y.astype(float)
    self.n_features_ = X.shape[1]
    self.is_fitted_ = True

    return self

# -----

def predict(self, X_query: np.ndarray) -> np.ndarray:
    self._check_is_fitted()

    if X_query.ndim != 2:
        raise ValueError("X_query must be 2D.")
    if X_query.shape[1] != self.n_features_:
        raise ValueError(
            f"Expected {self.n_features_} features, got {X_query.shape[1]}"
        )

    X_train = self.X_train_
    y_train = self.y_train_
    X_design = self._add_intercept(X_train)

    y_pred = np.zeros(X_query.shape[0])

    for i, x0 in enumerate(X_query):
        weights = self.kernel(X_train, x0)

        # Avoid explicit diagonal matrix
        W = weights[:, np.newaxis]
        X_weighted = X_design * W

        A = X_design.T @ X_weighted
        b = X_design.T @ (weights * y_train)

        beta = np.linalg.pinv(A) @ b

        x0_design = self._add_intercept(x0.reshape(1, -1))
        y_pred[i] = float(x0_design @ beta)

    return y_pred

# -----

def score(self, X: np.ndarray, y: np.ndarray) -> float:
    self._check_is_fitted()

```

```

        y_pred = self.predict(X)
        return float(np.mean((y - y_pred) ** 2))

# -----
# Interface compliance
# -----

@property
def is_fitted(self) -> bool:
    return self.is_fitted_

@property
def model_name(self) -> str:
    return "lwlr"

@property
def hyperparameters(self) -> dict:
    return {"bandwidth": self.bandwidth, "fit_intercept": self.fit_intercept}

def predict_with_uncertainty(self, X: np.ndarray):
    return self.predict(X), None

```

src\models\mc_linear.py

```

"""
Monte Carlo Linear Regression (MC-LR).

This module implements stochastic subsampling to estimate
uncertainty in global linear regression.

Scientific role
-----
Models:
    > Uncertainty in regression surface
    > Instability of global linear assumption
    > Sampling variability
"""

from __future__ import annotations
import numpy as np

from ..models.base import LocalRiskModel

class MonteCarloLinearRegression(LocalRiskModel):
    """
    Stochastic subsampling and prediction ensembling.

    Parameters
    -----
    n_simulations : int
        Number of Monte Carlo runs. Must be positive.

    subsample_size : int
        Number of samples per regression (m). Must satisfy 1 < m <= n.

    fit_intercept : bool, default=True
        Whether to include an intercept term.

    random_state : int | None
        Random seed for reproducibility.

    Attributes
    -----

```

```

X_train_ : np.ndarray of shape (n_samples, n_features)
    Stored training feature matrix.

y_train_ : np.ndarray of shape (n_samples,)
    Stored training targets.

n_features_ : int
    Number of features seen during fitting.

is_fitted_ : bool
    Indicates whether the model has been fitted.

_rng : np.random.Generator
    Internal random number generator used for subsampling.
"""

# -----
# Constructor
# -----

def __init__(
    self,
    n_simulations: int,
    subsample_size: int,
    fit_intercept: bool = True,
    random_state: int | None = None,
) -> None:

    if n_simulations <= 0:
        raise ValueError("n_simulations must be strictly positive.")

    if subsample_size <= 1:
        raise ValueError("subsample_size must be greater than 1.")

    self.n_simulations = n_simulations
    self.subsample_size = subsample_size
    self.fit_intercept = fit_intercept
    self.random_state = random_state

    # Runtime attributes
    self.X_train_: np.ndarray | None = None
    self.y_train_: np.ndarray | None = None
    self.n_features_: int | None = None
    self.is_fitted_: bool = False
    self.coefs_: np.ndarray | None = None

    # Internal RNG
    self._rng = np.random.default_rng(random_state)

def _simulate_coefficients(self) -> np.ndarray:
    """Draw Monte Carlo subsamples and fit one OLS model per draw."""
    n_train = self.X_train_.shape[0]
    n_coef = self.n_features_ + (1 if self.fit_intercept else 0)
    coefs = np.zeros((self.n_simulations, n_coef))

    for k in range(self.n_simulations):
        indices = self._rng.choice(
            n_train,
            size=self.subsample_size,
            replace=False,
        )
        X_sub = self.X_train_[indices]
        y_sub = self.y_train_[indices]
        X_sub_design = self._add_intercept(X_sub)
        coefs[k] = np.linalg.pinv(X_sub_design) @ y_sub

```

```

        return coefs

# -----
# Internal Utilities
# -----

def _check_is_fitted(self) -> None:
    if not self.is_fitted_:
        raise RuntimeError("Model must be fitted before prediction.")

def _add_intercept(self, X: np.ndarray) -> np.ndarray:
    if not self.fit_intercept:
        return X
    ones = np.ones((X.shape[0], 1))
    return np.hstack((ones, X))

# -----
# Fit
# -----

def fit(self, X: np.ndarray, y: np.ndarray) -> "MonteCarloLinearRegression":
    """
    Store training data for Monte Carlo subsampling.
    """

    if X.ndim != 2:
        raise ValueError("X must be a 2D array.")

    if y.ndim != 1:
        raise ValueError("y must be a 1D array.")

    if X.shape[0] != y.shape[0]:
        raise ValueError("X and y must have the same number of samples.")

    if X.shape[0] == 0:
        raise ValueError("Cannot fit on an empty dataset.")

    if self.subsample_size > X.shape[0]:
        raise ValueError(
            "subsample_size cannot exceed number of training samples."
        )

    self.X_train_ = X
    self.y_train_ = y
    self.n_features_ = X.shape[1]
    self.coefs_ = self._simulate_coefficients()
    self.is_fitted_ = True

    return self

# -----
# Predict
# -----

def predict(
    self,
    X: np.ndarray,
    return_std: bool = False,
) -> np.ndarray | tuple[np.ndarray, np.ndarray]:

    self._check_is_fitted()

    if X.ndim != 2:
        raise ValueError("X must be a 2D array.")

    if X.shape[1] != self.n_features_:

```

```

        raise ValueError(
            f"Expected {self.n_features_} features, got {X.shape[1]}"
        )

    n_test = X.shape[0]
    predictions = np.zeros((self.n_simulations, n_test))

    # Precompute test design matrix
    X_test_design = self._add_intercept(X)

    if self.coefs_ is None or self.coefs_.shape[0] != self.n_simulations:
        self.coefs_ = self._simulate_coefficients()

    for k, beta in enumerate(self.coefs_):
        predictions[k] = X_test_design @ beta

    # Aggregate
    mean_pred = predictions.mean(axis=0)

    if return_std:
        std_pred = predictions.std(axis=0)
        return mean_pred, std_pred

    return mean_pred

# -----
# Score
# -----

def score(self, X: np.ndarray, y: np.ndarray) -> float:
    """
    Compute Mean Squared Error (MSE) using mean prediction.
    """

    if y.ndim != 1:
        raise ValueError("y must be a 1D array.")

    if X.shape[0] != y.shape[0]:
        raise ValueError("X and y must have the same number of samples.")

    y_pred = self.predict(X)
    mse = np.mean((y - y_pred) ** 2)

    return float(mse)

# -----
# Interface compliance
# -----

@property
def is_fitted(self) -> bool:
    return self.is_fitted_

@property
def model_name(self) -> str:
    return "mc_linear"

@property
def hyperparameters(self) -> dict:
    return {
        "n_simulations": self.n_simulations,
        "subsample_size": self.subsample_size,
        "fit_intercept": self.fit_intercept,
    }

```

```
def predict_with_uncertainty(self, X: np.ndarray):
    return self.predict(X, return_std=True)
```

src\models\mcLLR.py

```
from __future__ import annotations

"""
Monte Carlo Local Linear Regression (MCLLR).

This module implements the proposed hybrid model combining:
- Geometric locality via Gaussian kernel weighting
- Stochastic locality via Monte Carlo sampling

It produces uncertainty-aware, locally adaptive predictions for non-stationary financial data.
"""

import numpy as np

from ..models.base import LocalRiskModel

class MonteCarloLocalLinearRegression(LocalRiskModel):
    """
    Monte Carlo Local Linear Regression (MCLLR).

    Combines:
    - Geometric locality via Gaussian kernel weighting
    - Stochastic neighborhood selection via Monte Carlo sampling

    For each query point, the model:
    1. Computes kernel-based similarity weights
    2. Converts them into a probability distribution
    3. Samples local neighborhoods
    4. Fits local linear models
    5. Aggregates predictions into mean and uncertainty

    This produces uncertainty-aware, locally adaptive predictions.

    Parameters
    -----
    bandwidth : float
        Gaussian kernel bandwidth controlling locality. Must be positive.

    n_simulations : int
        Number of Monte Carlo runs per query point. Must be positive.

    subsample_size : int
        Number of samples drawn per local neighborhood. Must satisfy 1 < m <= n.

    fit_intercept : bool, default=True
        Whether to include an intercept term.

    random_state : int | None
        Random seed for reproducibility.

    Attributes
    -----
    X_train_ : np.ndarray of shape (n_samples, n_features)
        Stored training feature matrix.

    y_train_ : np.ndarray of shape (n_samples,)
        Stored training targets.

    n_features_ : int
```

```

        Number of features seen during fitting.

is_fitted_ : bool
    Indicates whether the model has been fitted.

_rng : np.random.Generator
    Internal random number generator used for subsampling.
"""

# -----
# Constructor
# -----

def __init__(
    self,
    bandwidth: float,
    n_simulations: int,
    subsample_size: int,
    fit_intercept: bool = True,
    random_state: int | None = None,
) -> None:
    if bandwidth <= 0:
        raise ValueError("Bandwidth must be positive.")

    if n_simulations <= 0:
        raise ValueError("n_simulations must be strictly positive.")

    if subsample_size <= 1:
        raise ValueError("subsample_size must be greater than 1.")

    self.bandwidth = bandwidth
    self.n_simulations = n_simulations
    self.subsample_size = subsample_size
    self.fit_intercept = fit_intercept
    self.random_state = random_state

    # Runtime Attributes
    self.X_train_: np.ndarray | None = None
    self.y_train_: np.ndarray | None = None
    self.n_features_: int | None = None
    self.is_fitted_: bool = False

    # Internal RNG
    self._rng = np.random.default_rng(random_state)

# -----
# Internal Utilities
# -----

def _check_is_fitted(self) -> None:
    if not self.is_fitted_:
        raise RuntimeError("Model must be fitted before prediction.")

def _add_intercept(self, X: np.ndarray) -> np.ndarray:
    if not self.fit_intercept:
        return X
    ones = np.ones((X.shape[0], 1))
    return np.hstack((ones, X))

# -----
# Fit
# -----

def fit(self, X: np.ndarray, y: np.ndarray) -> "MonteCarloLocalLinearRegression":
    """
    Store training data for Monte Carlo local sampling.

    Parameters

```

```

-----
X : np.ndarray of shape (n_samples, n_features)
    Training feature matrix.

y : np.ndarray of shape (n_samples,)
    Training target vector.

Returns
-----
self : MonteCarloLocalLinearRegression
    Fitted model instance.
"""
if X.ndim != 2:
    raise ValueError("X must be a 2D array.")

if y.ndim != 1:
    raise ValueError("y must be a 1D array.")

if X.shape[0] != y.shape[0]:
    raise ValueError("X and y must have the same number of samples.")

if X.shape[0] == 0:
    raise ValueError("Cannot fit on an empty dataset.")

if self.subsample_size > X.shape[0]:
    raise ValueError(
        "subsample_size cannot exceed number of training samples."
    )

self.X_train_ = X.astype(float)
self.y_train_ = y.astype(float)
self.n_features_ = X.shape[1]
self.is_fitted_ = True

return self

# -----
# Compute Kernel Weights
# -----
def _compute_kernel_weights(self, x0: np.ndarray) -> np.ndarray:
    """
    Compute raw Gaussian kernel weights for a single query point.

    Parameters
    -----
    x0 : np.ndarray of shape (n_features,)
        Query point.

    Returns
    -----
    weights : np.ndarray of shape (n_train,)
        Raw (unnormalized) kernel weights.
    """

    if x0.ndim != 1:
        raise ValueError("x0 must be a 1D array")

    if x0.shape[0] != self.n_features_:
        raise ValueError(f"Expected {self.n_features_} features, got {x0.shape[0]}")

    # Compute squared Euclidean distances (no square root for numerical stability)
    diff = self.X_train_ - x0
    sq_dist = np.sum(diff ** 2, axis=1)

    # Gaussian kernel:  $w_i(x_0) = \exp(-||x_i - x_0||^2 / (2 * \tau^2))$ 
    weights = np.exp(-sq_dist / (2 * self.bandwidth ** 2))

```

```

        return weights

# -----
# Predict
# -----
def predict(
    self,
    X: np.ndarray,
    return_std: bool = False,
) -> np.ndarray | tuple[np.ndarray, np.ndarray]:
    """
    Predict target values for query points using MCLLR.

    For each test point x0:
    1. Compute kernel weights  $w_i(x_0) = \exp(-||x_i - x_0||^2 / (2*\tau^2))$ 
    2. Normalize to probabilities  $p_i = w_i / \sum(w_i) + \epsilon$ 
    3. For r=1 to n_simulations:
        a. Sample neighborhood  $S_r \sim w/o \text{ replacement}(m, p)$ 
        b. Fit plain OLS on  $S_r$ 
        c. Predict  $y_r$  at  $x_0$ 
    4. Aggregate:  $\text{mean}(y_r), \text{std}(y_r)$ 

    Parameters
    -----
    X : np.ndarray of shape (n_test, n_features)
        Query point matrix.

    return_std : bool, default=False
        Whether to return uncertainty (standard deviation).

    Returns
    -----
    y_pred : np.ndarray of shape (n_test,)
        Mean predicted values.

    y_std : np.ndarray of shape (n_test,), optional
        Predicted standard deviations (if return_std=True).
    """

    self._check_is_fitted()

    if X.ndim != 2:
        raise ValueError("X must be a 2D array.")

    if X.shape[1] != self.n_features_:
        raise ValueError(
            f"Expected {self.n_features_} features, got {X.shape[1]}"
        )

    n_test = X.shape[0]
    mean_predictions = np.zeros(n_test)
    std_predictions = np.zeros(n_test)
    eps = 1e-12

    # Outer loop over test points (query-dependent locality)
    for i in range(n_test):
        x0 = X[i]

        # Step 1: Compute raw kernel weights for this query point
        weights = self._compute_kernel_weights(x0)

        # Step 2: Normalize to probabilities (critical bridge)
        weight_sum = np.sum(weights)
        probabilities = weights / (weight_sum + eps)

```

```

# Store Monte Carlo predictions for aggregation
simulation_preds = np.zeros(self.n_simulations)

# Inner loop: Monte Carlo simulations with stochastic neighborhoods
for k in range(self.n_simulations):
    # Step 3: Sample local neighborhood without replacement
    indices = self.rng.choice(
        self.X_train_.shape[0],
        size=self.subsample_size,
        replace=False,
        p=probabilities,
    )

    # Create subsampled local neighborhood
    X_sub = self.X_train_[indices]
    y_sub = self.y_train_[indices]

    # Step 4: Fit plain OLS (unweighted) on local subset
    # Locality enforced via sampling, not regression weights
    X_sub_design = self._add_intercept(X_sub)
    beta = np.linalg.pinv(X_sub_design) @ y_sub

    # Predict at x0 with local linear model
    x0_design = self._add_intercept(x0.reshape(1, -1))
    simulation_preds[k] = (x0_design @ beta).item()

# Step 5: Aggregate Monte Carlo predictions
mean_predictions[i] = np.mean(simulation_preds)
std_predictions[i] = np.std(simulation_preds)

# Return after processing all test points
if return_std:
    return mean_predictions, std_predictions

return mean_predictions

# -----
# Score
# -----
def score(self, X: np.ndarray, y: np.ndarray)->float:
    """
    Compute Mean Squared Error (MSE) using mean prediction.
    """
    if y.ndim != 1:
        raise ValueError("y must be a 1D array.")

    if X.shape[0] != y.shape[0]:
        raise ValueError("X and y must have same number of samples.")

    y_pred = self.predict(X)
    mse = np.mean((y - y_pred) ** 2)

    return float(mse)

# -----
# Interface compliance
# -----

@property
def is_fitted(self) -> bool:
    return self.is_fitted_

@property
def model_name(self) -> str:
    return "mcllr"

```

```

@property
def hyperparameters(self) -> dict:
    return {
        "bandwidth": self.bandwidth,
        "n_simulations": self.n_simulations,
        "subsample_size": self.subsample_size,
        "fit_intercept": self.fit_intercept,
    }

def predict_with_uncertainty(self, X: np.ndarray):
    return self.predict(X, return_std=True)

```

src\models\mcllr_ablations.py

```

from __future__ import annotations

import numpy as np

from .base import LocalRiskModel

class MCLLRNoGeometry(LocalRiskModel):
    """Ablation: stochastic sampling without geometric kernel locality."""

    def __init__(
        self,
        n_simulations: int,
        subsample_size: int,
        fit_intercept: bool = True,
        random_state: int | None = None,
    ) -> None:
        if n_simulations <= 0:
            raise ValueError("n_simulations must be positive.")
        if subsample_size <= 1:
            raise ValueError("subsample_size must be greater than 1.")
        self.n_simulations = n_simulations
        self.subsample_size = subsample_size
        self.fit_intercept = fit_intercept
        self.random_state = random_state
        self._rng = np.random.default_rng(random_state)

        self.X_train_: np.ndarray | None = None
        self.y_train_: np.ndarray | None = None
        self.n_features_: int | None = None
        self.is_fitted_: bool = False

    def _add_intercept(self, X: np.ndarray) -> np.ndarray:
        if not self.fit_intercept:
            return X
        return np.hstack((np.ones((X.shape[0], 1)), X))

    def fit(self, X: np.ndarray, y: np.ndarray) -> "MCLLRNoGeometry":
        if X.ndim != 2 or y.ndim != 1:
            raise ValueError("X must be 2D and y must be 1D.")
        if X.shape[0] != y.shape[0]:
            raise ValueError("X and y must have same number of samples.")
        if self.subsample_size > X.shape[0]:
            raise ValueError("subsample_size cannot exceed number of samples.")
        self.X_train_ = X.astype(float)
        self.y_train_ = y.astype(float)
        self.n_features_ = X.shape[1]
        self.is_fitted_ = True
        return self

```

```

def predict(
    self, X: np.ndarray, return_std: bool = False
) -> np.ndarray | tuple[np.ndarray, np.ndarray]:
    if not self.is_fitted_:
        raise RuntimeError("Model must be fitted before prediction.")
    if X.ndim != 2 or X.shape[1] != self.n_features_:
        raise ValueError("Input has invalid shape.")

    n_test = X.shape[0]
    mean_preds = np.zeros(n_test)
    std_preds = np.zeros(n_test)
    X_train_n = self.X_train_.shape[0]
    X_test_design = self._add_intercept(X)

    for i in range(n_test):
        sim_preds = np.zeros(self.n_simulations)
        for k in range(self.n_simulations):
            idx = self._rng.choice(
                X_train_n, size=self.subsample_size, replace=False
            )
            X_sub = self._add_intercept(self.X_train_[idx])
            y_sub = self.y_train_[idx]
            beta = np.linalg.pinv(X_sub) @ y_sub
            sim_preds[k] = float(X_test_design[i] @ beta)
        mean_preds[i] = float(np.mean(sim_preds))
        std_preds[i] = float(np.std(sim_preds))

    return (mean_preds, std_preds) if return_std else mean_preds

def predict_with_uncertainty(self, X: np.ndarray):
    return self.predict(X, return_std=True)

@property
def is_fitted(self) -> bool:
    return self.is_fitted_

@property
def model_name(self) -> str:
    return "mcllr_no_geometry"

@property
def hyperparameters(self) -> dict:
    return {
        "n_simulations": self.n_simulations,
        "subsample_size": self.subsample_size,
        "fit_intercept": self.fit_intercept,
    }

class MCLLRNoStochasticity(LocalRiskModel):
    """Ablation: geometric locality without stochastic neighborhood sampling."""

    def __init__(
        self,
        bandwidth: float,
        subsample_size: int,
        fit_intercept: bool = True,
    ) -> None:
        if bandwidth <= 0:
            raise ValueError("bandwidth must be positive.")
        if subsample_size <= 1:
            raise ValueError("subsample_size must be greater than 1.")
        self.bandwidth = bandwidth
        self.subsample_size = subsample_size
        self.fit_intercept = fit_intercept

```

```

self.X_train_: np.ndarray | None = None
self.y_train_: np.ndarray | None = None
self.n_features_: int | None = None
self.is_fitted_: bool = False

def _add_intercept(self, X: np.ndarray) -> np.ndarray:
    if not self.fit_intercept:
        return X
    return np.hstack((np.ones((X.shape[0], 1)), X))

def fit(self, X: np.ndarray, y: np.ndarray) -> "MCLLRNoStochasticity":
    if X.ndim != 2 or y.ndim != 1:
        raise ValueError("X must be 2D and y must be 1D.")
    if X.shape[0] != y.shape[0]:
        raise ValueError("X and y must have same number of samples.")
    if self.subsample_size > X.shape[0]:
        raise ValueError("subsample_size cannot exceed number of samples.")
    self.X_train_ = X.astype(float)
    self.y_train_ = y.astype(float)
    self.n_features_ = X.shape[1]
    self.is_fitted_ = True
    return self

def _kernel_weights(self, x0: np.ndarray) -> np.ndarray:
    diff = self.X_train_ - x0
    sq_dist = np.sum(diff**2, axis=1)
    return np.exp(-sq_dist / (2 * self.bandwidth**2))

def predict(self, X: np.ndarray) -> np.ndarray:
    if not self.is_fitted_:
        raise RuntimeError("Model must be fitted before prediction.")
    if X.ndim != 2 or X.shape[1] != self.n_features_:
        raise ValueError("Input has invalid shape.")

    y_pred = np.zeros(X.shape[0])
    for i, x0 in enumerate(X):
        weights = self._kernel_weights(x0)
        idx = np.argsort(weights)[-self.subsample_size :]
        X_sub = self._add_intercept(self.X_train_[idx])
        y_sub = self.y_train_[idx]
        beta = np.linalg.pinv(X_sub) @ y_sub
        x0_design = self._add_intercept(x0.reshape(1, -1))
        y_pred[i] = float(x0_design @ beta)
    return y_pred

@property
def is_fitted(self) -> bool:
    return self.is_fitted_

@property
def model_name(self) -> str:
    return "mcllr_no_stochasticity"

@property
def hyperparameters(self) -> dict:
    return {
        "bandwidth": self.bandwidth,
        "subsample_size": self.subsample_size,
        "fit_intercept": self.fit_intercept,
    }

```

src\models\ridge_linear.py

```

from __future__ import annotations

import numpy as np

from .base import LocalRiskModel

class RidgeLinearRegression(LocalRiskModel):
    """Regularized linear baseline (closed-form ridge regression)."""

    def __init__(self, alpha: float = 1.0, fit_intercept: bool = True) -> None:
        if alpha < 0:
            raise ValueError("alpha must be non-negative.")
        self.alpha = float(alpha)
        self.fit_intercept = fit_intercept

        self.coef_: np.ndarray | None = None
        self.intercept_: float | None = None
        self.n_features_: int | None = None
        self.is_fitted_: bool = False

    def _add_intercept(self, X: np.ndarray) -> np.ndarray:
        if not self.fit_intercept:
            return X
        return np.hstack((np.ones((X.shape[0], 1)), X))

    def fit(self, X: np.ndarray, y: np.ndarray) -> "RidgeLinearRegression":
        if X.ndim != 2 or y.ndim != 1:
            raise ValueError("X must be 2D and y must be 1D.")
        if X.shape[0] != y.shape[0]:
            raise ValueError("X and y must have same number of samples.")
        if X.shape[0] == 0:
            raise ValueError("Cannot fit on empty dataset.")

        self.n_features_ = X.shape[1]
        X_design = self._add_intercept(X)

        p = X_design.shape[1]
        reg = self.alpha * np.eye(p)
        if self.fit_intercept:
            reg[0, 0] = 0.0

        beta = np.linalg.pinv(X_design.T @ X_design + reg) @ X_design.T @ y
        if self.fit_intercept:
            self.intercept_ = float(beta[0])
            self.coef_ = beta[1:]
        else:
            self.intercept_ = 0.0
            self.coef_ = beta

        self.is_fitted_ = True
        return self

    def predict(self, X: np.ndarray) -> np.ndarray:
        if not self.is_fitted_:
            raise RuntimeError("Model must be fitted before prediction.")
        if X.ndim != 2 or X.shape[1] != self.n_features_:
            raise ValueError("Input has invalid shape.")

        if self.fit_intercept:
            beta = np.concatenate([self.intercept_, self.coef_])
        else:
            beta = self.coef_
        return self._add_intercept(X) @ beta

```

```

@property
def is_fitted(self) -> bool:
    return self.is_fitted_

@property
def model_name(self) -> str:
    return "ridge_linear"

@property
def hyperparameters(self) -> dict:
    return {"alpha": self.alpha, "fit_intercept": self.fit_intercept}

```

src\models\utils.py

```

import numpy as np

def add_intercept(X: np.ndarray, fit_intercept: bool) -> np.ndarray:
    """Shared intercept-addition logic."""
    if not fit_intercept:
        return
    ones = np.ones((X.shape[0],1),dtype=X.dtype)
    return np.hstack((ones, X))

def gaussian_kernel(X: np.ndarray, x0:np.ndarray, bandwidth: float) -> np.ndarray:
    """Shared Gaussian Kernel."""
    diff = X - x0
    dist_sq = np.sum(diff**2, axis=1)
    return np.exp(-dist_sq / (2 * bandwidth ** 2))

def mormalize_weights(weights: np.ndarray, eps: float = 1e-10) -> np.ndarray:
    """Normalize weights to probabilities."""
    w_sum = np.sum(weights)
    if w_sum < eps:
        w_sum = eps
    return weights / w_sum

```

src\utils\kernels.py

```

"""
Kernels utilities

Kernel classes and functions used by local models (e.g., Gaussian,
Epanechnikov). This module is intentionally independent of any specific
model so that kernels can be swapped or extended without modifying the
algorithms themselves.

The original kernel implementations lived in `src/models/kernels.py`; they
have been consolidated here and the models module removed.
"""

from __future__ import annotations
from abc import ABC, abstractmethod
import numpy as np
from typing import Tuple

class Kernel(ABC):
    """Base class for kernel functions."""

    @abstractmethod
    def __call__(self, X: np.ndarray, x0: np.ndarray) -> np.ndarray:
        """Compute weights for each row of ``X`` relative to ``x0``."""

```

```

        pass

    @property
    @abstractmethod
    def name(self) -> str:
        """Human-readable identifier for the kernel."""
        pass

class GaussianKernel(Kernel):
    def __init__(self, bandwidth: float):
        if bandwidth <= 0:
            raise ValueError("bandwidth must be positive")
        self.bandwidth = float(bandwidth)

    def __call__(self, X: np.ndarray, x0: np.ndarray) -> np.ndarray:
        diff = X - x0
        dist_sq = np.sum(diff ** 2, axis=1)
        return np.exp(-dist_sq / (2 * self.bandwidth ** 2))

    @property
    def name(self) -> str:
        return f"gaussian_bw{self.bandwidth}"

class EpanechnikovKernel(Kernel):
    def __init__(self, bandwidth: float):
        if bandwidth <= 0:
            raise ValueError("bandwidth must be positive")
        self.bandwidth = float(bandwidth)

    def __call__(self, X: np.ndarray, x0: np.ndarray) -> np.ndarray:
        diff = X - x0
        dist = np.sqrt(np.sum(diff ** 2, axis=1))
        u = dist / self.bandwidth
        return np.maximum(0, 0.75 * (1 - u**2)) / self.bandwidth

    @property
    def name(self) -> str:
        return f"epanechnikov_bw{self.bandwidth}"

```

src\utils\plotting.py

```

"""
Plotting helpers

Common plotting utilities for figures used in notebooks and reports.
"""

import matplotlib.pyplot as plt
import numpy as np
from typing import Dict, Optional, Any

def plot_predictions_vs_true(y_true: np.ndarray, y_pred: np.ndarray, save_path: Optional[str] = None):
    """
    Scatter plot of predictions vs true values with perfect fit line.
    """
    plt.figure(figsize=(8, 6))
    plt.scatter(y_true, y_pred, alpha=0.6, edgecolors='k', s=50)
    min_val = min(y_true.min(), y_pred.min())
    max_val = max(y_true.max(), y_pred.max())
    plt.plot([min_val, max_val], [min_val, max_val], 'r--', linewidth=2, label='Perfect Fit')
    plt.xlabel('True Values')

```

```

plt.ylabel('Predicted Values')
plt.title('Predictions vs True Values')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
if save_path:
    plt.savefig(save_path, dpi=300, bbox_inches='tight')
plt.close()

def plot_uncertainty(y_pred: np.ndarray, y_std: np.ndarray, save_path: Optional[str] = None):
    """
    Error bar plot showing predictions with uncertainty.
    """
    plt.figure(figsize=(10, 6))
    indices = np.arange(len(y_pred))
    plt.errorbar(indices, y_pred, yerr=y_std, fmt='o', alpha=0.7, capsize=3, elinewidth=1)
    plt.xlabel('Sample Index')
    plt.ylabel('Prediction')
    plt.title('Predictions with Uncertainty (95% CI)')
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.close()

def plot_kernel_weights(distances: np.ndarray, weights: np.ndarray, kernel_name: str, save_path: Optional[str] =
None):
    """
    Plot kernel weights as a function of distance.
    """
    plt.figure(figsize=(8, 6))
    plt.plot(distances, weights, 'b-', linewidth=2, marker='o', markersize=4)
    plt.xlabel('Distance')
    plt.ylabel('Weight')
    plt.title(f'{kernel_name} Kernel Weights')
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.close()

def plot_model_comparison(results: Dict[str, Any], metric: str = 'r2', save_path: Optional[str] = None):
    """
    Bar plot comparing models on a given metric.

    Args:
        results: Dict of model_name -> EvaluationResult object with .metrics dict
        metric: Metric key to plot (e.g., 'r2', 'rmse')
    """
    models = list(results.keys())
    values = [results[m].metrics.get(metric, np.nan) for m in models]

    plt.figure(figsize=(10, 6))
    bars = plt.bar(models, values, alpha=0.8, edgecolor='k')
    plt.xlabel('Model')
    plt.ylabel(metric.upper().replace('_', ' '))
    plt.title(f'Model Comparison: {metric.upper().replace("_", " ")}')
    plt.xticks(rotation=45, ha='right')
    plt.grid(True, axis='y', alpha=0.3)

    # Add value labels on bars
    for bar, val in zip(bars, values):
        if not np.isnan(val):

```

```

        plt.text(bar.get_x() + bar.get_width()/2, bar.get_height(),
                  f'{val:.3f}', ha='center', va='bottom', fontweight='bold')

plt.tight_layout()
if save_path:
    plt.savefig(save_path, dpi=300, bbox_inches='tight')
plt.close()

def plot_time_series(y: np.ndarray, dates: Optional[np.ndarray] = None, title: str = 'Time Series', save_path:
Optional[str] = None):
    """
    Plot time series data.
    """
    plt.figure(figsize=(12, 6))
    if dates is not None:
        plt.plot(dates, y, 'b-', linewidth=1)
        plt.xlabel('Date')
    else:
        plt.plot(y, 'b-', linewidth=1)
        plt.xlabel('Time Step')
    plt.ylabel('Value')
    plt.title(title)
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.close()

def plot_residuals(y_true: np.ndarray, y_pred: np.ndarray, save_path: Optional[str] = None):
    """
    Plot residuals (true - pred) vs predicted values.
    """
    residuals = y_true - y_pred
    plt.figure(figsize=(8, 6))
    plt.scatter(y_pred, residuals, alpha=0.6, edgecolors='k', s=50)
    plt.axhline(y=0, color='r', linestyle='--', linewidth=2)
    plt.xlabel('Predicted Values')
    plt.ylabel('Residuals')
    plt.title('Residual Plot')
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.close()

```

src\utils\sampling.py

```

"""
Sampling helpers

Subsampling and Monte Carlo sampling utilities. Provides a thin wrapper
around NumPy's random choice so that the sampling strategy is centralized
and easy to test. This module currently defines:

* ``subsample_indices`` - select a fixed-size random subset of indices,
  optionally weighted and/or without replacement.

Future work may add stratified, bootstrap, or kernel-based sampling helpers.
"""

from __future__ import annotations
import numpy as np

```

```

from typing import Optional

def subsample_indices(
    n: int,
    k: int,
    replace: bool = False,
    weights: Optional[np.ndarray] = None,
    random_state: Optional[int] = None,
) -> np.ndarray:
    """Return ``k`` random indices from ``0..n-1``.

    Parameters
    -----
    n : int
        Population size.
    k : int
        Number of indices to draw.
    replace : bool, default=False
        Whether to sample with replacement.
    weights : array-like, optional
        Probabilities for each element. Must sum to 1 if provided.
    random_state : int or None
        Seed for reproducibility.

    Returns
    -----
    np.ndarray
        Array of shape (k,) of selected indices.
    """
    if k < 0 or k > n:
        raise ValueError("k must be between 0 and n")

    rng = np.random.default_rng(random_state)

    if weights is not None:
        weights = np.asarray(weights, dtype=float)
        if weights.shape != (n,):
            raise ValueError("weights must have length n")
        total = weights.sum()
        if total <= 0:
            raise ValueError("weights must sum to positive value")
        probs = weights / total
        return rng.choice(n, size=k, replace=replace, p=probs)
    else:
        return rng.choice(n, size=k, replace=replace)

```

experiments\config_comparison.yaml

```

dataset_path: data/raw/F-F_Research_Data_5_Factors_2x3.csv
train_ratio: 0.8
random_state: 42
models:
  - model_type: global_linear
    fit_intercept: true
  - model_type: ridge_linear
    fit_intercept: true
    alpha: 1.0
  - model_type: lwlr
    fit_intercept: true
    bandwidth: 1.0
  - model_type: mc_linear
    fit_intercept: true

```

```

n_simulations: 50
subsample_size: 80
random_state: 42
- model_type: mcllr
  fit_intercept: true
  bandwidth: 1.0
  n_simulations: 50
  subsample_size: 80
  random_state: 42
- model_type: mcllr_no_geometry
  fit_intercept: true
  n_simulations: 50
  subsample_size: 80
  random_state: 42
- model_type: mcllr_no_stochasticity
  fit_intercept: true
  bandwidth: 1.0
  subsample_size: 80

```

experiments\config_global.yaml

```

# Global linear baseline configuration (placeholder)
dataset: data/raw/F-F_Research_Data_5_Factors_2x3.csv
model: global_linear
params: {}

```

experiments\config_lwlr.yaml

```

# LWLR experiment configuration (placeholder)
dataset: data/raw/F-F_Research_Data_5_Factors_2x3.csv
model: lwlr
params: {}

```

experiments\config_mc.yaml

```

# Monte Carlo experiment configuration (placeholder)
dataset: data/raw/F-F_Research_Data_5_Factors_2x3.csv
model: mc_linear
params: {}

```

experiments\config_mcllr.yaml

```

# MCLLR proposed experiment configuration (placeholder)
dataset: data/raw/F-F_Research_Data_5_Factors_2x3.csv
model: mcllr
params: {}

```

scripts\peek_csv.py

```

import sys

path='data/raw/F-F_Research_Data_5_Factors_2x3.csv'
with open(path,'r') as f:
    for i in range(10):
        line=f.readline()
        sys.stdout.write(f"{i}: {line!r}\n")

```

figures\rolling_folds_results.csv

```
fold_id,train_start,train_end,test_start,test_end,model,rmse,mae,mape,r2,explained_variance,median_ae,coverage_95,avg_std,
interval_width_95,gaussian_nll,interval_score_95
0,0,240,240,264,global_linear,0.04631930876755509,0.036534900215729076,1.1735561674024693,-0.016759384988811465,-0.01
1973929214618373,0.03147578762932232,,,,,
1,24,264,264,288,global_linear,0.05018124620887595,0.04044826414020616,0.9661087530659006,-0.00427377451461175,-0.003
524321884371018,0.0333151764697075,,,,,
2,48,288,288,312,global_linear,0.04482745854317079,0.035718871989604,0.9642180674019568,-0.010543495395518576,0.01800
6196777239136,0.03169039455012504,,,,,
3,72,312,312,336,global_linear,0.03879035053269408,0.02980922086907287,1.1093890242119544,-0.008918582166910438,-0.00
6979873314306717,0.01813063741130158,,,,,
4,96,336,336,360,global_linear,0.06713320168494337,0.04813541128333241,1.0373543602749236,0.038529947308511225,0.0398
0652525261863,0.037356512616424524,,,,,
5,120,360,360,384,global_linear,0.03446908547207755,0.02943812447686282,1.0880223879964812,-0.18023977328498408,-0.04
2599527624082256,0.0292353013398354,,,,,
6,144,384,384,408,global_linear,0.042213675957208395,0.03402168444164444,1.087306153832743,0.15756842741681643,0.1617
260693424678,0.028554406651813247,,,,,
7,168,408,408,432,global_linear,0.03047247245499651,0.020585199297255705,1.2841998128799446,-0.137992941308954,-0.106
78244113421864,0.013757096576551235,,,,,
8,192,432,432,456,global_linear,0.026302892107949036,0.022358377147166857,0.9157353292002224,-0.03631279374936014,-0.
01998218764508053,0.020287628926438067,,,,,
9,216,456,456,480,global_linear,0.03873721849752939,0.032671771191134645,1.0648288709632683,-0.12129710745325739,-0.0
7417146851515155,0.028794126155894585,,,,,
10,240,480,480,504,global_linear,0.056548616469334304,0.04731862999361732,1.0831982786504162,-0.21002266125972202,-0.
16190905001040123,0.04694255100857593,,,,,
11,264,504,504,528,global_linear,0.06305503425507114,0.055193956215546326,1.114690201366118,-0.184310741643678,-0.056
718815136730605,0.051145516288456094,,,,,
12,288,528,528,552,global_linear,0.05008797073839048,0.03976041612596225,0.9749130771229755,-0.03550784185957134,-0.0
20146031839567646,0.039580835398713515,,,,,
13,312,552,552,576,global_linear,0.024985225288976502,0.02109308543465845,1.7025415566533029,-0.04476589971899991,-0.
029971066118091416,0.022796128674073178,,,,,
14,336,576,576,600,global_linear,0.023968155214191367,0.019642560115388702,1.3491818444561847,-0.10293687443650024,-0.
09204354931137404,0.017184307990714298,,,,,
15,360,600,600,624,global_linear,0.07064802761491777,0.05671098560018035,1.1615204502069416,-0.10444732609149754,-0.0
4395293715383519,0.04569549845284504,,,,,
16,384,624,624,648,global_linear,0.05093161415333817,0.041894840588036525,1.1562490263432508,0.00851653736619773,0.01
706391045188993,0.030786109684866465,,,,,
17,408,648,648,672,global_linear,0.030661221915886026,0.025926673922060318,0.902768718648626,-0.20381401472818306,-0.
04222231143316035,0.02586159401405017,,,,,
18,432,672,672,696,global_linear,0.032737623715637765,0.026065829977286237,1.5127754912028022,-0.07475013025640576,-0.
07437379565618563,0.022553239068330552,,,,,
19,456,696,696,720,global_linear,0.02716734563670621,0.019685737992065033,3.0149809989600427,-0.22888149580565775,-0.
18061806871118558,0.013830071107313203,,,,,
20,480,720,720,744,global_linear,0.04380065170477064,0.03303917976047215,1.2736635908366292,-0.039807524531074945,-0.
03941853140702478,0.027140138375656327,,,,,
21,504,744,744,768,global_linear,0.06033457017306479,0.04700172783119069,1.5181068919560918,-0.09701582705071576,-0.0
3855488122305073,0.03387542353234661,,,,,
22,528,768,768,792,global_linear,0.06133886988647043,0.05204691228489785,1.2686617051681206,-0.17274241437417315,-0.1
497614265622773,0.03983050540056738,,,,,
23,552,792,792,816,global_linear,0.03200337467521524,0.026030488500272442,1.3830372617988962,0.018884351694807622,0.0
4470675948488767,0.02423466563858888,,,,,
0,0,240,240,264,ridge_linear,0.04612088887899357,0.03635026677609042,1.1863905274669153,-0.008066976702904682,-0.0044
29446738956955,0.0308141078641739,,,,,
1,24,264,264,288,ridge_linear,0.049939409352048866,0.03985919059139876,0.9609325173851913,0.005382629124742744,0.0061
26538726428765,0.032428287003764635,,,,,
2,48,288,288,312,ridge_linear,0.0449257088349357,0.03565144357278432,0.9400467653835639,-0.01497805409812103,0.008801
544925701732,0.032892892768371734,,,,,
3,72,312,312,336,ridge_linear,0.03873054864766771,0.02956835153953748,1.0861924979916104,-0.005810142890575465,-0.002
609435901275381,0.018288426866054256,,,,,
4,96,336,336,360,ridge_linear,0.06806379731941999,0.04845770977731395,0.9139556504073484,0.011689539854238373,0.01215
263135354705,0.036516063084075304,,,,,
5,120,360,360,384,ridge_linear,0.034192921227971115,0.028581186517255646,1.027635709733831,-0.16140351381080587,-0.02
1863142279208603,0.026511717721580327,,,,,
6,144,384,384,408,ridge_linear,0.04552029398361131,0.037208963833818894,1.0603858740763343,0.0204233887241988,0.02307
```

9829903302373,0.028087147523122566,,,,,
7,168,408,408,432,ridge_linear,0.029461151859238217,0.020227917736103444,1.1702358713068044,-0.06371094359500029,-0.0
26266314913564015,0.011207609000975286,,,,,
8,192,432,432,456,ridge_linear,0.026315662185271007,0.022453615076758377,0.08093429973427552,-0.037319299570929276,-0.
009414324504670502,0.023142473672063357,,,,,
9,216,456,456,480,ridge_linear,0.037609003666517,0.031477029288776444,0.9075439690530319,-0.056933082942573154,0.0048
18573472955334,0.028072915442354966,,,,,
10,240,480,480,504,ridge_linear,0.05182166321994318,0.041573205343067056,1.0624995567014095,-0.016183716255262492,-0.
0030371609086248252,0.03720111583378978,,,,,
11,264,504,504,528,ridge_linear,0.06039959639806503,0.05253062397159919,1.0076595200871536,-0.08666132270818316,0.014
374102881752893,0.0545146809842514,,,,,
12,288,528,528,552,ridge_linear,0.049481493971636856,0.03936822915600518,0.9291224713388555,-0.010583318683916598,0.0
08878050050580844,0.030214969709227997,,,,,
13,312,552,552,576,ridge_linear,0.024478379243074194,0.02084854266770329,1.0573275128298454,-0.0028079485097234347,-0
.0010794302041921,0.022644423174698024,,,,,
14,336,576,576,600,ridge_linear,0.02297757614296772,0.01805400266680202,1.114806596805517,-0.013654301531615287,-0.00
63328414565611,0.012793101254626552,,,,,
15,360,600,600,624,ridge_linear,0.06893316534342227,0.0548029207290614,1.0414437624351187,-0.05148085042064876,0.0002
491609387950122,0.0425137770860541,,,,,
16,384,624,624,648,ridge_linear,0.0513799254147926,0.042079303622802655,1.054299676243956,-0.00901479254323001,-0.002
7330050430376396,0.030230086482300286,,,,,
17,408,648,648,672,ridge_linear,0.030833630410853657,0.02589199320877142,0.8530162080774252,-0.21739020455575453,-0.0
0885693449965319,0.0241545105701591,,,,,
18,432,672,672,696,ridge_linear,0.03207507144147339,0.025639018697001432,1.6203647154427745,-0.03168820611184198,-0.0
3105439099125018,0.020414652015822196,,,,,
19,456,696,696,720,ridge_linear,0.025563046286353412,0.018276959458002832,2.161016322587807,-0.08802982744744936,-0.0
2615642452083855,0.012519568337869331,,,,,
20,480,720,720,744,ridge_linear,0.04339624075084538,0.032929787598740566,1.277811278584953,-0.020695102387751296,-0.0
19088169851064762,0.02718862245379293,,,,,
21,504,744,744,768,ridge_linear,0.059513714745950026,0.04580590015493587,1.585077584686293,-0.06736895053798397,-0.00
9893078458596438,0.03325225267808759,,,,,
22,528,768,768,792,ridge_linear,0.05776874306566473,0.04932881533129052,1.0870796240313674,-0.040200189410883436,-0.0
1736585383588068,0.04241010070871602,,,,,
23,552,792,792,816,ridge_linear,0.0329461258690398,0.027151887262089846,1.1410294618390175,-0.03977017806258609,0.008
411093587598839,0.020321234591032633,,,,,
0,0,240,240,264,lwlr,0.04631768024862537,0.03652839271076117,1.1736310813035802,-0.01668789070957799,-0.0118946482287
3143,0.03148609564364192,,,,,
1,24,264,264,288,lwlr,0.05017640023802944,0.040442973058053826,0.9640906421880996,-0.00407981972812399,-0.00330012199
54433836,0.03329027146078533,,,,,
2,48,288,288,312,lwlr,0.044827943179316425,0.035711658688393245,0.9625239190222304,-0.01056534577756363,0.0181221533
35517033,0.03170229308383865,,,,,
3,72,312,312,336,lwlr,0.03878776093192679,0.02981316601738926,1.110084598676145,-0.008783878092918584,-0.006792350735
340236,0.018161027715669995,,,,,
4,96,336,336,360,lwlr,0.06713814453219233,0.04813574882687734,1.036322914756955,0.03838836089704323,0.039647087755241
62,0.037320349922292384,,,,,
5,120,360,360,384,lwlr,0.03446853554170399,0.029435590392285955,1.0878288199595625,-0.1802021137699168,-0.04278633506
8899506,0.02923178759139939,,,,,
6,144,384,384,408,lwlr,0.042208558572049006,0.03401563111026617,1.0866038130920892,0.1577726638603204,0.1619670617487
3565,0.028545860268589997,,,,,
7,168,408,408,432,lwlr,0.030471703685192156,0.02059231992476554,1.284171314692876,-0.13793552269439457,-0.10664937094
146332,0.01372186183884406,,,,,
8,192,432,432,456,lwlr,0.026299938102585834,0.022350841773100556,0.9152323500537012,-0.03608003594945486,-0.019889496
584573285,0.020280167038643524,,,,,
9,216,456,456,480,lwlr,0.03873064318456391,0.032669292527353934,1.064525828890515,-0.1209164784895378,-0.073740740194
9568,0.028811565004250243,,,,,
10,240,480,480,504,lwlr,0.05656328237619919,0.04732745747223868,1.0828767069347223,-0.2106503823955903,-0.16235950084
149708,0.04692371055225978,,,,,
11,264,504,504,528,lwlr,0.06310900612252653,0.055244221540371986,1.116645249925548,-0.18633902753761755,-0.0575206982
64596826,0.05097191242263784,,,,,
12,288,528,528,552,lwlr,0.05009812466492305,0.039756650568224215,0.9742831810298096,-0.03592772456400639,-0.020826262
668143247,0.03937715912939069,,,,,
13,312,552,552,576,lwlr,0.024996647900741575,0.021098611664307914,1.711922688334363,-0.04572139906477379,-0.030607618
620725807,0.022815530563656317,,,,,
14,336,576,576,600,lwlr,0.023970214008509783,0.019642990667830956,1.3490756927788017,-0.103126360667249,-0.0921313324
9929029,0.017116037695637933,,,,,
15,360,600,600,624,lwlr,0.0706626150153165,0.05671432044221969,1.1615586788525383,-0.1049034656104535,-0.044307746401

3415,0.04572167994738967,,,,,
16,384,624,624,648,linear,0.05092561182616621,0.041893702340503304,1.156491375232729,0.00875021766674089,0.017233558426
08587,0.03076716034863234,,,,,
17,408,648,648,672,linear,0.030659043934932222,0.02592253383700931,0.9026606776229208,-0.20364299800340335,-0.042243001
116371026,0.02585678484093671,,,,,
18,432,672,672,696,linear,0.03273642442316239,0.026066328240701153,1.5169430647751476,-0.0746713880638652,-0.0742720687
5897796,0.02256183254010031,,,,,
19,456,696,696,720,linear,0.027170145323788284,0.019689434813666484,3.0190103575812643,-0.22913478966601253,-0.18100098
413504706,0.013820733409219714,,,,,
20,480,720,720,744,linear,0.04379925374315184,0.033029652925346514,1.2744672229638192,-0.03974115164589609,-0.039370460
96982311,0.027106853761965748,,,,,
21,504,744,744,768,linear,0.060325857302912816,0.04700287256883832,1.515017729035393,-0.0966990114577575,-0.03834155292
308483,0.03390607502450893,,,,,
22,528,768,768,792,linear,0.06133150594426055,0.05203558765687796,1.26731094133134,-0.17246084775935544,-0.149430979556
71747,0.03982509074255153,,,,,
23,552,792,792,816,linear,0.03201279355124348,0.02603988232808226,1.3802164298889126,0.018306764699091538,0.04415516337
174774,0.024181403679646517,,,,,
0,0,240,240,264,mc_linear,0.04625776548705891,0.03632240926819668,1.1110586598342247,-0.014059295588319864,-0.0073883
50731750037,0.031050431593362217,0.375,0.008100927609770558,0.03175563623030059,25.766640229662542,0.976072065344678
1,24,264,264,288,mc_linear,0.050007021492993985,0.04011008825072366,0.9693814872596981,0.002687613950989376,0.0031515
288264009067,0.032253327660058215,0.2083333333333334,0.009434662137552037,0.03698387557920399,17.254900459959654,1.0
205227902797402
2,48,288,288,312,mc_linear,0.044843514881473145,0.03537287191350192,0.9277174934640099,-0.011267539747510602,0.023205
015371639215,0.03142607122310166,0.2916666666666667,0.01125642094425993,0.04412517010149893,9.418875136387149,0.80159
08462731126
3,72,312,312,336,mc_linear,0.038762456007852726,0.029655612242343466,1.0983212980578287,-0.007468057229523595,-0.0044
63627619851751,0.018661401917960077,0.5416666666666666,0.01110782931459025,0.04354269091319377,4.147464033540232,0.59
03761393812905
4,96,336,336,360,mc_linear,0.06741665038306269,0.048575454458462446,1.048212295437432,0.030393801309688162,0.03259056
4179618364,0.03853909237838782,0.2916666666666667,0.01157340680772194,0.04536775468627,21.41291084074213,1.2294088093
548772
5,120,360,360,384,mc_linear,0.034714169508192436,0.02951684197519001,1.0891983052940877,-0.19708305602722787,-0.06033
3449151152685,0.028608681560396563,0.25,0.008645329528459917,0.033889691751562874,7.815934214820344,0.646218942834980
8
6,144,384,384,408,mc_linear,0.04158163285147094,0.03387041835385263,1.1050630633003988,0.1826061426144211,0.184641131
85730935,0.02817034689130811,0.4166666666666667,0.01215877875903016,0.04766241273539823,3.890640016032425,0.655188523
4999805
7,168,408,408,432,mc_linear,0.031218260807260357,0.021412866407565467,1.3039990727830644,-0.19437743831698628,-0.1593
7367905892486,0.011842433260509511,0.5833333333333334,0.008759843392392821,0.034338586098179864,3.411099324678803,0.4
4991092879226885
8,192,432,432,456,mc_linear,0.026342466669948526,0.022415667383317916,0.9310597092578553,-0.039433551511220744,-0.022
805808332627686,0.019713539257906233,0.25,0.006840914706744046,0.02681638565043666,3.903435419463191,0.46115393294011
24
9,216,456,456,480,mc_linear,0.03886804595590032,0.032894966569125085,1.0795525879670074,-0.12888382512590346,-0.08782
864794146295,0.02910496887705092,0.2916666666666667,0.010648860728479212,0.0417435340556385,3.179863036774671,0.6215
357158400832
10,240,480,480,504,mc_linear,0.057400569132370474,0.04820812887081687,1.1098712774218609,-0.24675733834359215,-0.1857
7881843683253,0.04803269290221898,0.2083333333333334,0.013876161772786786,0.054394554149324204,6.441016532377273,1.0
125785295616339
11,264,504,504,528,mc_linear,0.06217755124043853,0.054405786758871234,1.0873396446393662,-0.15157801362049916,-0.0333
396537917201,0.05071363851745924,0.4166666666666667,0.024923586243950305,0.0977004580762852,1.106568650175828,0.73839
18381600698
12,288,528,528,552,mc_linear,0.05089311328934573,0.04018632580507419,0.9971174536100471,-0.0690660934588434,-0.059536
54632935668,0.03336295896688436,0.4583333333333333,0.01594137747739259,0.06249019971137896,5.787316473933521,0.800350
0522946979
13,312,552,552,576,mc_linear,0.025213144585199576,0.021205607155803174,1.8412141710288743,-0.06391388826540179,-0.049
32067636779891,0.022578171442338618,0.4166666666666667,0.00893908354780232,0.03504120750738509,3.236698820774656,0.39
902064722865954
14,336,576,576,600,mc_linear,0.024017525734387722,0.01973038207753428,1.2770455109733547,-0.10748529699842857,-0.1055
1655598079646,0.016100987004203,0.375,0.007520345912334456,0.029479755976351066,2.573057507011461,0.38058344346697615
15,360,600,600,624,mc_linear,0.07070383482484756,0.056781149948314326,1.1570483439406782,-0.10619289408737775,-0.0468
26890011129274,0.044855650456336826,0.25,0.015540089495829842,0.060917150823652975,15.308856146375225,1.3028400327357
146
16,384,624,624,648,mc_linear,0.05088419407619648,0.041924981755426104,1.1868065157508854,0.010361926943804667,0.01863
9186688881537,0.03042239432551336,0.25,0.01212246754543677,0.04752007277811215,5.216670437664641,0.8785203814352621
17,408,648,648,672,mc_linear,0.030517700463629907,0.025955725781802965,0.9261358223532508,-0.19257057693623159,-0.043

027870068276464,0.027140165257653212,0.333333333333333,0.007335448202935742,0.0287549569555081,6.6548544394196325,0.5851441266906791

18,432,672,672,696,mc_linear,0.0325249853512763,0.025682725165544026,1.5806874154133197,-0.06083397529491097,-0.05846251257014945,0.02314769339247328,0.375,0.009300190072912828,0.03645674508581829,4.539162891127394,0.5223476987455872

19,456,696,696,720,mc_linear,0.027195984022182925,0.019602903826299114,2.8070870357553175,-0.23147370572213277,-0.1776265539490285,0.01356724160949074,0.625,0.010501485888126291,0.04116582468145505,-0.6141578004868773,0.31581116964557493

20,480,720,720,744,mc_linear,0.04368920800120462,0.032951751368932164,1.257632614641193,-0.03452301035033978,-0.03417125373568908,0.0276113978025421,0.375,0.011062226632936786,0.0433639284011122,13.710249149702369,0.7340761157364145

21,504,744,744,768,mc_linear,0.060068036870065906,0.046981855529529916,1.573135554889439,-0.08734490642016879,-0.03144216764387786,0.03877913667992074,0.333333333333333,0.01595014137168613,0.06252455417700963,4.282864983731469,0.8671605351100342

22,528,768,768,792,mc_linear,0.0626778426675799,0.052710838200825866,1.2762870490433784,-0.22450107811178,-0.20595773785144367,0.040969730933131054,0.333333333333333,0.01725577147516513,0.0676426241826473,4.3842033309988615,1.0650783523634406

23,552,792,792,816,mc_linear,0.03176428872073796,0.02606509910887381,1.4176800188078487,0.033488734825861344,0.06830846453249162,0.026662518868801437,0.5,0.013451023676442435,0.052728012811654346,0.04084782118638138,0.36923098247121233

0,0,240,240,264,mcllr,0.0466280036381835,0.03673191172706474,1.26018354151582,-0.030356923580641126,-0.025583862609224628,0.031491232238800015,0.375,0.008699600540459708,0.03410243411860206,22.967612609003094,0.9755290358805381

1,24,264,264,288,mcllr,0.050593911869602805,0.04041663923363194,0.9916667489066318,-0.020858987620300917,-0.020827705638079452,0.03287384257780698,0.25,0.011037855808796624,0.04326839477048277,13.004286683445079,0.9577882551695681

2,48,288,288,312,mcllr,0.04487354174758875,0.0356927768794092,0.9505437154722243,-0.012622266791170844,0.017200428546318758,0.03220405041086183,0.25,0.010779066605231383,0.04225394109250702,9.495795401989342,0.824049766351406

3,72,312,312,336,mcllr,0.0379087749308107,0.02879894044820512,1.0391514627620744,0.036419034585971954,0.038323254011661034,0.019225273614202835,0.5416666666666666,0.010309044973912542,0.04041145629773716,4.37001176758783,0.615736013195813

4,96,336,336,360,mcllr,0.06721890139863573,0.04780325011731995,1.0630117831600001,0.03607362766119426,0.037341205568110825,0.0375856723920122,0.2916666666666667,0.01128140100553063,0.04422309194176801,19.38428135999428,1.2140255614339468

5,120,360,360,384,mcllr,0.03474110431113674,0.029735605119252845,1.1104887310341962,-0.19894141562295609,-0.057827353302954076,0.028253831174639386,0.25,0.009100165210609891,0.035672647625590775,6.5153241439767156,0.6150588051990338

6,144,384,384,408,mcllr,0.042113235137488055,0.033713931199257696,1.079531383310252,0.16157252572413638,0.16716255438663996,0.02816785087938209,0.4166666666666667,0.011699062342268439,0.045860324381692276,5.201952913482628,0.7155788980715402

7,168,408,408,432,mcllr,0.030962247929026833,0.02129683877393104,1.3191895822727269,-0.17486820219305166,-0.13412121174346758,0.013870938680516844,0.5416666666666666,0.009511387133691611,0.03728463756407112,4.35144943218691,0.4374001378487953

8,192,432,432,456,mcllr,0.026325696328658867,0.022195674789134454,0.8912258610553456,-0.038110508617125616,-0.023118001041243952,0.019284229543406667,0.375,0.007843955644194456,0.030748306125242264,2.515593380053967,0.4249810908267125

9,216,456,456,480,mcllr,0.03830887746120723,0.03223041861787209,1.0613835259357522,-0.09663647811062681,-0.04992202601385998,0.02852830804404822,0.25,0.009684455711296051,0.03796306638828053,6.313388178197059,0.6630425962663299

10,240,480,480,504,mcllr,0.056289226898150205,0.04719459287435257,1.0991238131919603,-0.1989473274582092,-0.1532415125497757,0.047353900791782796,0.25,0.013465167092989333,0.05278345500451818,6.575226526151009,1.00558924588319

11,264,504,504,528,mcllr,0.061368870884143764,0.053560017307338836,1.0685487714293627,-0.12181799469704879,-0.006438140665227188,0.04736635110276332,0.458333333333333,0.027683774681800468,0.10852039675265784,0.6725581874462353,0.679780754384191

12,288,528,528,552,mcllr,0.05063003628023119,0.040099424517021705,0.9740151025467659,-0.05804221297174039,-0.0458578898213402,0.03720964928238607,0.458333333333333,0.01586086661246258,0.06217459712085329,6.772397106562065,0.7965792142297259

13,312,552,552,576,mcllr,0.024933786740183542,0.021197291012327035,1.9079963522796157,-0.04046848627039079,-0.02449599240791489,0.02296377718489729,0.458333333333333,0.009494573222943238,0.03721872703393749,2.586988408929059,0.3890316901978828

14,336,576,576,600,mcllr,0.024161793142711416,0.019647291079686973,1.3859015150571141,-0.1208300434236802,-0.10540829138326235,0.017768477544992653,0.375,0.007612670764923943,0.029841669398501858,1.946664874858646,0.36761322191094575

15,360,600,600,624,mcllr,0.07124790357700662,0.057381240938973226,1.202813577725885,-0.12328279025198619,-0.05938575436956839,0.04595887677670081,0.2916666666666667,0.014108613534604445,0.05530576505564943,20.081428083720823,1.4126716372948416

16,384,624,624,648,mcllr,0.05066197179350538,0.04177918009646956,1.1569870981061632,0.018986979076969335,0.02681845763158963,0.028567711815541262,0.2916666666666667,0.011361433843625054,0.0445368206670102,5.7255067171382095,0.9202146798274686

17,408,648,648,672,mcllr,0.030512873757280946,0.025835006386116532,0.9054172036739435,-0.19219337074533493,-0.039438520378972886,0.027366055390795965,0.25,0.007675156850366792,0.03008661485343782,5.763729894034828,0.5591861960480399

18,432,672,672,696,mcllr,0.03235978949011058,0.025981895428949336,1.8771778603241493,-0.050085294511064804,-0.049090794428973794,0.02342134511973709,0.375,0.009872118765009513,0.038698705558837296,3.24015818977783,0.4913959915348398

19,456,696,696,720,mcllr,0.026815449760679497,0.019931365240177055,2.9244320158876467,-0.1972525195118373,-0.14232601

499431974,0.01436714980468314,0.7083333333333334,0.010516963645733119,0.04122649749127382,-0.8442866622235656,0.28921099538772466
20,480,720,720,744,mcllr,0.04468681463605689,0.03345627365780613,1.3413232986052401,-0.08230735038975134,-0.08214327230543583,0.026296539908681776,0.375,0.01018641878642168,0.03993076164277299,20.04018556931815,0.7988686376731806
21,504,744,744,768,mcllr,0.061618686132459806,0.04824428659400876,1.5248956330532792,-0.14420888201994142,-0.08359112178346839,0.03897486289064443,0.2916666666666667,0.01479383407065008,0.057991829556948295,5.035634739513607,0.9630819934209928
22,528,768,768,792,mcllr,0.06159117476041641,0.052029765160558426,1.2011531461014768,-0.1824099275253226,-0.16118408294688535,0.043266984531689964,0.375,0.01718359985428276,0.06735971142878842,4.439293630586023,1.0129800604469041
23,552,792,792,816,mcllr,0.03242381719087771,0.02670392969949913,1.4609030789652628,-0.007063683544332511,0.01743079929948632,0.025614551183550802,0.5,0.013256199733064277,0.05196430295361196,0.12755924521350678,0.3878457826366899
0,0,240,240,264,mcllr_no_geometry,0.04626051485166902,0.036390749694398206,1.2215213314739806,-0.014179841903710466,-0.009420902191254354,0.03186272362987989,0.3333333333333333,0.00865842702877747,0.03394103395280768,23.829681589121645,0.9565990459904032
1,24,264,264,288,mcllr_no_geometry,0.05024461637038152,0.04029021221384657,1.0001939669484206,-0.006811821288408071,-0.006122011517803783,0.03305738165341979,0.2916666666666667,0.010525103427271079,0.041258405434902626,15.748383065209168,0.9810497008785809
2,48,288,288,312,mcllr_no_geometry,0.044887118463980906,0.035521290423849695,0.9445758129526887,-0.013235107434872662,0.020406827299852104,0.032035035283661514,0.2916666666666667,0.010461159145814102,0.041007743851591276,10.632477426806494,0.828227005732575
3,72,312,312,336,mcllr_no_geometry,0.03827802319286543,0.029541893852623336,1.121168105284623,0.01755620268490965,0.019233735312066735,0.018441601141089264,0.5,0.01004086067115853,0.039360173830941446,3.600437805196366,0.6040963980498629
4,96,336,336,360,mcllr_no_geometry,0.0673471098953465,0.04818925619636614,1.0270062271434621,0.03239307336022024,0.03320228696169647,0.036944485766526924,0.25,0.011236709758628146,0.04404790225382233,25.517783852987403,1.2206404487065947
5,120,360,360,384,mcllr_no_geometry,0.034338944980965014,0.02916153646236766,1.0833050392534211,-0.17134443861383586,-0.0302088171704753,0.028108530624883332,0.25,0.008655783742247163,0.03393067226960888,7.290504718261876,0.6240847042854727
6,144,384,384,408,mcllr_no_geometry,0.0420462627381454,0.034004354635009004,1.083900158823436,0.16423709664891195,0.16624990067365242,0.02865851680255821,0.4583333333333333,0.011524992155386085,0.04517796924911346,5.527828112259766,0.7071171882014844
7,168,408,408,432,mcllr_no_geometry,0.0305866763386521,0.02075336906830651,1.2760927505204458,-0.14653880183395795,-0.1085628804641765,0.011779187429951415,0.625,0.009635625000217096,0.03777165000085101,2.907327852091621,0.42444431465840865
8,192,432,432,456,mcllr_no_geometry,0.026238862962758697,0.02242277933757658,0.9293035600427331,-0.03127354110740366,-0.016487659611821526,0.018866085025127224,0.375,0.007722621233683791,0.030272675236040464,3.2142466707749677,0.422333513148226
9,216,456,456,480,mcllr_no_geometry,0.03811614288121156,0.03206989168269433,1.0534014425485674,-0.08562972968760607,-0.04625910454874704,0.026963570109729926,0.375,0.010489151534932612,0.041117474016935836,3.810648686865298,0.6216276823000436
10,240,480,480,504,mcllr_no_geometry,0.05666138887023004,0.04755908787281669,1.14284227119127,-0.21485366277354379,-0.16676777724374614,0.04491639386230419,0.2083333333333334,0.013463719078753135,0.05277777878871229,7.067813929602707,1.0038001630431055
11,264,504,504,528,mcllr_no_geometry,0.06392776240557224,0.05570709662672533,1.1258563894368345,-0.217321082503509,-0.07229827305748238,0.04803365976128661,0.375,0.027014471779677265,0.10589672937633487,0.6940496475822099,0.7147937792854769
12,288,528,528,552,mcllr_no_geometry,0.05022809646332282,0.03991116434154218,0.9783805908699877,-0.041309803967382486,-0.02919445936686227,0.038195895656291085,0.4583333333333333,0.015246947865257161,0.059768035631808075,5.545251100279674,0.794629226507736
13,312,552,552,576,mcllr_no_geometry,0.024949590873836067,0.021057008802123574,1.7698209015688757,-0.041787893912859086,-0.022838751340837682,0.021536155353199124,0.5416666666666666,0.008951152498286478,0.035088517793282994,3.8963549289494437,0.3952606964749171
14,336,576,576,600,mcllr_no_geometry,0.023886528185322692,0.019403424402036868,1.3328729913441073,-0.09543724377283236,-0.07830736675858163,0.01684291835757844,0.4166666666666667,0.007501776768392293,0.029406964932097792,2.463475989768144,0.3713839708693909
15,360,600,600,624,mcllr_no_geometry,0.07010163897399459,0.056194146987239325,1.1325959275346202,-0.08742989652113042,-0.027896387859568383,0.04681340814569314,0.25,0.014237906233007724,0.05581259243339028,18.474707367752192,1.3680847041455018
16,384,624,624,648,mcllr_no_geometry,0.051495859442789824,0.0423364070666529,1.1594923822472403,-0.013573426077775563,-0.003913172213609872,0.03245010590290086,0.25,0.010726142229328621,0.042046477538968195,8.305498048619407,1.0026083348255337
17,408,648,648,672,mcllr_no_geometry,0.030306053227277696,0.025744635107836727,0.9165784476090799,-0.17608643676398494,-0.02850964835470693,0.02627899246745395,0.2916666666666667,0.007612403142416929,0.029840620318274363,6.454084534238426,0.5576504722519724
18,432,672,672,696,mcllr_no_geometry,0.03283039069035906,0.02596653791759219,1.4522604991510697,-0.08084969193282121,

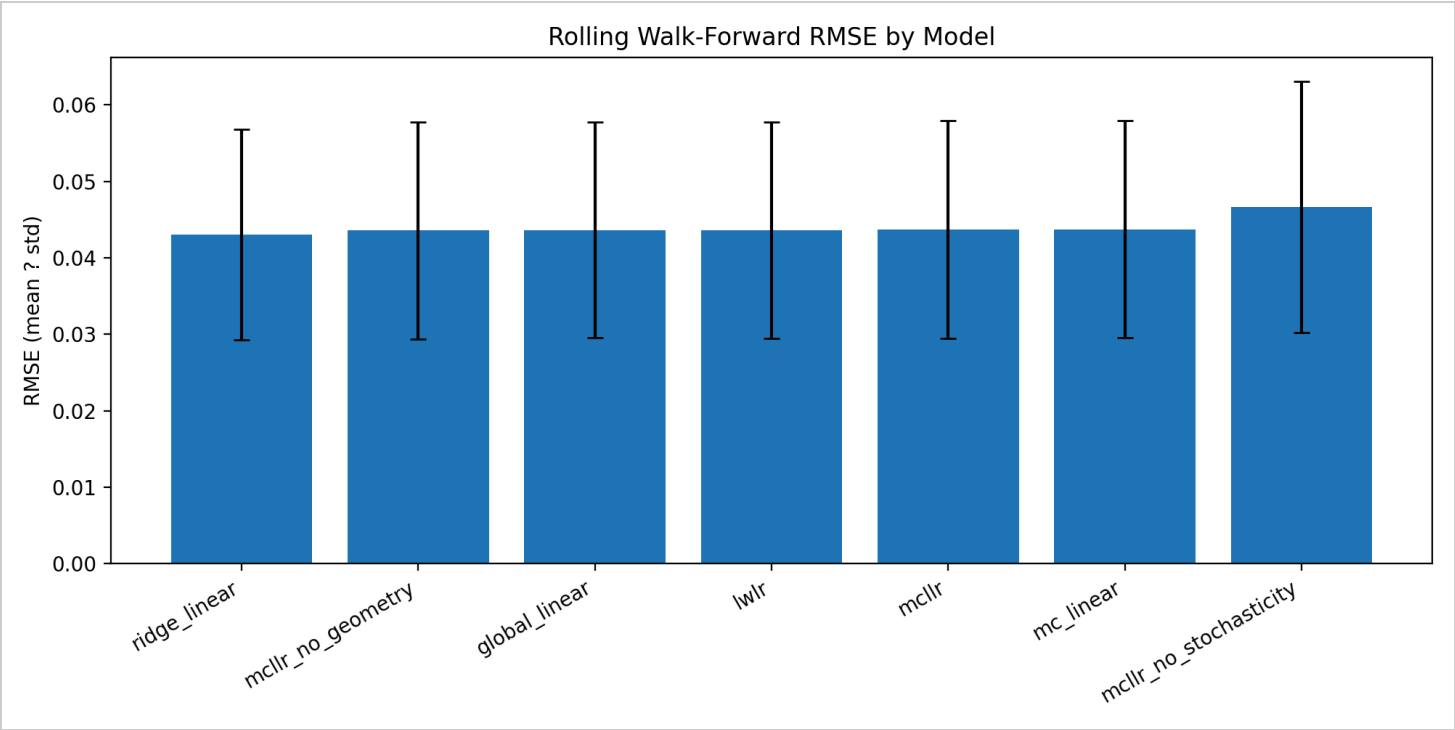
-0.07940993881586644,0.023400757490428636,0.375,0.009165191640938125,0.03592755123247745,3.866604832695023,0.520866467524019
19,456,696,696,720,mcllr_no_geometry,0.027044692532446537,0.019585796226598312,2.7818911526245045,-0.21781041888134367,-0.1634463154848973,0.011558015235505922,0.6666666666666666,0.009998886415721843,0.039195634749629615,-0.21190020158445058,0.30055211440699153
20,480,720,720,744,mcllr_no_geometry,0.04351502700524118,0.032385170492929514,1.208703703200528,-0.026290540341096458,-0.02626865627301478,0.027129979590982733,0.4583333333333333,0.010192085146605653,0.039952973774694166,15.97445969680694,0.7764413497718291
21,504,744,744,768,mcllr_no_geometry,0.05990558728136017,0.04692284579570614,1.5820434529380052,-0.0814715704901543,-0.02233207929846448,0.03497283676833746,0.375,0.016033229253205845,0.06285025867256692,3.7586395660926537,0.8743755694252447
22,528,768,768,792,mcllr_no_geometry,0.06092884882866183,0.05138362228394255,1.2750508179365383,-0.1571163696526665,-0.1292820728475148,0.04150380745941436,0.2916666666666667,0.01787811633230974,0.07008221602265419,3.3520335508194585,0.9624879235659355
23,552,792,792,816,mcllr_no_geometry,0.03155749467098312,0.026215926304276094,1.2404426494388539,0.04603226638099567,0.07865035433017387,0.02498795054079117,0.5416666666666666,0.013259609011941571,0.05197766732681095,0.038295541077583116,0.3633727415852013
0,0,240,240,264,mcllr_no_stochasticity,0.047588743724799876,0.03750030574860969,1.1511158401528636,-0.07325403701500943,-0.05772903702434884,0.031115032619551185,,,,,
1,24,264,264,288,mcllr_no_stochasticity,0.05155037875625407,0.042263908550753994,1.446715656760638,-0.05982206694292036,-0.014520244248172443,0.0337350379463699,,,,,
2,48,288,288,312,mcllr_no_stochasticity,0.049955707522492176,0.0399951440983251,1.1393797391038127,-0.2549805767684301,-0.1928093464611329,0.039037210859973,,,,,
3,72,312,312,336,mcllr_no_stochasticity,0.04225332630090105,0.03340216939025394,1.4714395093817956,-0.19710024480151533,-0.19094769840307757,0.028822950827848772,,,,,
4,96,336,336,360,mcllr_no_stochasticity,0.06778867172991122,0.04861649639430427,0.9725287012958921,0.01966323383386437,0.01976056667995041,0.03623306439220002,,,,,
5,120,360,360,384,mcllr_no_stochasticity,0.03429122974634375,0.029429484964070107,1.28342135491497,-0.1680914473846038,-0.0884738837125505,0.03044811457095479,,,,,
6,144,384,384,408,mcllr_no_stochasticity,0.044209587069276995,0.03551468751041367,1.2777649686285752,0.07602289989664446,0.07835758702330209,0.03290783822696809,,,,,
7,168,408,408,432,mcllr_no_stochasticity,0.03074542207266881,0.023174269383168286,1.6053560776161415,-0.1584708244844626,-0.151716935645136,0.017712069629795053,,,,,
8,192,432,432,456,mcllr_no_stochasticity,0.026773618140139566,0.02113279581895065,0.9194685621016051,-0.07373715784052148,-0.07369226779938076,0.016803451375172453,,,,,
9,216,456,456,480,mcllr_no_stochasticity,0.03892272474370369,0.030915021621148796,0.9732272680756354,-0.1320622414861099,-0.11289161002472414,0.024221589189695823,,,,,
10,240,480,480,504,mcllr_no_stochasticity,0.06009889042276677,0.04805802827495301,1.3304605655244532,-0.36672910807057857,-0.3076876187893458,0.03887115269038113,,,,,
11,264,504,504,528,mcllr_no_stochasticity,0.08385552098552956,0.06928496558099918,1.6823918308261085,-1.0945438622241146,-0.5433744358026682,0.05536904366968655,,,,,
12,288,528,528,552,mcllr_no_stochasticity,0.06008138406307925,0.046431115674851337,1.3824934807700557,-0.48993175180352466,-0.4381933449687281,0.04156397301424134,,,,,
13,312,552,552,576,mcllr_no_stochasticity,0.028042671051706717,0.024029025053123237,4.1085091975675985,-0.3161070176016447,-0.2104995291547378,0.024114454618252384,,,,,
14,336,576,576,600,mcllr_no_stochasticity,0.026052764504365198,0.02167172381125197,1.578218178881771,-0.3031339741028096,-0.29884782740878,0.01873139819859419,,,,,
15,360,600,600,624,mcllr_no_stochasticity,0.07519648227365182,0.058564650361447694,1.263087094326753,-0.2512381405793327,-0.1732552401019516,0.04764876307542883,,,,,
16,384,624,624,648,mcllr_no_stochasticity,0.05084755937216782,0.041654279807634316,1.626000118817134,0.011786418277788036,0.014016900072421956,0.0383158848695814,,,,,
17,408,648,648,672,mcllr_no_stochasticity,0.03187818728544335,0.025960138352262335,0.864108131366719,-0.30127088769454335,-0.14936149527259124,0.025135320100232353,,,,,
18,432,672,672,696,mcllr_no_stochasticity,0.03353169603183621,0.02742856705314838,2.8739459448016693,-0.12751996926070963,-0.11392807042357855,0.02411580368775494,,,,,
19,456,696,696,720,mcllr_no_stochasticity,0.030731935656485878,0.021846229990079753,3.134428619681693,-0.5725172529830189,-0.5274806902583564,0.011780442033659959,,,,,
20,480,720,720,744,mcllr_no_stochasticity,0.03905363547183386,0.028206058283507224,1.171463944712566,0.17336317641750887,0.1789271993036219,0.017394297229070106,,,,,
21,504,744,744,768,mcllr_no_stochasticity,0.06546969815977678,0.04685173914965513,2.232718937428318,-0.2916984082234604,-0.28378710918731764,0.027734971600382085,,,,,
22,528,768,768,792,mcllr_no_stochasticity,0.06479154902654381,0.05372300470737291,1.405421447832655,-0.3084821975368457,-0.24252381544957946,0.05397917743357965,,,,,
23,552,792,792,816,mcllr_no_stochasticity,0.03574314583814811,0.029522986427196498,1.1173294873255666,-0.22381056143033473,-0.16626910719421217,0.02672418878201887,,,,,

figures\rolling_summary_results.csv

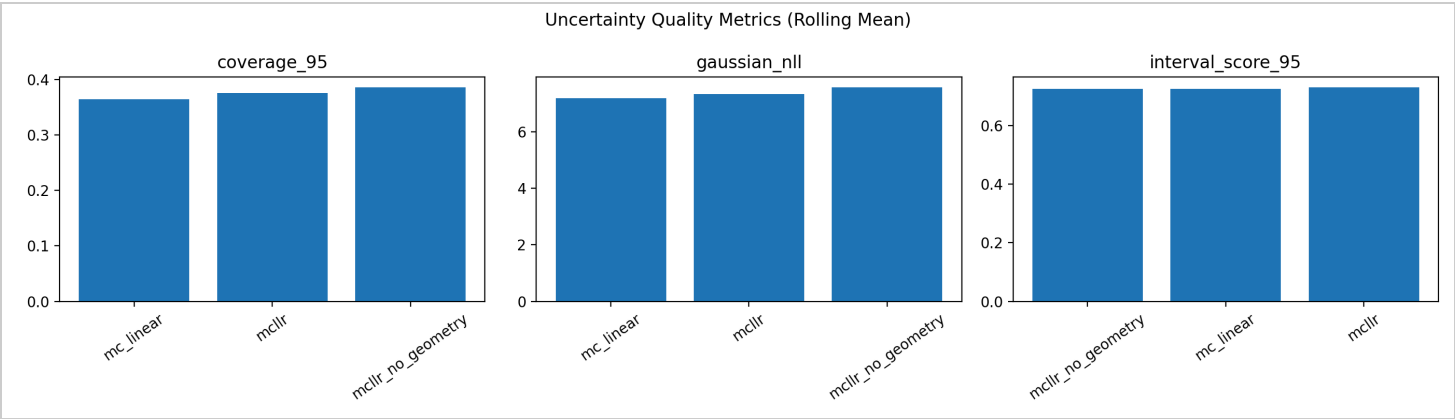
```
metric,mean,std,model
rmse,0.043654800486207124,0.014141747267291489,global_linear
mae,0.03504720205806849,0.011577465503878653,global_linear
mape,1.254458667525011,0.4263514895049818,global_linear
r2,-0.0746600558680106,0.09422765096181157,global_linear
explained_variance,-0.038099781357733765,0.07264672634751333,global_linear
median_ae,0.029752235540104633,0.010103907509773713,global_linear
rmse,0.043018656594156525,0.013761034896206604,ridge_linear
mae,0.03433820269073796,0.011028811688760801,ridge_linear
mape,1.1273006946599533,0.2930763668583425,ridge_linear
r2,-0.041927306878106675,0.05455235544141616,ridge_linear
explained_variance,-0.004303684511113311,0.014083646846745313,ridge_linear
median_ae,0.02826394736779018,0.01021384616919697,ridge_linear
rmse,0.04365782643141749,0.014145463198200012,lmnr
mae,0.03504832758731107,0.011580384182047772,lmnr
mape,1.2547289699551276,0.42765434655365764,lmnr
r2,-0.07479381768851939,0.09435432661567014,lmnr
explained_variance,-0.03818084357782068,0.07269150729085343,lmnr
median_ae,0.029729301842770395,0.01008508745902336,lmnr
rmse,0.04373916512602823,0.014192048046150315,mc_linear
mae,0.03510585242399697,0.011590926324968677,mc_linear
mape,1.256610516705184,0.40055050090850364,mc_linear
r2,-0.0795531383963181,0.10382321476733408,mc_linear
explained_variance,-0.04344437117148033,0.08574260946619883,mc_linear
median_ae,0.029721444699569267,0.010265016929064924,mc_linear
coverage_95,0.3645833333333333,0.11476846046239451,mc_linear
avg_std,0.011760265551864138,0.004052460783573815,mc_linear
interval_width_95,0.046100240963307414,0.015885646271609356,mc_linear
gaussian_nll,7.202915504002207,6.723752953330002,mc_linear
interval_score_95,0.7259630250077951,0.27641585089283865,mc_linear
rmse,0.04370735374150638,0.014198855367047075,mcllr
mae,0.03506906445409851,0.011506425824148235,mcllr
mape,1.2832110334322036,0.4407413336096714,mcllr
r2,-0.07659385411701007,0.09307143617664229,mcllr
explained_variance,-0.039988535372506996,0.07342462818429961,mcllr
median_ae,0.029915894286855187,0.00992810700898258,mcllr
coverage_95,0.375,0.11912523349835938,mcllr
avg_std,0.011792391101453794,0.00422911786215014,mcllr
interval_width_95,0.04622617311769887,0.016578142019628546,mcllr
gaussian_nll,7.345114182539306,6.752313850191415,mcllr
interval_score_95,0.7298850108800125,0.2899954609247131,mcllr
rmse,0.04357013467197391,0.01420501213755057,mcllr_no_geometry
mae,0.03494659182479402,0.01158979157843997,mcllr_no_geometry
mape,1.2433041905034705,0.3862008783244501,mcllr_no_geometry
r2,-0.06917219501607737,0.09462961647850983,mcllr_no_geometry
explained_variance,-0.031240965410082918,0.07241194148967164,mcllr_no_geometry
median_ae,0.029389083085791468,0.01020791194503136,mcllr_no_geometry
coverage_95,0.3854166666666667,0.12484894254300385,mcllr_no_geometry
avg_std,0.011676336295581603,0.004255126363426008,mcllr_no_geometry
interval_width_95,0.045771238278679884,0.016680095344629952,mcllr_no_geometry
gaussian_nll,7.573278679678084,7.173230503484414,mcllr_no_geometry
interval_score_95,0.7248551337697995,0.2845520020518315,mcllr_no_geometry
rmse,0.04664393874790943,0.016445688442965394,mcllr_no_stochasticity
mae,0.03688236650031173,0.012888275867932363,mcllr_no_stochasticity
mape,1.5837914440789582,0.7748476052001316,mcllr_no_stochasticity
r2,-0.22848608332536188,0.25238843566981495,mcllr_no_stochasticity
explained_variance,-0.16862196059379472,0.17552934760175515,mcllr_no_stochasticity
median_ae,0.030938134610058038,0.01146154223455779,mcllr_no_stochasticity
```

Figure Outputs

figures\rolling_rmse_by_model.png



figures\rolling_uncertainty_metrics.png



Notebooks (Code + Outputs)

notebooks\01_dataset_exploration.ipynb

Cell 0 - markdown

Dataset Exploration

This notebook demonstrates loading the Fama-French 5-factor dataset, inspecting its structure, and visualizing the factor returns over time. The helper function `load\_fama\_french` from the `src.data.loader` module is used so the same preprocessing logic as the rest of the project is applied.

Cell 1 - code

```
import sys, os
root = os.path.abspath(os.path.join(os.getcwd(), ".."))
print("notebook cwd", os.getcwd())
print("workspace root", root)
# ensure workspace root and src folder are in path
sys.path.insert(0, root)
sys.path.insert(0, os.path.join(root, "src"))
print("sys.path[0:3]", sys.path[0:3])

import pandas as pd
import matplotlib.pyplot as plt

from src.data.loader import load_fama_french
from src.utils.plotting import plot_time_series # optional helper
```

Output: stream

```
notebook cwd C:\Users\lovey\OneDrive\Desktop\local-risk-dynamics\notebooks
workspace root C:\Users\lovey\OneDrive\Desktop\local-risk-dynamics
sys.path[0:3] ['C:\\Users\\lovey\\OneDrive\\Desktop\\local-risk-dynamics\\src',
'C:\\Users\\lovey\\OneDrive\\Desktop\\local-risk-dynamics', 'C:\\Users\\lovey\\OneDrive\\Desktop\\local-risk-dynamics\\notebooks']
```

Cell 2 - code

```
# load dataset
csv_path = os.path.abspath(os.path.join(root, "data/raw/F-F_Research_Data_5_Factors_2x3.csv"))
X, y, dates = load_fama_french(csv_path)

df = pd.DataFrame(X, columns=["Mkt-RF", "SMB", "HML", "RMW", "CMA"])
df["Date"] = pd.to_datetime(dates, format="%Y%m")
df["ExcessRet"] = y

df.head()
```

Output: text/plain

| | Mkt-RF | SMB | HML | RMW | CMA | Date | ExcessRet |
|---|---------|---------|---------|--------|---------|------------|-----------|
| 0 | -0.0039 | -0.0048 | -0.0081 | 0.0064 | -0.0115 | 1963-07-01 | 0.0508 |
| 1 | 0.0508 | -0.0080 | 0.0170 | 0.0040 | -0.0038 | 1963-08-01 | -0.0157 |

| | | | | | | | |
|---|---------|---------|---------|---------|---------|------------|---------|
| 2 | -0.0157 | -0.0043 | 0.0000 | -0.0078 | 0.0015 | 1963-09-01 | 0.0254 |
| 3 | 0.0254 | -0.0134 | -0.0004 | 0.0279 | -0.0225 | 1963-10-01 | -0.0086 |
| 4 | -0.0086 | -0.0085 | 0.0173 | -0.0043 | 0.0227 | 1963-11-01 | 0.0183 |

Output: text/html

```

<div>
<style scoped>
  .dataframe tbody tr th:only-of-type {
    vertical-align: middle;
  }

  .dataframe tbody tr th {
    vertical-align: top;
  }

  .dataframe thead th {
    text-align: right;
  }
</style>
<table border="1" class="dataframe">
  <thead>
    <tr style="text-align: right;">
      <th></th>
      <th>Mkt-RF</th>
      <th>SMB</th>
      <th>HML</th>
      <th>RMW</th>
      <th>CMA</th>
      <th>Date</th>
      <th>ExcessRet</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th>0</th>
      <td>-0.0039</td>
      <td>-0.0048</td>
      <td>-0.0081</td>
      <td>0.0064</td>
      <td>-0.0115</td>
      <td>1963-07-01</td>
      <td>0.0508</td>
    </tr>
    <tr>
      <th>1</th>
      <td>0.0508</td>
      <td>-0.0080</td>
      <td>0.0170</td>
      <td>0.0040</td>
      <td>-0.0038</td>
      <td>1963-08-01</td>
      <td>-0.0157</td>
    </tr>
    <tr>
      <th>2</th>
      <td>-0.0157</td>
      <td>-0.0043</td>
      <td>0.0000</td>
      <td>-0.0078</td>
      <td>0.0015</td>
      <td>1963-09-01</td>
      <td>0.0254</td>
    </tr>
    <tr>

```

```

<th>3</th>
<td>0.0254</td>
<td>-0.0134</td>
<td>-0.0004</td>
<td>0.0279</td>
<td>-0.0225</td>
<td>1963-10-01</td>
<td>-0.0086</td>
</tr>
<tr>
<th>4</th>
<td>-0.0086</td>
<td>-0.0085</td>
<td>0.0173</td>
<td>-0.0043</td>
<td>0.0227</td>
<td>1963-11-01</td>
<td>0.0183</td>
</tr>
</tbody>
</table>
</div>

```

Cell 3 - code

```

# summary statistics and date range
print(df.describe())
print("Date range", df.Date.min(), "to", df.Date.max())

```

Output: stream

| | Mkt-RF | SMB | HML | RMW | CMA \ |
|-------|------------|------------|------------|------------|------------|
| count | 816.000000 | 816.000000 | 816.000000 | 816.000000 | 816.000000 |
| mean | 0.005947 | 0.002523 | 0.002917 | 0.002351 | 0.002227 |
| min | -0.231900 | -0.155400 | -0.138300 | -0.189500 | -0.070800 |
| 25% | -0.019125 | -0.014825 | -0.013525 | -0.008500 | -0.010500 |
| 50% | 0.009750 | 0.000800 | 0.002300 | 0.002150 | 0.000800 |
| 75% | 0.033375 | 0.020725 | 0.017000 | 0.012900 | 0.014725 |
| max | 0.161000 | 0.184600 | 0.128600 | 0.130500 | 0.090100 |
| std | 0.043668 | 0.030247 | 0.028918 | 0.021773 | 0.020649 |

| | Date | ExcessRet |
|------------|--|------------|
| count | 816 | 816.000000 |
| mean | 1992-04-06 01:40:35.294117632 | 0.005951 |
| min | 1963-07-01 00:00:00 | -0.231900 |
| 25% | 1974-10-24 06:00:00 | -0.019125 |
| 50% | 1991-10-16 12:00:00 | 0.009750 |
| 75% | 2008-10-08 18:00:00 | 0.033375 |
| max | 2025-10-01 00:00:00 | 0.161000 |
| std | NaN | 0.043667 |
| Date range | 1963-07-01 00:00:00 to 2025-10-01 00:00:00 | |

Cell 4 - code

```

# plot factor returns over time
fig, ax = plt.subplots(figsize=(10,6))
for col in ["Mkt-RF", "SMB", "HML", "RMW", "CMA"]:
    ax.plot(df.Date, df[col], label=col)
ax.set_title("Fama-French factor returns")
ax.set_xlabel("Date")

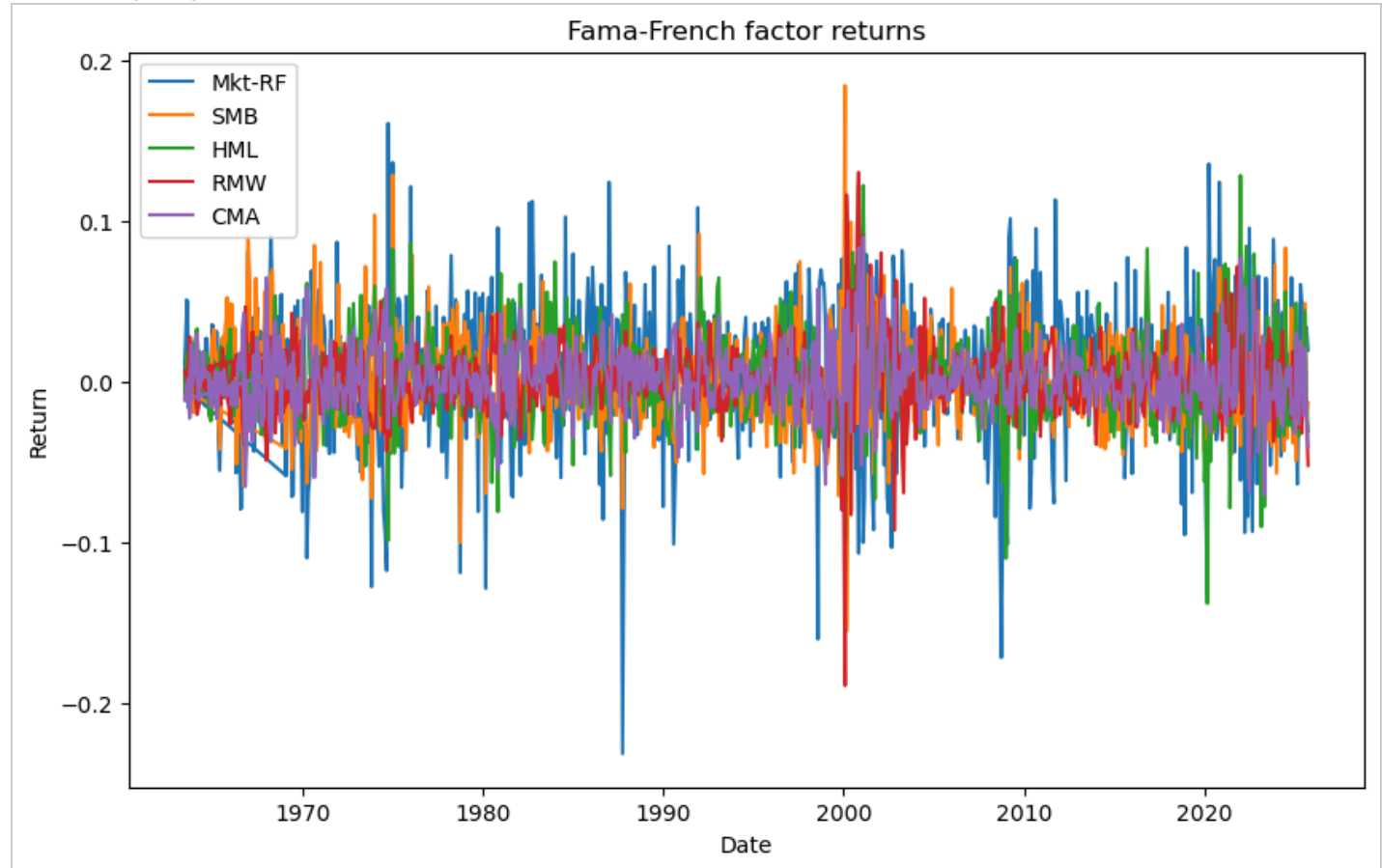
```

```
ax.set_ylabel("Return")
ax.legend()
plt.show()
```

Output: text/plain

<Figure size 1000x600 with 1 Axes>

Output: image/png



Cell 5 - markdown

```
## Next steps

* Use this data as input to the various regression models.
* Perform feature construction or normalization if desired.
* Run time-series splits before training.
```

notebooks\01_eda_fama_french.ipynb

Cell 0 - markdown

```
# Exploratory Data Analysis of Fama-French dataset
```

This notebook is intended for detailed exploratory analysis of the raw factor series: plots, correlations, seasonal effects, etc. (Content to be added.)

Cell 1 - code

```

import sys, os
root = os.path.abspath(os.path.join(os.getcwd(), ".."))
sys.path.insert(0, root)
sys.path.insert(0, os.path.join(root, "src"))

import pandas as pd
import matplotlib.pyplot as plt
from src.data.loader import load_fama_french

# load data
X, y, dates = load_fama_french('data/raw/F-F_Research_Data_5_Factors_2x3.csv')
df = pd.DataFrame(X, columns=['Mkt-RF', 'SMB', 'HML', 'RMW', 'CMA'])
df['Date'] = pd.to_datetime(dates, format='%Y%m')
df['ExcessRet'] = y

df.head()

```

Output: text/plain

| | Mkt-RF | SMB | HML | RMW | CMA | Date | ExcessRet |
|---|---------|---------|---------|---------|---------|------------|-----------|
| 0 | -0.0039 | -0.0048 | -0.0081 | 0.0064 | -0.0115 | 1963-07-01 | 0.0508 |
| 1 | 0.0508 | -0.0080 | 0.0170 | 0.0040 | -0.0038 | 1963-08-01 | -0.0157 |
| 2 | -0.0157 | -0.0043 | 0.0000 | -0.0078 | 0.0015 | 1963-09-01 | 0.0254 |
| 3 | 0.0254 | -0.0134 | -0.0004 | 0.0279 | -0.0225 | 1963-10-01 | -0.0086 |
| 4 | -0.0086 | -0.0085 | 0.0173 | -0.0043 | 0.0227 | 1963-11-01 | 0.0183 |

Output: text/html

```

<div>
<style scoped>
  .dataframe tbody tr th:only-of-type {
    vertical-align: middle;
  }

  .dataframe tbody tr th {
    vertical-align: top;
  }

  .dataframe thead th {
    text-align: right;
  }
</style>
<table border="1" class="dataframe">
  <thead>
    <tr style="text-align: right;">
      <th></th>
      <th>Mkt-RF</th>
      <th>SMB</th>
      <th>HML</th>
      <th>RMW</th>
      <th>CMA</th>
      <th>Date</th>
      <th>ExcessRet</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th>0</th>
      <td>-0.0039</td>
      <td>-0.0048</td>
      <td>-0.0081</td>

```

```

        <td>0.0064</td>
        <td>-0.0115</td>
        <td>1963-07-01</td>
        <td>0.0508</td>
    </tr>
    <tr>
        <th>1</th>
        <td>0.0508</td>
        <td>-0.0080</td>
        <td>0.0170</td>
        <td>0.0040</td>
        <td>-0.0038</td>
        <td>1963-08-01</td>
        <td>-0.0157</td>
    </tr>
    <tr>
        <th>2</th>
        <td>-0.0157</td>
        <td>-0.0043</td>
        <td>0.0000</td>
        <td>-0.0078</td>
        <td>0.0015</td>
        <td>1963-09-01</td>
        <td>0.0254</td>
    </tr>
    <tr>
        <th>3</th>
        <td>0.0254</td>
        <td>-0.0134</td>
        <td>-0.0004</td>
        <td>0.0279</td>
        <td>-0.0225</td>
        <td>1963-10-01</td>
        <td>-0.0086</td>
    </tr>
    <tr>
        <th>4</th>
        <td>-0.0086</td>
        <td>-0.0085</td>
        <td>0.0173</td>
        <td>-0.0043</td>
        <td>0.0227</td>
        <td>1963-11-01</td>
        <td>0.0183</td>
    </tr>
</tbody>
</table>
</div>

```

Cell 2 - code

```

# basic statistics
print(df.describe())

# correlation matrix
corr = df[['Mkt-RF', 'SMB', 'HML', 'RMW', 'CMA', 'ExcessRet']].corr()
print(corr)

# pairplot-like scattermatrix
pd.plotting.scatter_matrix(df[['Mkt-RF', 'SMB', 'HML', 'RMW', 'CMA']], figsize=(10,10));

```

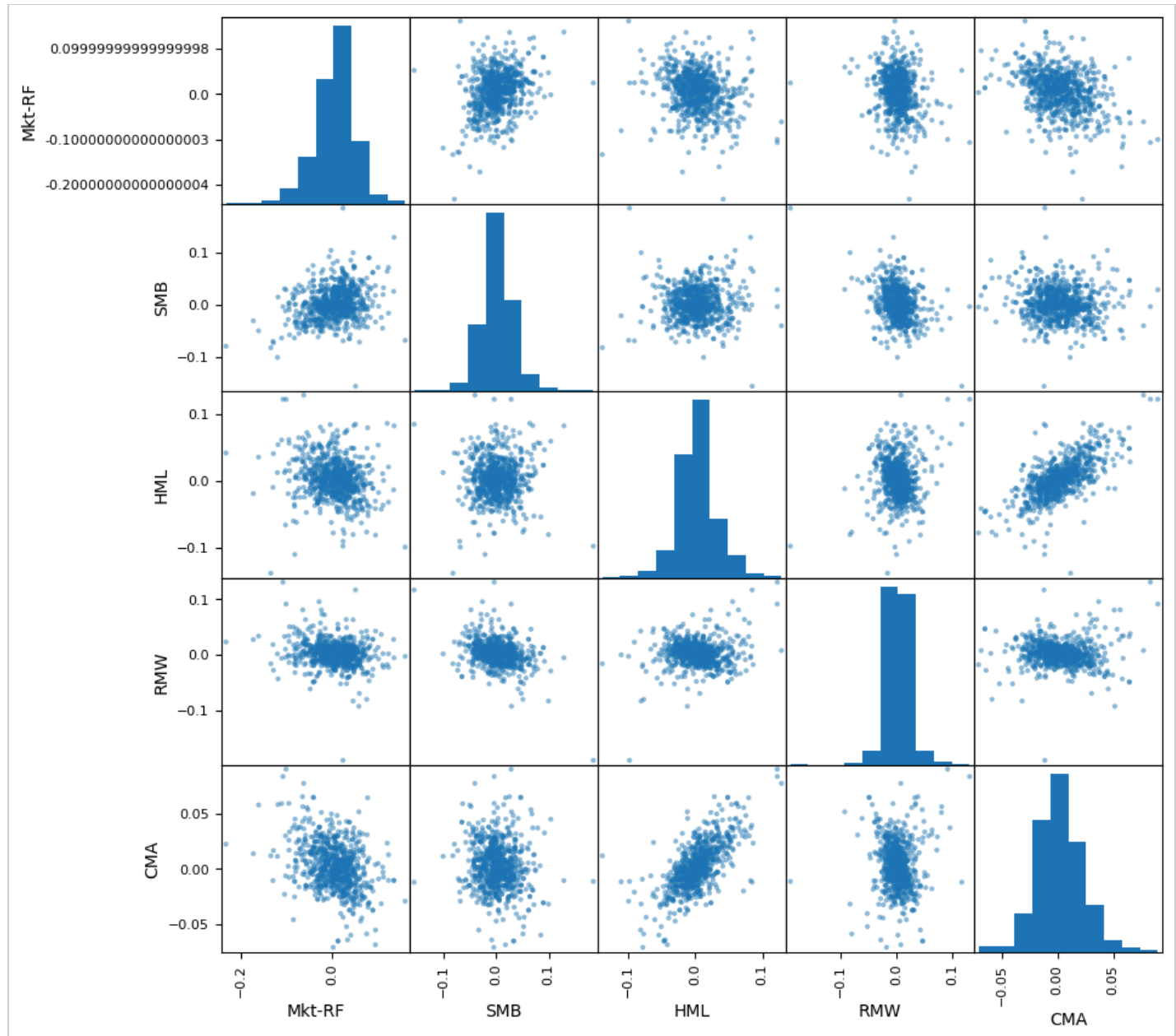
Output: stream

| | Mkt-RF | SMB | HML | RMW | CMA | \ |
|-----------|---------------------|--------------------|------------|------------|------------|-----------|
| count | 816.000000 | 816.000000 | 816.000000 | 816.000000 | 816.000000 | |
| mean | 0.005947 | 0.002523 | 0.002917 | 0.002351 | 0.002227 | |
| min | -0.231900 | -0.155400 | -0.138300 | -0.189500 | -0.070800 | |
| 25% | -0.019125 | -0.014825 | -0.013525 | -0.008500 | -0.010500 | |
| 50% | 0.009750 | 0.000800 | 0.002300 | 0.002150 | 0.000800 | |
| 75% | 0.033375 | 0.020725 | 0.017000 | 0.012900 | 0.014725 | |
| max | 0.161000 | 0.184600 | 0.128600 | 0.130500 | 0.090100 | |
| std | 0.043668 | 0.030247 | 0.028918 | 0.021773 | 0.020649 | |
| | | | | | | |
| | Date | | ExcessRet | | | |
| count | 816 | | 816.000000 | | | |
| mean | 1992-04-06 | 01:40:35.294117632 | 0.005951 | | | |
| min | 1963-07-01 00:00:00 | | -0.231900 | | | |
| 25% | 1974-10-24 06:00:00 | | -0.019125 | | | |
| 50% | 1991-10-16 12:00:00 | | 0.009750 | | | |
| 75% | 2008-10-08 18:00:00 | | 0.033375 | | | |
| max | 2025-10-01 00:00:00 | | 0.161000 | | | |
| std | NaN | | 0.043667 | | | |
| | | | | | | |
| | Mkt-RF | SMB | HML | RMW | CMA | ExcessRet |
| Mkt-RF | 1.000000 | 0.284770 | -0.198924 | -0.171902 | -0.351917 | 0.041123 |
| SMB | 0.284770 | 1.000000 | 0.012086 | -0.322559 | -0.101810 | 0.038870 |
| HML | -0.198924 | 0.012086 | 1.000000 | 0.060728 | 0.677292 | -0.066373 |
| RMW | -0.171902 | -0.322559 | 0.060728 | 1.000000 | -0.040079 | -0.045540 |
| CMA | -0.351917 | -0.101810 | 0.677292 | -0.040079 | 1.000000 | -0.076382 |
| ExcessRet | 0.041123 | 0.038870 | -0.066373 | -0.045540 | -0.076382 | 1.000000 |

Output: text/plain

<Figure size 1000x1000 with 25 Axes>

Output: image/png



Cell 3 - code

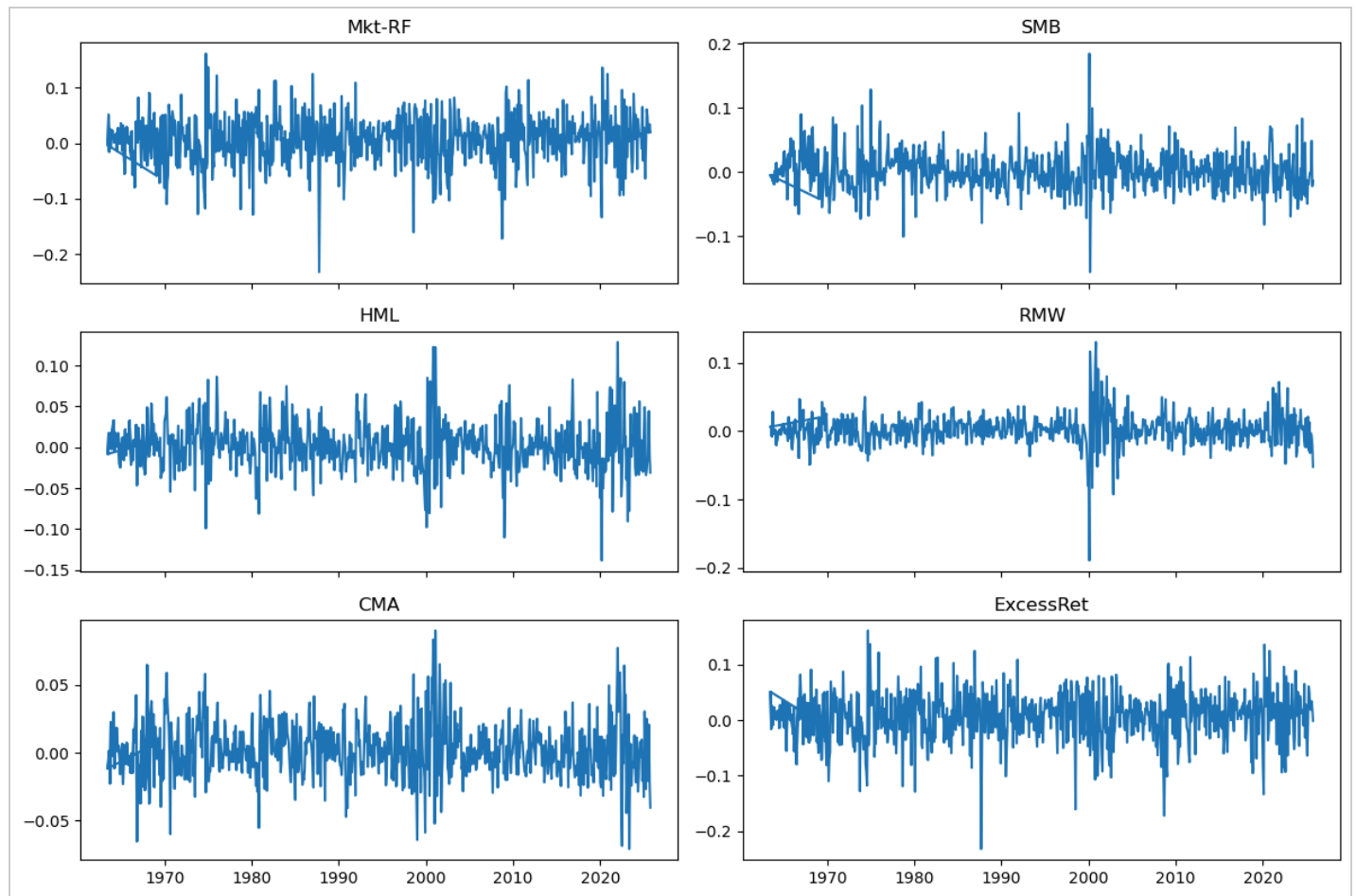
```
# time series plots of each factor
import matplotlib.pyplot as plt

fig, axes = plt.subplots(3,2,figsize=(12,8), sharex=True)
for ax, col in zip(axes.flatten(), ['Mkt-RF', 'SMB', 'HML', 'RMW', 'CMA', 'ExcessRet']):
    ax.plot(df['Date'], df[col])
    ax.set_title(col)
plt.tight_layout()
plt.show()
```

Output: text/plain

<Figure size 1200x800 with 6 Axes>

Output: image/png



Cell 4 - markdown

Additional ideas

- * Check for autocorrelation or seasonality.
- * Compute rolling means/volatility.
- * Visualize distributions (histograms, KDE).

notebooks\02_global_linear_baseline.ipynb

Cell 0 - markdown

Global Linear Baseline

Outline of fitting and evaluating the global linear regression model. Should show coefficient estimates, residual diagnostics, etc.

Cell 1 - code

```
import sys
sys.path.append('.')

import numpy as np
from src.data.loader import load_fama_french
from src.data.preprocessing import time_series_split
```

```
from src.models.global_linear import GlobalLinearRegression

# load and split
X,y,dates = load_fama_french('../Data/raw/F-F_Research_Data_5_Factors_2x3.csv')
X_train,X_test,y_train,y_test,_,_ = time_series_split(X,y,dates,0.8)

model = GlobalLinearRegression().fit(X_train,y_train)
print('Coefficients:', model.coef_)
print('Intercept:', model.intercept_)
print('Train MSE:', model.score(X_train,y_train))
print('Test MSE:', model.score(X_test,y_test))
```

Output: stream

```
Coefficients: [ 0.03033999  0.00429928  0.05081666 -0.13261183 -0.23861171]
Intercept: 0.005410191895800055
Train MSE: 0.0019059633202919697
Test MSE: 0.0018526128458236097
```

Cell 2 - code

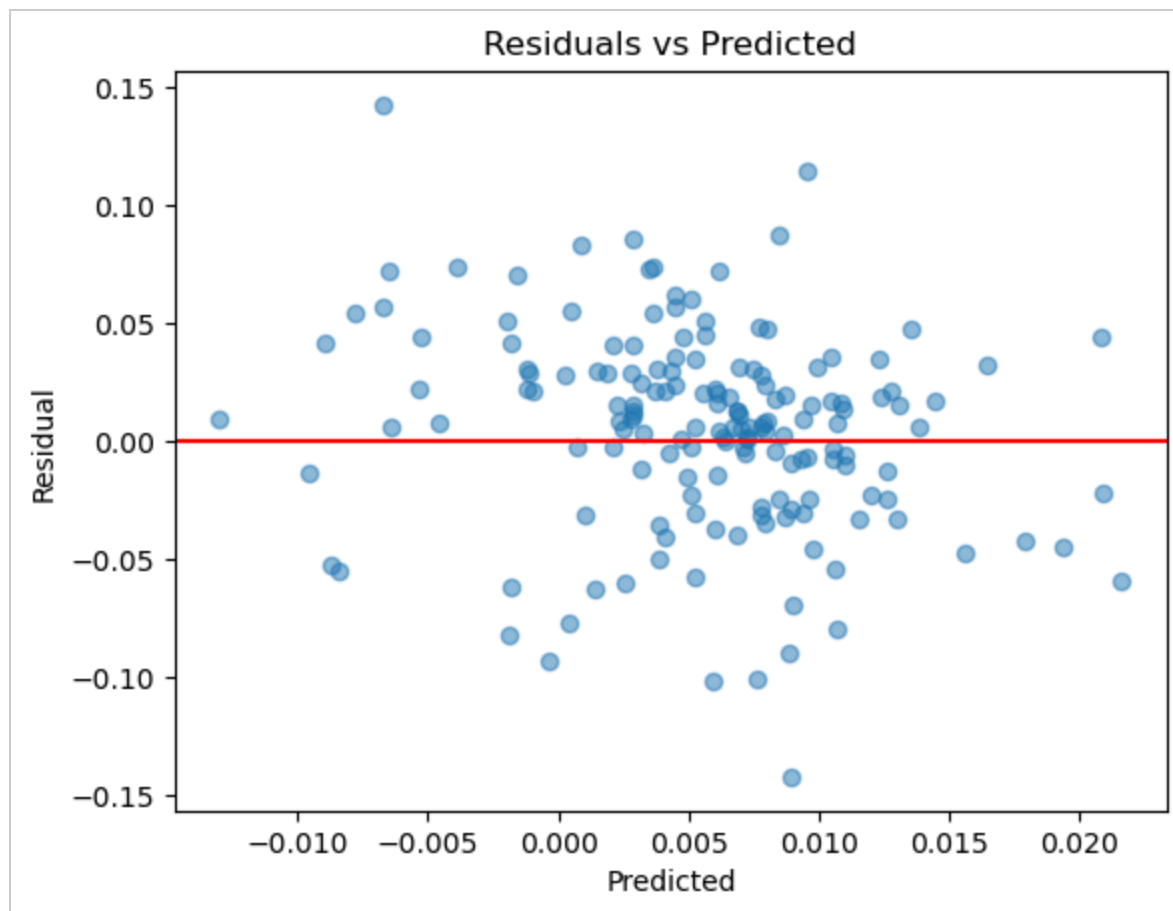
```
# residual plot
import matplotlib.pyplot as plt

y_pred = model.predict(X_test)
plt.scatter(y_pred, y_test - y_pred, alpha=0.5)
plt.axhline(0, color='red')
plt.xlabel('Predicted')
plt.ylabel('Residual')
plt.title('Residuals vs Predicted')
plt.show()
```

Output: text/plain

```
<Figure size 640x480 with 1 Axes>
```

Output: image/png



notebooks\02_lwlr_matrix_solution.ipynb

Cell 0 - markdown

LWLR Matrix Solution

Derivation and implementation details for solving the locally weighted linear regression in matrix form. Example usage goes here.

Cell 1 - code

```
import numpy as np
from src.models.lwlr import LocallyWeightedLinearRegression

# simple synthetic example
np.random.seed(0)
X = np.linspace(0,10,100).reshape(-1,1)
y = 2*X.squeeze() + np.random.normal(scale=1.0, size=X.shape[0])

lwlr = LocallyWeightedLinearRegression(bandwidth=1.0)
lwlr.fit(X,y)

# predict at a few points
x_new = np.array([[2.5],[5.0],[7.5]])
print('Predictions', lwlr.predict(x_new))
```

Output: stream

```
Predictions [ 5.10410846  9.66441935 14.88813446]
```

Cell 2 - markdown

```
# the math is implemented in the model; the above shows basic usage.
```

notebooks\03_lwlr_deterministic.ipynb

Cell 0 - markdown

```
# LWLR Deterministic Example
```

A notebook showcasing a deterministic run of the locally weighted linear regression (no Monte Carlo). Compare predictions to the global model.

Cell 1 - code

```
import sys
from pathlib import Path

cwd = Path.cwd()
root = cwd if (cwd / 'src').exists() else cwd.parent
sys.path.insert(0, str(root))
```

Cell 2 - code

```
import numpy as np
import matplotlib.pyplot as plt
from src.models.lwlr import LocallyWeightedLinearRegression

# generate synthetic data
np.random.seed(1)
X = np.linspace(0,10,100).reshape(-1,1)
y = np.sin(X).ravel() + np.random.normal(scale=0.2, size=100)

# fit lwlr and global for comparison
lwlr = LocallyWeightedLinearRegression(bandwidth=0.5).fit(X,y)

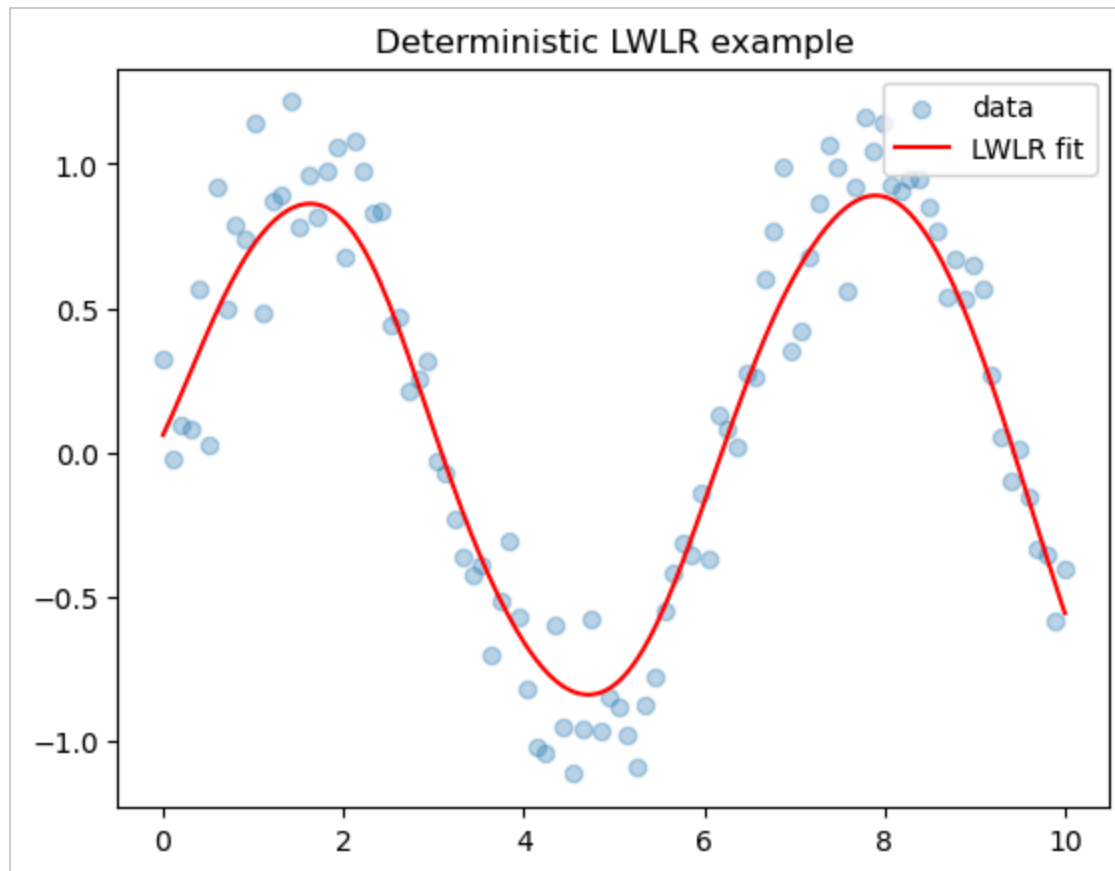
X_plot = np.linspace(0,10,200).reshape(-1,1)
y_pred = lwlr.predict(X_plot)

plt.scatter(X, y, alpha=0.3, label='data')
plt.plot(X_plot, y_pred, color='red', label='LWLR fit')
plt.legend()
plt.title('Deterministic LWLR example')
plt.show()
```

Output: text/plain

```
<Figure size 640x480 with 1 Axes>
```

Output: image/png



notebooks\03_sgd_monte_carlo.ipynb

Cell 0 - markdown

SGD Monte Carlo

Demonstrate training a linear model using stochastic gradient descent with Monte Carlo subsampling. Compare convergence and variance of estimates.

Cell 1 - code

```
import numpy as np
import matplotlib.pyplot as plt
from src.models.mc_linear import MonteCarloLinearRegression
from src.utils.sampling import subsample_indices
from src.data.loader import load_fama_french
from src.data.preprocessing import time_series_split

# load dataset and split
X,y,dates = load_fama_french('data/raw/F-F_Research_Data_5_Factors_2x3.csv')
X_train,X_test,y_train,y_test,_,_ = time_series_split(X,y,dates,0.8)

# show sampling helper usage
indices = subsample_indices(X_train.shape[0], 50, random_state=0)
print('example subsample indices', indices[:5])

mc = MonteCarloLinearRegression(n_simulations=100, subsample_size=50, random_state=0)
mc.fit(X_train, y_train)
print('Prediction sample', mc.predict(X_test[:5]))
```

```
# coefficient distribution
coefs = mc.coefs_
plt.hist(coefs[:,0], bins=20)
plt.title('Distribution of first coefficient across simulations')
plt.show()
```

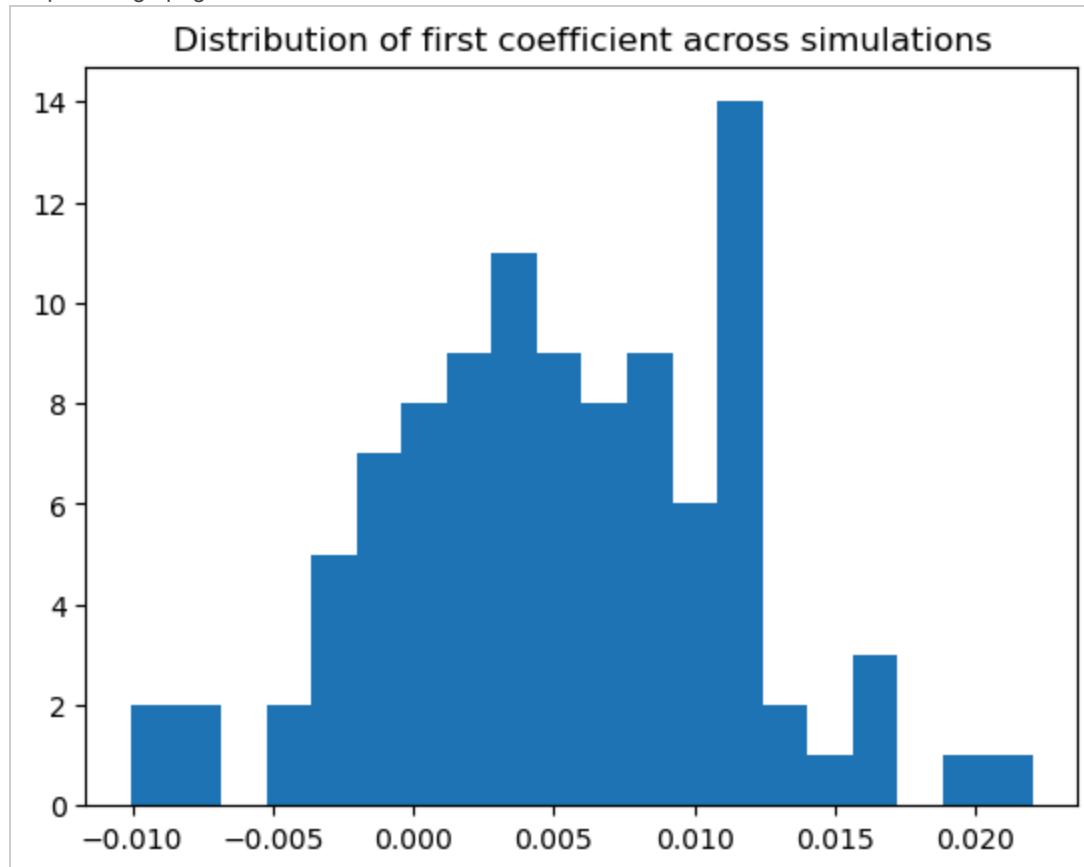
Output: stream

```
example subsample indices [ 45  18 535 512 450]
Prediction sample [ 0.00508517  0.00223274 -0.00332997  0.00599035  0.00406527]
```

Output: text/plain

<Figure size 640x480 with 1 Axes>

Output: image/png



notebooks\04_mc_subsampled_linear.ipynb

Cell 0 - markdown

```
# MC Subsampled Linear
```

Exploration of the Monte Carlo linear regression model using subsampled datasets. Visualize the distribution of coefficients across simulations.

Cell 1 - code

```
import numpy as np
import matplotlib.pyplot as plt
from src.models.mc_linear import MonteCarloLinearRegression
from src.data.loader import load_fama_french
from src.data.preprocessing import time_series_split

X,y,dates = load_fama_french('data/raw/F-F_Research_Data_5_Factors_2x3.csv')
X_train,X_test,y_train,y_test,_,_ = time_series_split(X,y,dates,0.8)

mc = MonteCarloLinearRegression(n_simulations=50, subsample_size=100, random_state=1)
mc.fit(X_train, y_train)

print('Mean coefficients', mc.coefs_.mean(axis=0))
plt.boxplot(mc.coefs_)
plt.title('Coefficient variability')
plt.show()
```

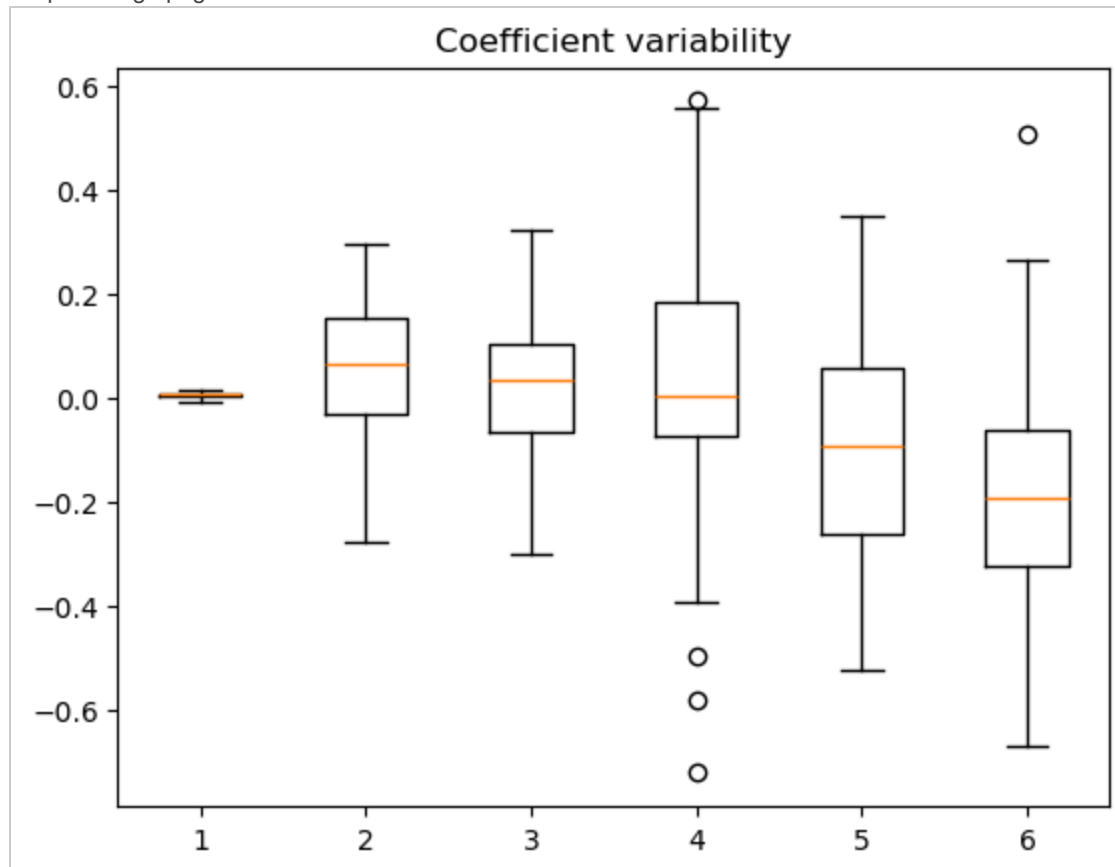
Output: stream

```
Mean coefficients [ 0.00571659  0.06001182  0.02185844  0.01558333 -0.09229529 -0.18128553]
```

Output: text/plain

```
<Figure size 640x480 with 1 Axes>
```

Output: image/png



notebooks\04_model_comparison.ipynb

Cell 0 - markdown

Model Comparison

This notebook uses the `ExperimentRunner` to train and evaluate all models (global linear, LWLR, MC, and MCLLR) on the Fama-French dataset. It demonstrates how to load configurations, run experiments, and inspect results.

Cell 1 - code

```
import pandas as pd

from src.Experiments.runner import ExperimentRunner
from src.data.preprocessing import time_series_split
```

Cell 2 - code

```
# specify configuration yaml (must exist in experiments/ folder)
config_path = "experiments/config_comparison.yaml"

runner = ExperimentRunner(config_path)
results_df = runner.run()

results_df
```

Output: stream

```
Loading dataset...
{'n_samples': 816, 'n_features': 5, 'features': ['Mkt-RF', 'SMB', 'HML', 'RMW', 'CMA'], 'date_range': '1963-07-01
00:00:00to2025-10-01 00:00:00', 'source_hash': '9cb42506'}
Splitting data...
Train: 652, Test: 164

Training global_linear...
<EvaluationResult global_linear | rmse: 0.0430, mae: 0.0331, mape: 1.6328, r2: -0.0568, explained_variance:
-0.0414, median_ae: 0.0278>

Training lwlr...
```

Output: stream

```
<EvaluationResult lwlr | rmse: 0.0430, mae: 0.0330, mape: 1.6329, r2: -0.0565, explained_variance: -0.0411,
median_ae: 0.0279>

Training mc_linear...
<EvaluationResult mc_linear | rmse: 0.0431, mae: 0.0331, mape: 1.6627, r2: -0.0589, explained_variance: -0.0419,
median_ae: 0.0279, coverage_95: 0.3841, avg_std: 0.0116, interval_width_95: 0.0454, gaussian_nll: 4.1627,
interval_score_95: 0.6502>

Training mcllr...
```

Output: stream

```
<EvaluationResult mcllr | rmse: 0.0431, mae: 0.0331, mape: 1.6266, r2: -0.0579, explained_variance: -0.0433,
median_ae: 0.0278, coverage_95: 0.4024, avg_std: 0.0114, interval_width_95: 0.0449, gaussian_nll: 5.0509,
interval_score_95: 0.6622>
```

Output: text/plain

	model	rmse	mae	mape	r2	explained_variance	\
0	global_linear	0.043042	0.033052	1.632833	-0.056799	-0.041407	
1	lwlr	0.043036	0.033048	1.632906	-0.056515	-0.041140	
2	mc_linear	0.043086	0.033131	1.662660	-0.058949	-0.041851	
3	mcllr	0.043064	0.033075	1.626648	-0.057868	-0.043293	
	median_ae	coverage_95	avg_std	interval_width_95	gaussian_nll	\	
0	0.027843	NaN	NaN	NaN	NaN	NaN	
1	0.027905	NaN	NaN	NaN	NaN	NaN	
2	0.027854	0.384146	0.011585	0.045412	4.162731		
3	0.027836	0.402439	0.011447	0.044870	5.050862		
	interval_score_95						
0	NaN						
1	NaN						
2	0.650221						
3	0.662238						

Output: text/html

```
<div>
<style scoped>
  .dataframe tbody tr th:only-of-type {
    vertical-align: middle;
  }

  .dataframe tbody tr th {
    vertical-align: top;
  }

  .dataframe thead th {
    text-align: right;
  }
</style>
<table border="1" class="dataframe">
  <thead>
    <tr style="text-align: right;">
      <th></th>
      <th>model</th>
      <th>rmse</th>
      <th>mae</th>
      <th>mape</th>
      <th>r2</th>
      <th>explained_variance</th>
      <th>median_ae</th>
      <th>coverage_95</th>
      <th>avg_std</th>
      <th>interval_width_95</th>
      <th>gaussian_nll</th>
      <th>interval_score_95</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th>0</th>
      <td>global_linear</td>
      <td>0.043042</td>
      <td>0.033052</td>
      <td>1.632833</td>
      <td>-0.056799</td>
      <td>-0.041407</td>
      <td>0.027843</td>
      <td>NaN</td>
      <td>NaN</td>
      <td>NaN</td>
      <td>NaN</td>
      <td>NaN</td>
    </tr>
```

```

        <td>NaN</td>
        <td>NaN</td>
        <td>NaN</td>
    </tr>
    <tr>
        <th>1</th>
        <td>lwlr</td>
        <td>0.043036</td>
        <td>0.033048</td>
        <td>1.632906</td>
        <td>-0.056515</td>
        <td>-0.041140</td>
        <td>0.027905</td>
        <td>NaN</td>
        <td>NaN</td>
        <td>NaN</td>
        <td>NaN</td>
        <td>NaN</td>
    </tr>
    <tr>
        <th>2</th>
        <td>mc_linear</td>
        <td>0.043086</td>
        <td>0.033131</td>
        <td>1.662660</td>
        <td>-0.058949</td>
        <td>-0.041851</td>
        <td>0.027854</td>
        <td>0.384146</td>
        <td>0.011585</td>
        <td>0.045412</td>
        <td>4.162731</td>
        <td>0.650221</td>
    </tr>
    <tr>
        <th>3</th>
        <td>mcllr</td>
        <td>0.043064</td>
        <td>0.033075</td>
        <td>1.626648</td>
        <td>-0.057868</td>
        <td>-0.043293</td>
        <td>0.027836</td>
        <td>0.402439</td>
        <td>0.011447</td>
        <td>0.044870</td>
        <td>5.050862</td>
        <td>0.662238</td>
    </tr>
</tbody>
</table>
</div>

```

Cell 3 - code

```

# simple bar chart of RMSE
import matplotlib.pyplot as plt

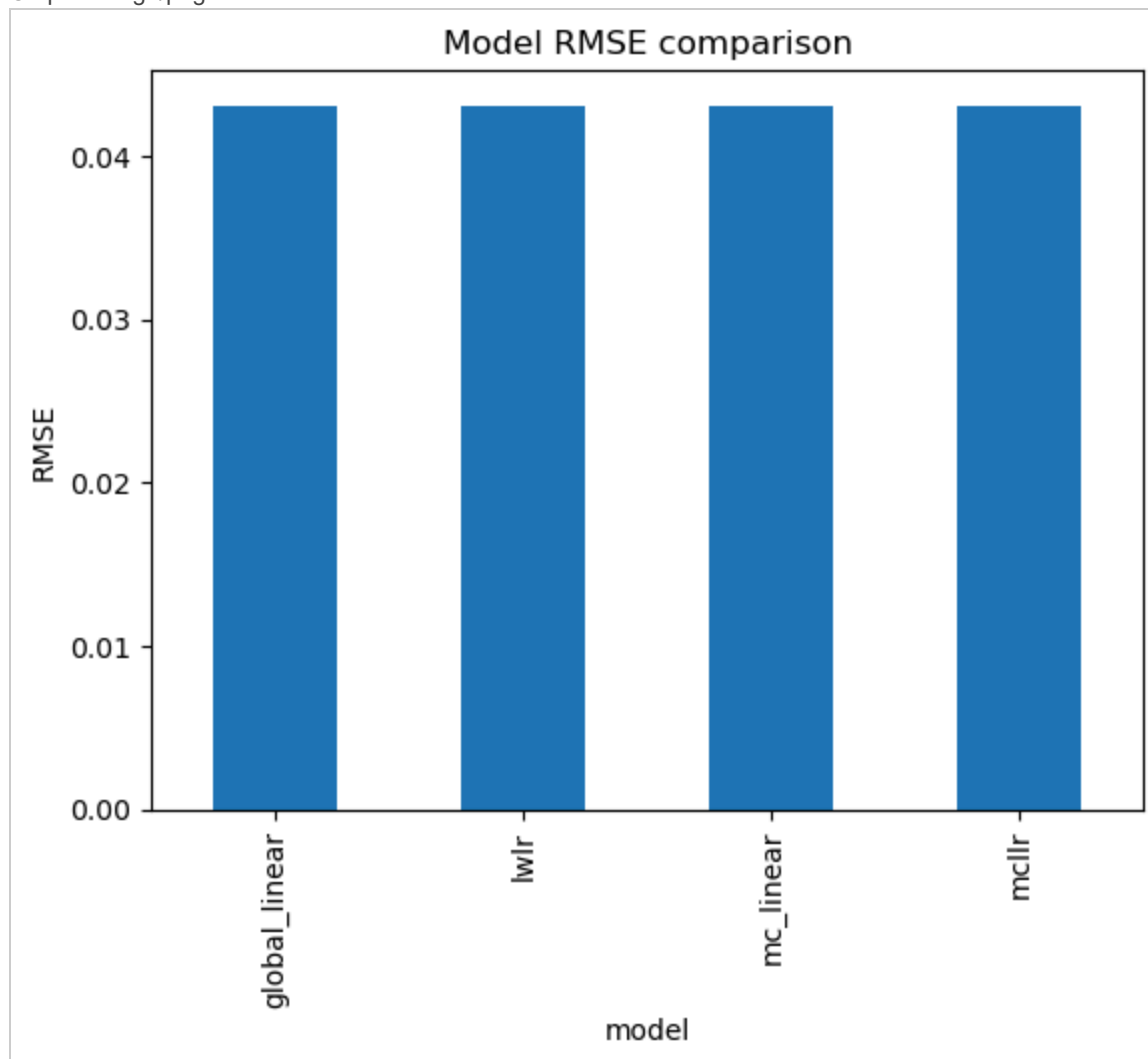
if not results_df.empty:
    results_df.plot.bar(x="model", y="rmse", legend=False)
    plt.ylabel("RMSE")
    plt.title("Model RMSE comparison")
    plt.show()

```

Output: text/plain

<Figure size 640x480 with 1 Axes>

Output: image/png



Cell 4 - markdown

Observations

After running the above, inspect the printed DataFrame and chart to compare model performance. Use `runner.save_results()` if you want to persist these metrics to disk.

notebooks\05_mcllr_proposed.ipynb

Cell 0 - markdown

Proposed MCLLR Model

Notebook for the proposed Monte Carlo Local Linear Regression (MCLLR) approach. Should contain algorithm description and a working example.

Cell 1 - code

```
import numpy as np
import matplotlib.pyplot as plt
from src.models.mcllr import MonteCarloLocalLinearRegression
from src.data.loader import load_fama_french
from src.data.preprocessing import time_series_split

X,y,dates = load_fama_french('data/raw/F-F_Research_Data_5_Factors_2x3.csv')
X_train,X_test,y_train,y_test,_,_ = time_series_split(X,y,dates,0.8)

model = MonteCarloLocalLinearRegression(bandwidth=1.0, n_simulations=50, subsample_size=100, random_state=0)
model.fit(X_train, y_train)

pred, std = model.predict(X_test[:5], return_std=True)
print('predictions', pred)
print('std', std)

# plot first test point uncertainty
plt.errorbar(range(len(y_test)), model.predict(X_test), yerr=model.predict(X_test, return_std=True)[1], fmt='.',
alpha=0.5)
plt.title('MCLLR predictions with uncertainty')
plt.show()
```

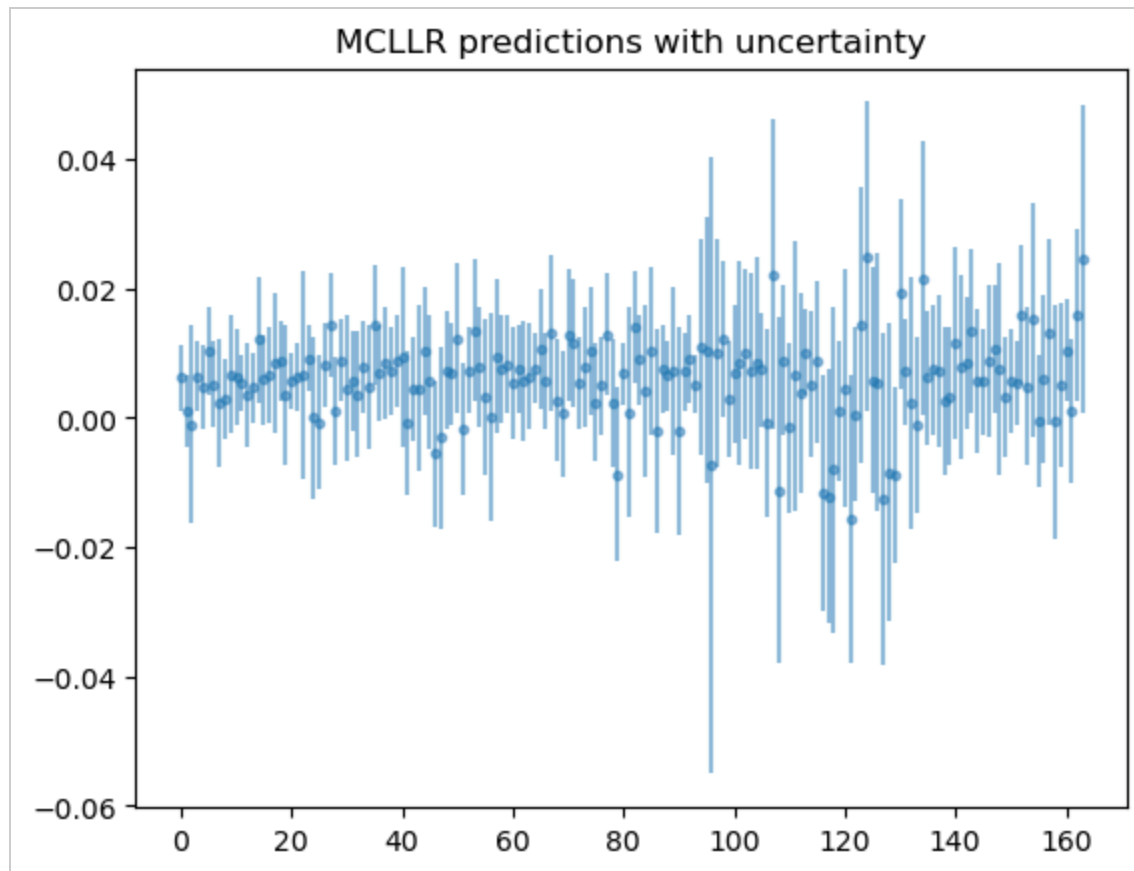
Output: stream

```
predictions [ 0.00694126  0.00214085 -0.00718407  0.00765221  0.00378184]
std [0.00518498 0.00666758 0.01343833 0.00500132 0.00658927]
```

Output: text/plain

<Figure size 640x480 with 1 Axes>

Output: image/png



notebooks\06_results_comparison.ipynb

Cell 0 - markdown

Results Comparison

Summarize and visualize results obtained from running experiments. This notebook can ingest output CSVs created by `runner.save\_results` and produce comparative plots.

Cell 1 - code

```
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path

# assume a results CSV exists from a previous run
df = pd.read_csv('results.csv') if Path('results.csv').exists() else pd.DataFrame()

if not df.empty:
    display(df.head())
    df.plot(x='model', y=['rmse', 'mae'], kind='bar')
    plt.title('Metrics comparison');
    plt.show()
else:
    print('Run an experiment and save results to results.csv to view them here.')
```

Output: stream

Run an experiment and save results to results.csv to view them here.